

位；对位进行换位。它可以通过软件或硬件的方式实现换位，但硬件实现比较快。与S-盒类似，P-盒通常也是没有密钥的。在P-盒中，我们通常有3种类型的置换：直接置换（straight permutation）、扩充置换（expansion permutation）、以及压缩置换（compression permutation）。如图30.12所示。

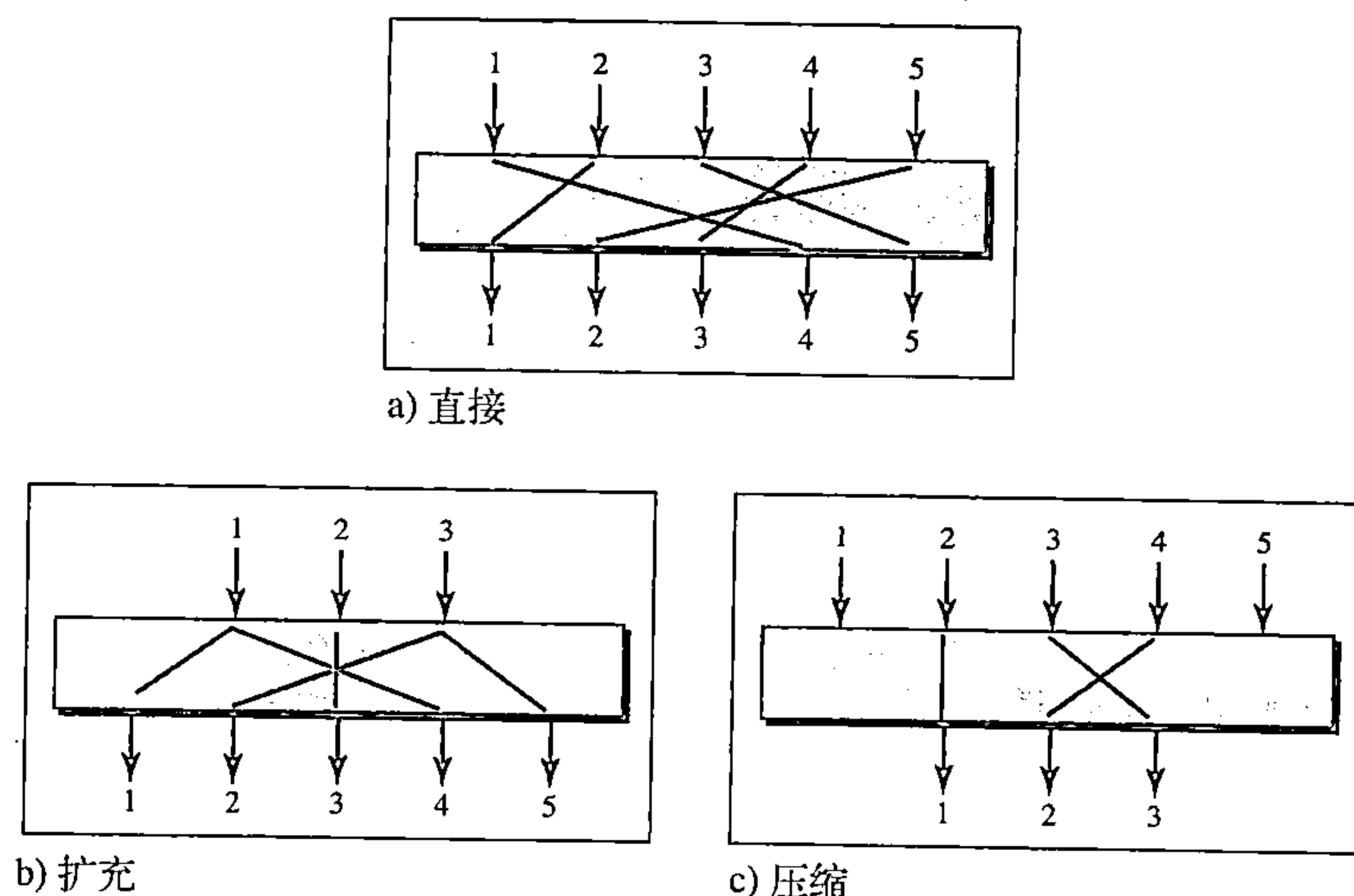


图30.12 P-盒：直接、扩充及压缩置换

在直接置换算法或直接置换的P-盒中，输入与输出的长度是相同的。换句话说，输入的长度是 N ，则输出的长度也是 N 。在扩充置换算法中，输出端口的数量要大于输入端口；在压缩置换算法中，输出端口的数量要小于输入端口。

30.2.3 现代迭代加密

如今的加密算法被称为迭代加密（round ciphers），因为它包含了多次迭代（rounds），每一次迭代都是由我们前面介绍的简单加密算法组成的复杂加密算法。在每次迭代中使用的密钥是一个子集或者是由称为迭代密钥的通用密钥衍生而成。如果加密算法有 N 次迭代，则密钥生成器产生 N 个密钥 K_1 、 K_2 、……， K_N ， K_1 在第1次迭代中使用， K_2 在第2次迭代中使用，以此类推。

在本节中，我们介绍两种现代对称密钥加密算法：DES与AES。这些加密算法被认为是分组加密（block ciphers），因为它们把明文分成多个分组，然后使用相同的密钥对各分组进行加密与解密。到现在为止，DES已经成为事实上的标准；AES是现在正式的标准。

数据加密标准（DES）

复杂的分组加密算法的一个例子是数据加密标准（Data Encryption Standard, DES）。DES是由IBM设计并被美国政府接受作为非军事和非机密用途的加密标准。该算法采用64位的密钥对64位的明文进行加密，如图30.13所示。

DES有2个换位模块（P-盒）和16个复杂的迭代加密模块（它们一直在重复）。尽管16个迭代算法概念上是相同的，但每个模块使用的是由源密钥衍生的不同密钥。

在最初与最终的置换加密是无密钥的、相互之间进行反转的直接置换。这个置换加密根据预定义的值对64位的输入进行置换。

DES的每次迭代都是一个复杂的迭代加密算法。如图30.14所示。需要注意的是：加密迭代算法的结构与解密的结构是不相同的。

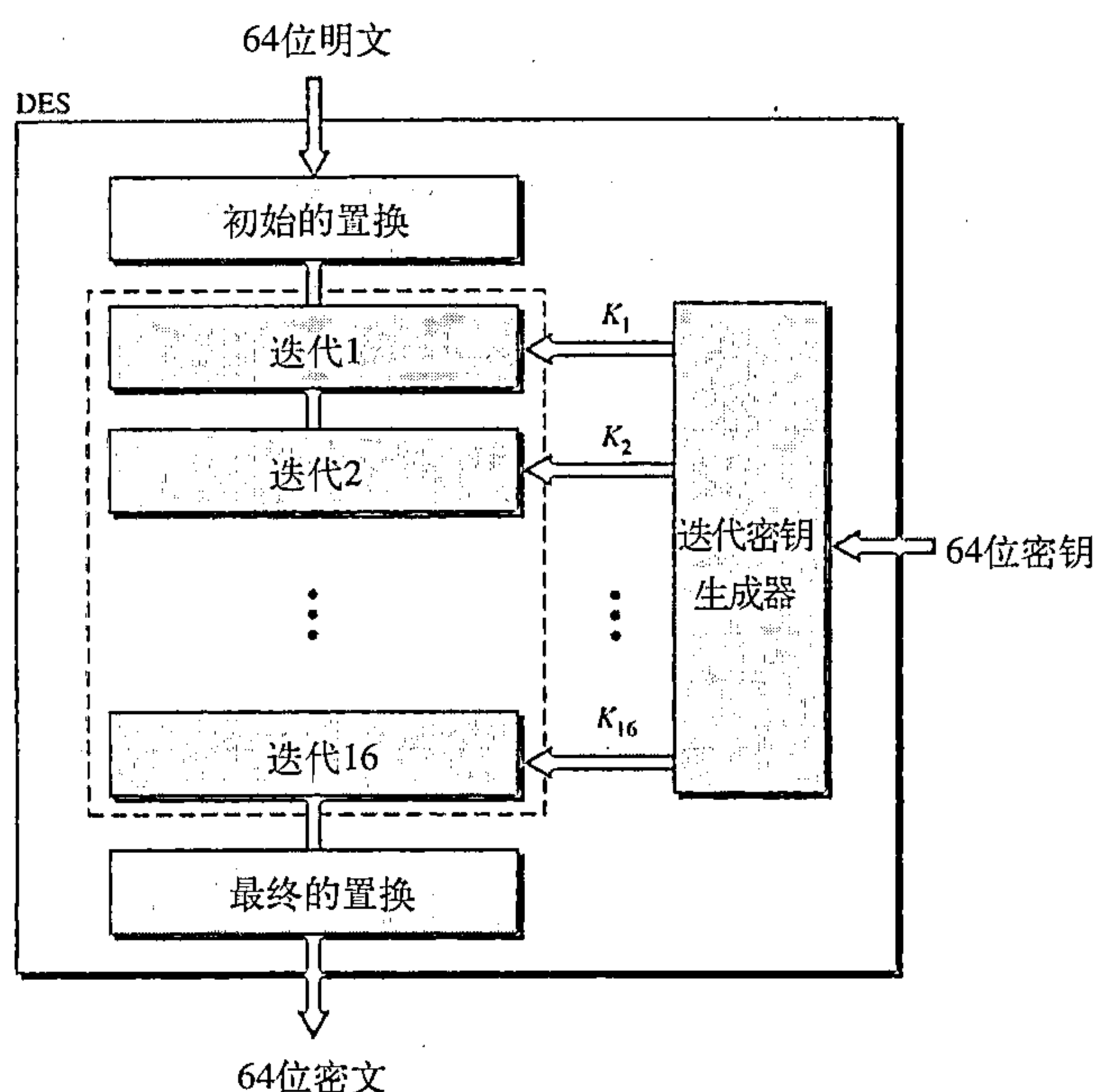


图30.13 数据加密标准 (DES)

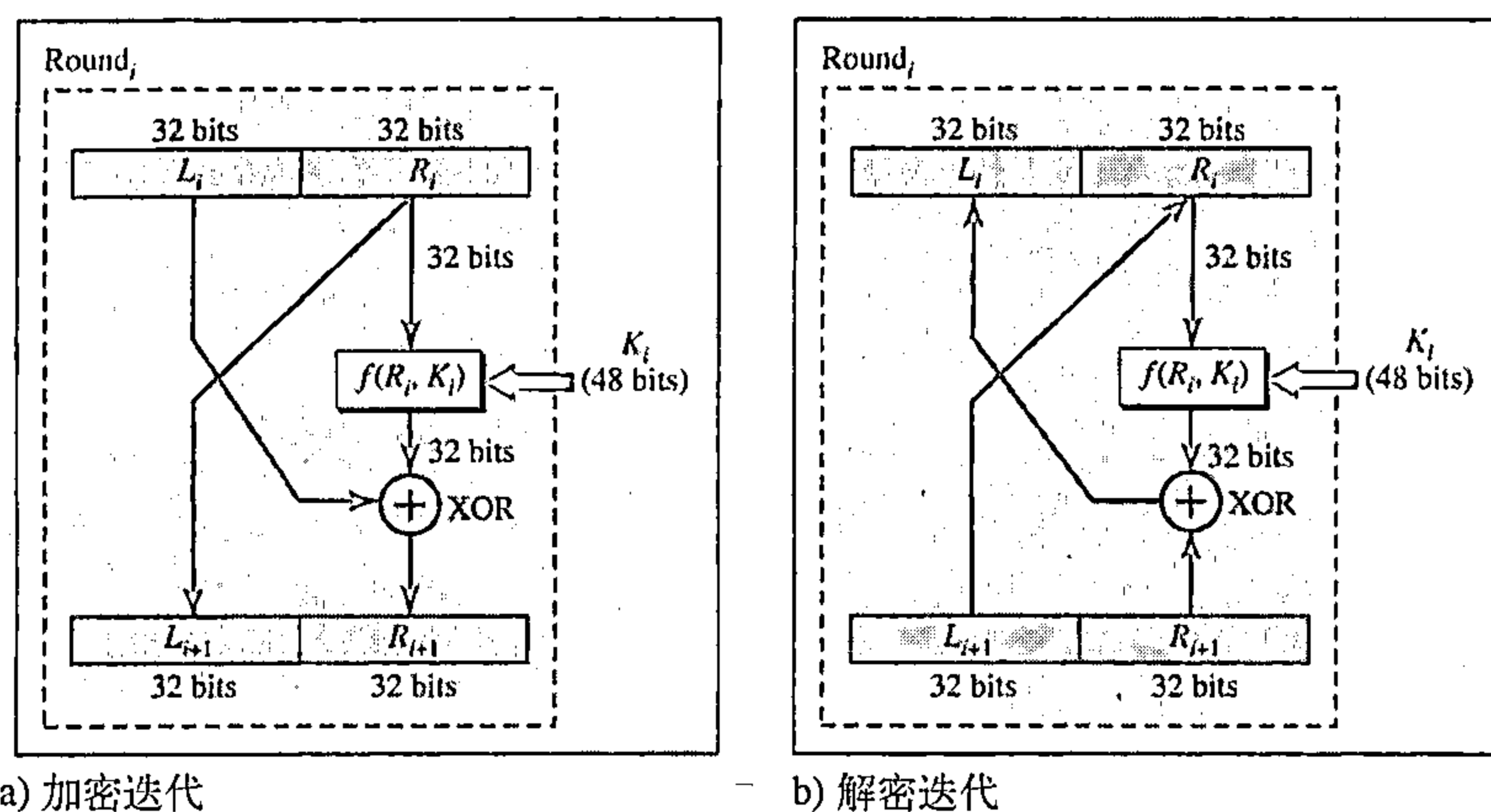


图30.14 在DES加密中的一次迭代

DES 函数

DES的核心是DES函数（DES function）。DES函数采用了48位的密钥，根据最右面32位 R_i 产生一个32位的输出。这个函数由4个操作组成：XOR、扩充置换、一组S-盒、以及一个直接置换。如图30.15所示。

三重DES

对DES的批评是密钥太短了。为了加长密钥，提出并实现了三重DES或者3DES。它使用三个DES模块，如图30.16所示。注意，加密模块使用的是DES加密-解密-加密的组合，而解密模块使用的是解密-加密-解密的组合。两个不同版本的3DES分别使用：2

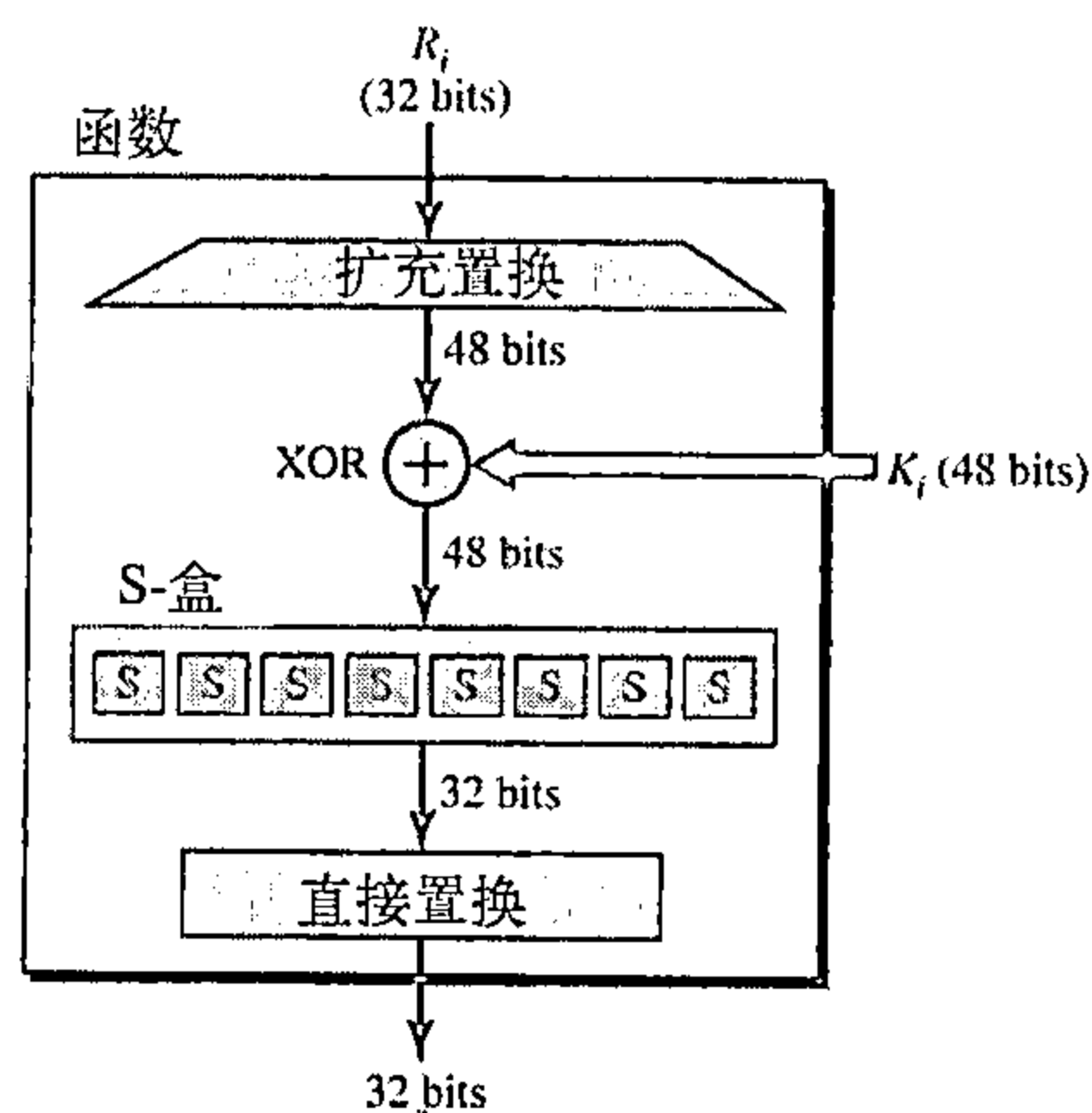


图30.15 DES函数

个密钥的3DES和3个密钥的3DES。为了使密钥长度达到112位，并且同时避免DES遭受类似中间人（man-in-middle）的攻击，设计出使用2个密钥的3DES。在这个版本中，第一个和第三个密钥是相同的（密钥1=密钥3）；这有一个优势：通过一个单一的DES模块加密的文本可以用一个新的3DES模块解密，我们只是设置所有的密钥等于密钥1。许多算法使用了3个密钥的3DES加密算法，这使密钥的长度增加到168位。

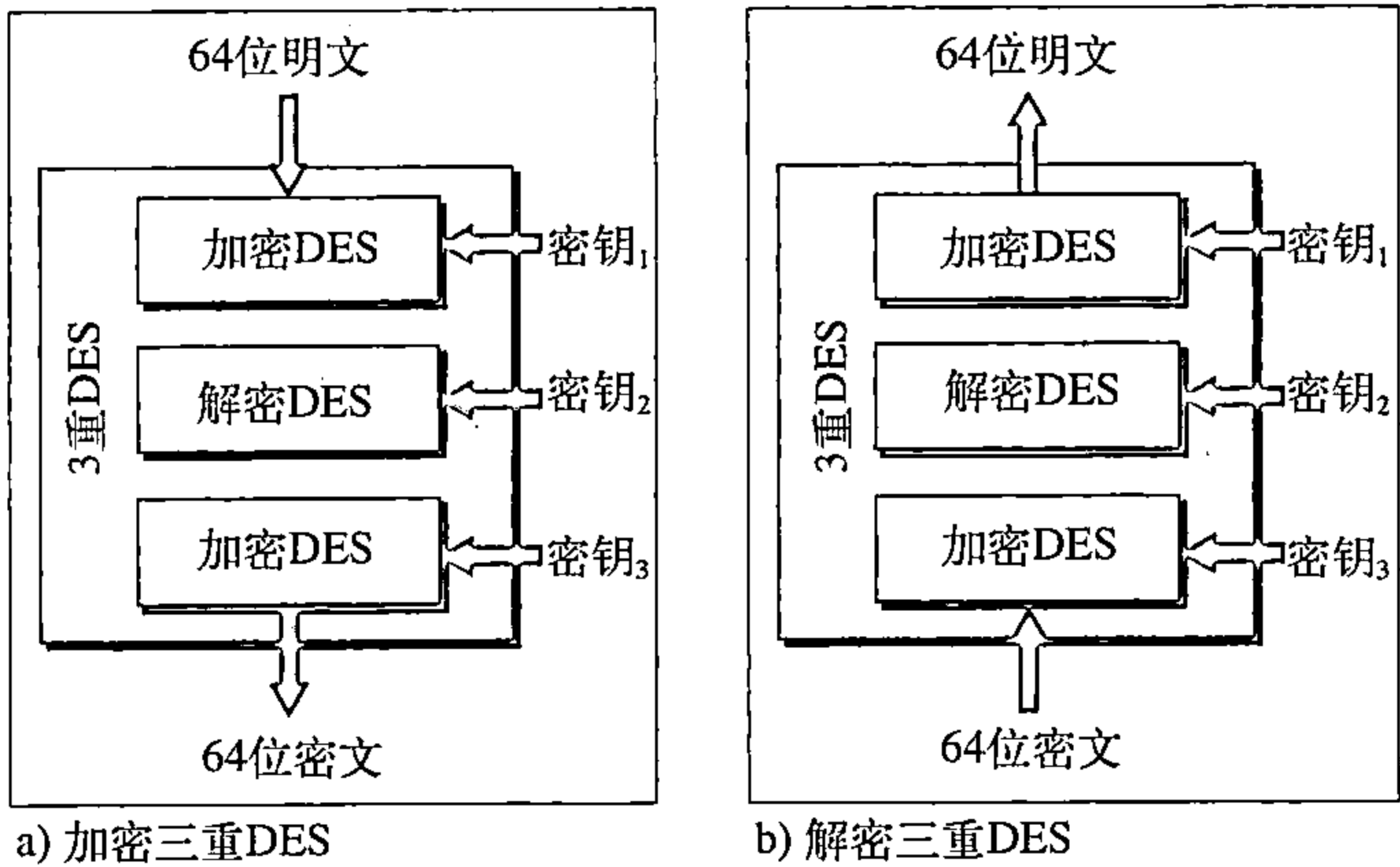


图30.16 三重DES

高级加密标准

因为DES的密钥太短，高级加密标准（Advanced Encryption Standard，AES）应运而生。尽管三重DES（3DES）增加了密钥的长度，但处理的过程太慢。国家标准与技术协会（National Institute of Standards and Technology，NIST）选择Rijndael算法（Rijndael algorithm，这算法是根据两个比利时发明人Vincent Rijmen与Joan Daemen命名的）作为AES的基础理论。AES是一个复杂的迭代加密算法，它有3种密钥长度：128位、192位及256位。表30.1所示为数据块、迭代次数及密钥长度之间的相互关系。

表30.1 AES配置

数据块的长度	迭代次数	密钥长度
128位	10	128位
	12	192位
	14	256位

AES有三种不同的配置，具有不同的迭代次数和密钥长度。

在本节中，我们仅讨论10次迭代、128位密钥的配置。其他配置的结构和操作是类似的；它们的区别就在于密钥的生成方式。

通用的结构如图30.17所示。在初始的异或操作后面是10次迭代加密，最后一次迭代与前面的迭代稍微有点区别，它缺少一步操作。

尽管10次反复的模块几乎是相同的，但每一个模块使用的是从原始密钥衍生出来的不同的密钥。

每次迭代的结构 AES的每次迭代，除了最后一个之外，都是由4个可倒转的操作组成的加密算法。最后一次迭代仅有3个操作。图30.18所示为每次迭代中操作的流程图。每次迭代中，4个操作中的每个操作都使用了一个复杂的加密算法，这个课题超出了本书的范围。

其他的加密算法

在过去的二十年里，设计并使用了許多其他的对称分组加密算法。大部分的这些加密算法有与我们本章中讨论的两种加密算法（DES和AES）相似的特征。通常情况下，它们之间

的区别在于分组和密钥的长度、迭代的次数及所用的函数，但基本原理是一致的。为了在这些加密算法的细节方面不给读者带来负担，我们对每一个加密算法给出了简单的描述。

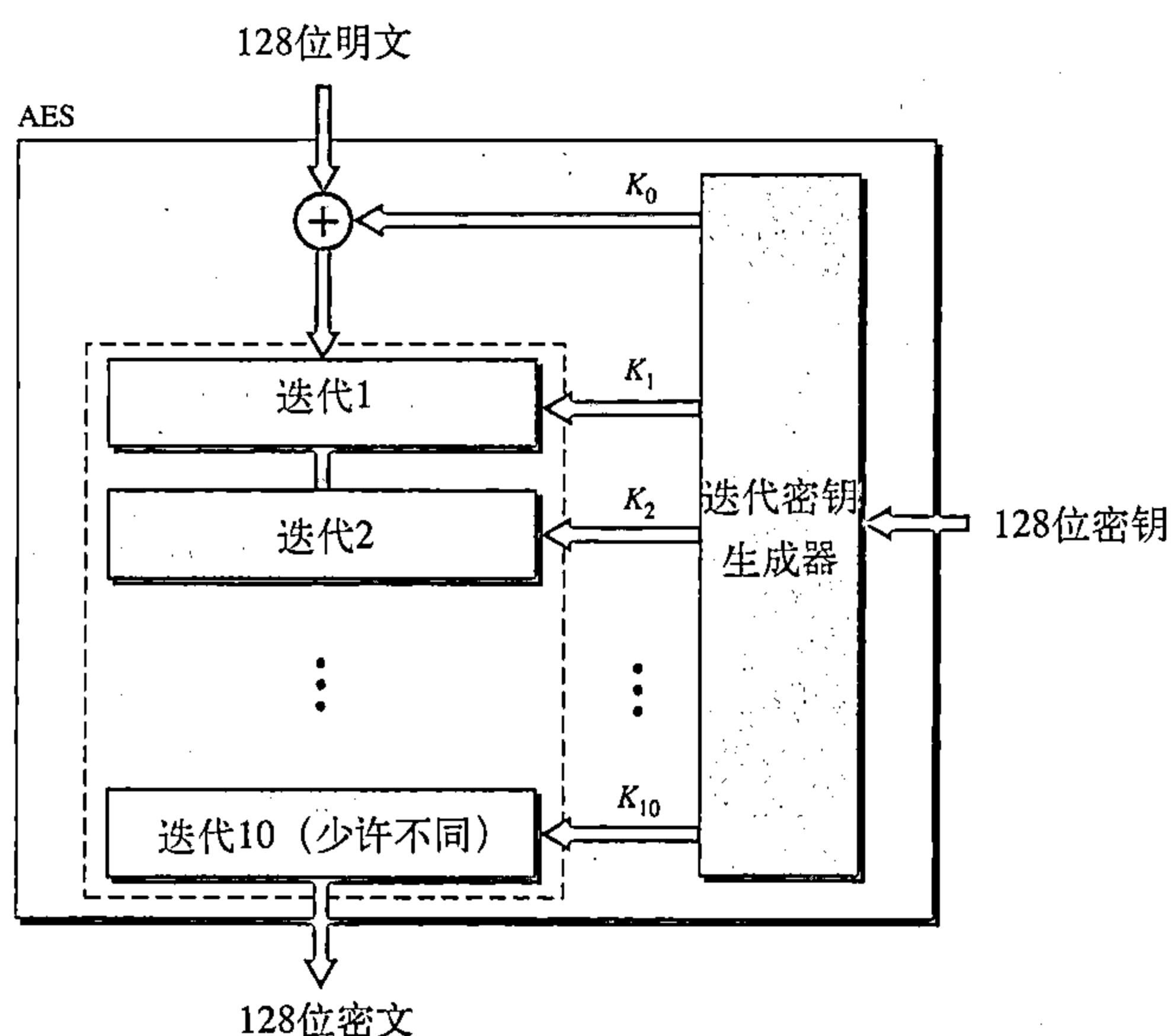


图30.17 高级加密标准

IEDA 国际数据加密算法 (IEDA) 是由 XuejiaLai 和 James Massey 发明的。分组长度为 64 位、密钥长度为 128 位，可以通过硬件和软件的方法实现。

Blowfish Blowfish 算法是由 Bruce Schneier 发明。分组长度为 64 位，密钥长度介于 32 位与 448 位之间。

CAST-128 CAST-128 算法是由 Carlisle Adams 与 Stafford Tavares 共同发明的。它是由 16 次迭代的 Feistel 加密算法组成的。分组长度为 64 位、密钥长度为 128 位。

RC5 RC5 算法是由 Ron Rivest 发明。这是由不同的分组长度、密钥长度、迭代次数组成的加密算法组合。

30.2.4 运算模式

运算模式 (mode of operation) 是采用诸如我们前面讨论的 DES、AES 之类的现代分组加密算法的技术 (如图 30.19)。

电子代码本

电子代码本模式 (electronic code block mode, ECB) 是纯分组加密技术。明文被划分成 N 位长度的分组；密文也是由 N 位长度的分组组成。值 N 依赖于使用的加密算法的类型。方法如图 30.20 所示。

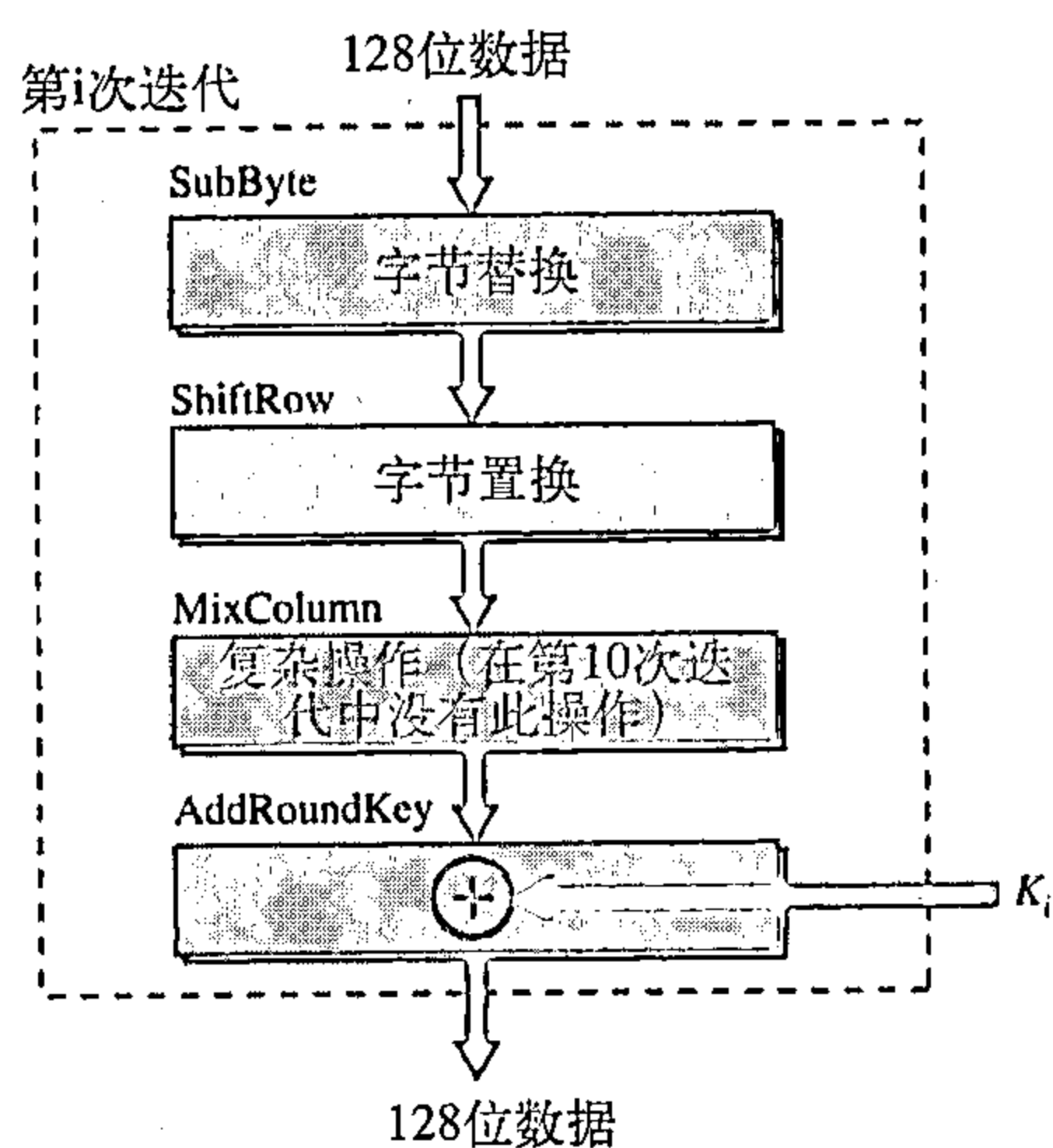


图30.18 每次迭代的结构

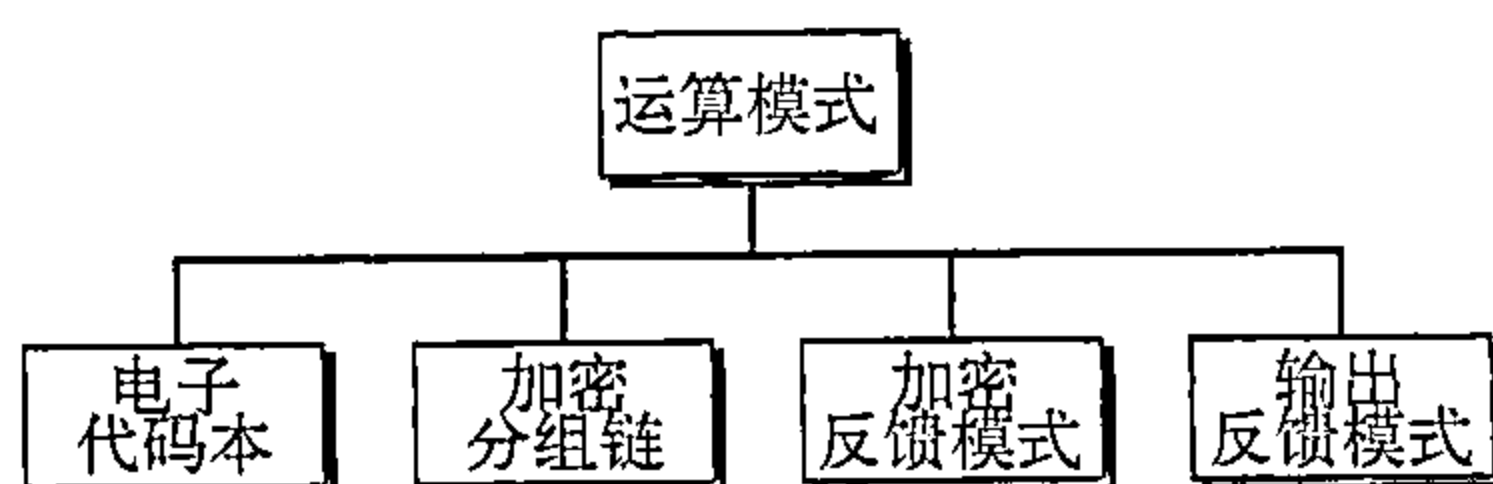


图30.19 分组加密算法的运算模式

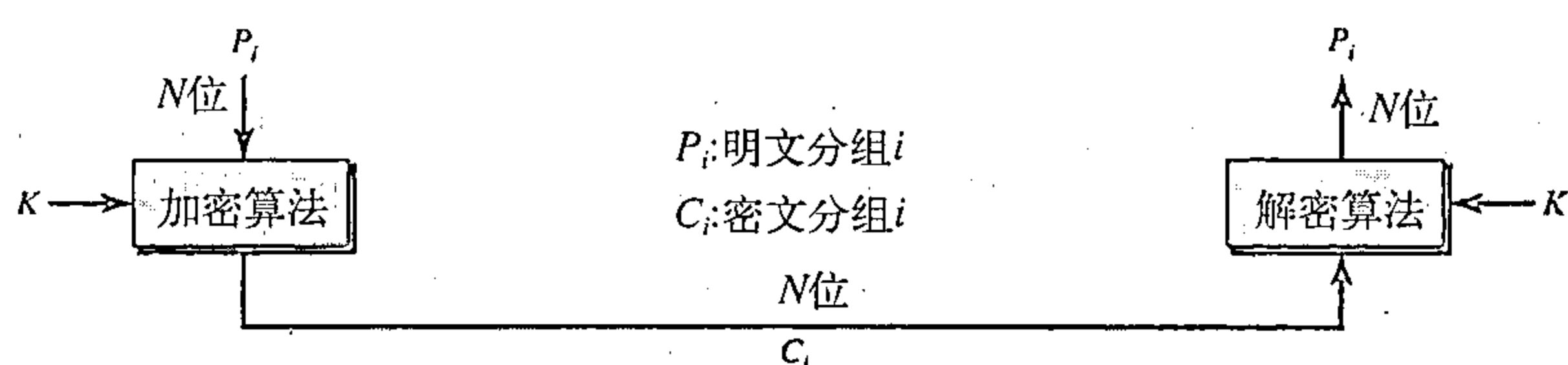


图30.20 电子代码本模式

这个模式有4个特征：

1. 因为密钥、以及加密/解密的算法是相同的，因此在明文中相同的一些分组，在密文中对应的分组也是相同的。例如，在明文中分组1、5、9是相同的，则密文中分组1、5、9也是相同的。这就会导致安全隐患；敌人会因为对应的密文分组相同，而猜测推断明文分组是相同的。
2. 如果我们对明文分组重新排序，则密文分组也会重新排序。
3. 每一个分组之间都是独立的。每一个分组都可以独立地进行加密与解密。其中一个分组在加密与解密时发生问题不影响其他的分组。
4. 在一个分组中发生错误不会传播到其他的分组。在传输过程中，如果一个或多个位被破坏，在加密后，这只影响这些位所对应的明文，其他的明文并没有受到影响。如果通道没有噪声，那么这将成为一个真正的优点。

加密分组链

加密分组链模式 (cipher block chaining mode, CBC) 通过将先前的密文分组应用到现在的分组加密中，以此来减少ECB模式中发生的问题。如果现在的分组是 i ，前面的密文分组 C_{i-1} 则应用到分组 i 的加密中。换句话说，当一个分组完全被加密后，分组被发送，但它的一份拷贝仍保存在寄存器（数据存放的地方）中，用于对下一个分组的加密。读者可能对最初的分组有些疑惑，在第一个分组前面是没有密文分组的。在这种情况下，使用了称为初始化向量 (initiation vector, IV) 的假冒分组。发送方与接收方都认可这个预先定义的IV。换句话说，这个IV替代了不存在的 C_0 。CBC模式，如图30.21所示。

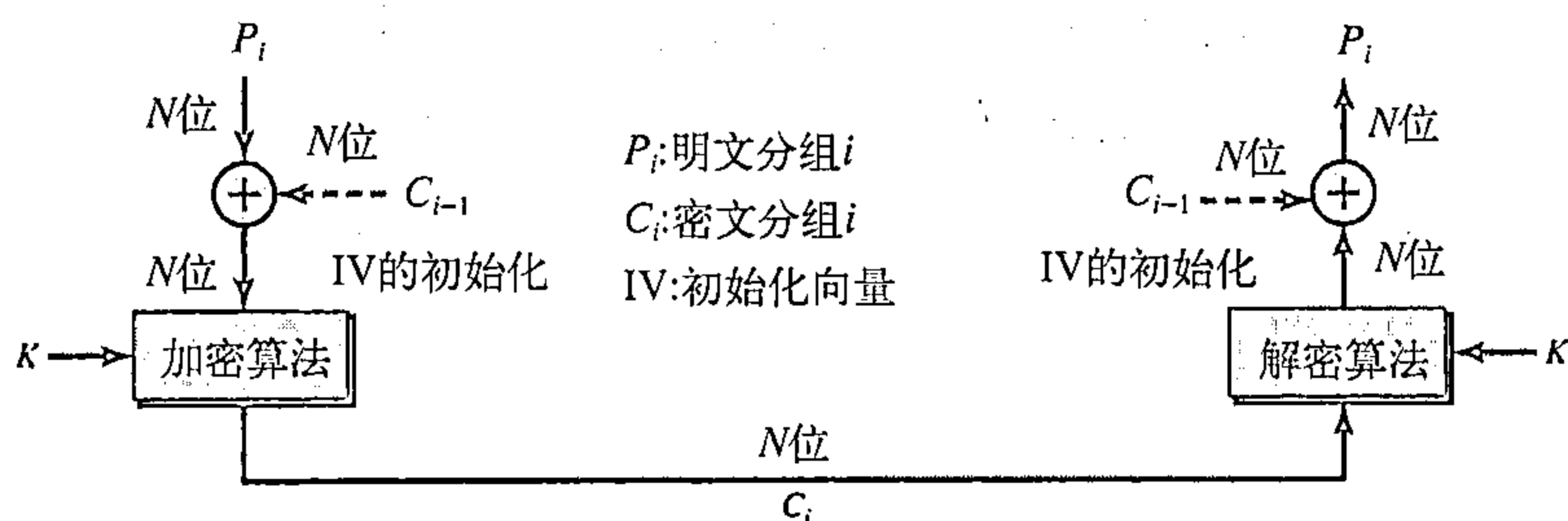


图30.21 加密分组链模式

下面是加密分组链模式的一些特征：

1. 即使密钥和加密/解密算法都相同，在明文中的相同分组对应的密文分组也不相同。例如，在明文中，分组1、5、9相同，但在密文中的分组1、5、9并不相同。敌人就不能从密文中猜测出那两个分组是相同的。
2. 分组之间是相互依赖的。每一个分组的加密与解密都是基于前一个分组。当一个分组在加密与解密中发生问题会影响到其他的分组。
3. 在一个分组中发生错误会传播到其他的分组。如果在传输过程中，一个或多个位被破坏，那么在加密后，它会影响到所有的在明文中下一分组的所有位。

加密反馈

加密反馈模式 (cipher feedback mode, CFB) 是专门为这个情况而设计的：我们需要发送或接收 r 位的数据，这里 r 是一个数值，不同于下面要采用的加密算法的分组长度。 r 的值可以是1、4、8或其他任意的位。因为所有的分组加密算法都是一次性地对数据分组进行加密，现在的问题就是如何对仅仅只有 r 位的数据进行加密。解决的方法是：用加密算法对分组的位进行加密，仅仅使用前面的 r 位作为新的密钥（密钥流），对 r 位的用户数据进行加密。如图30.22所示。

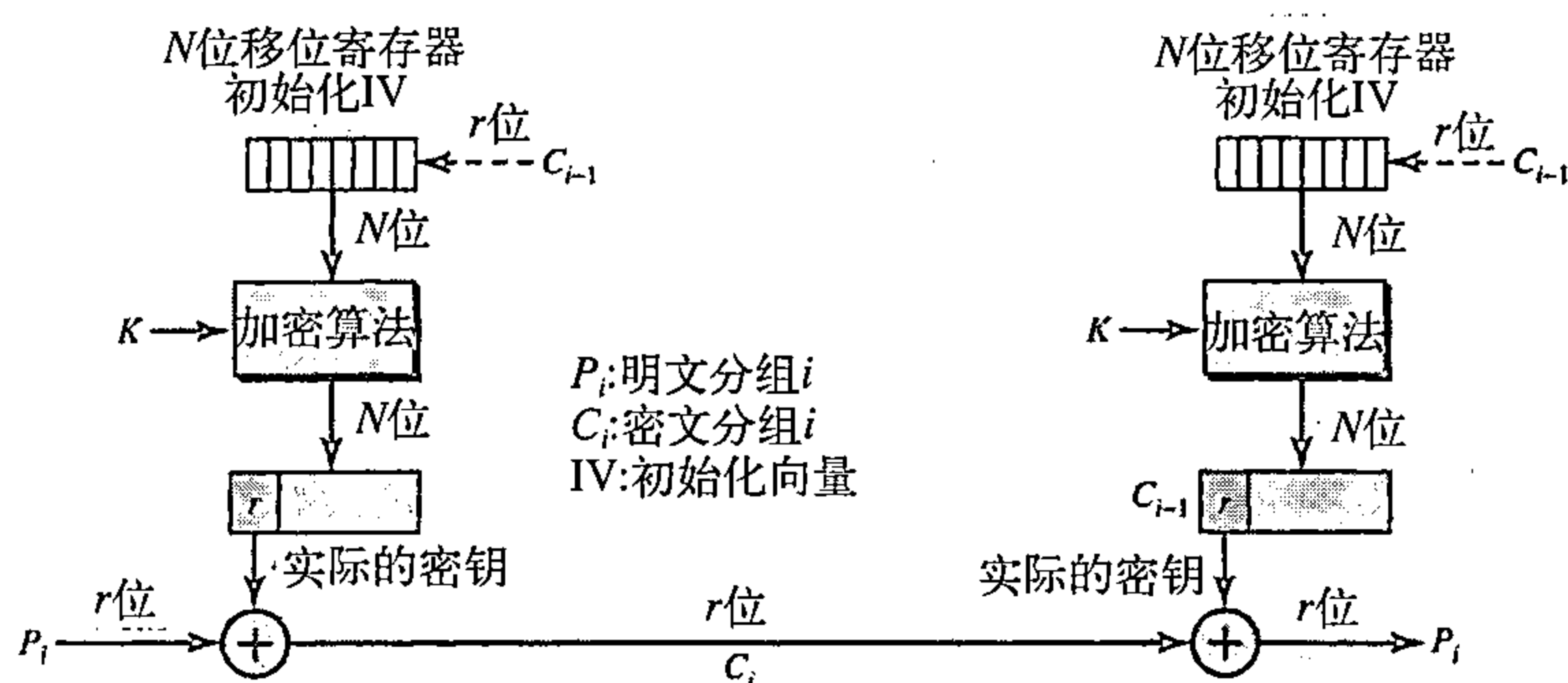


图30.22 加密反馈模式

下面是加密反馈模式的一些特征：

1. 对相同的明文进行加密，如果改变初始化向量，则密文是不相同的。
2. 密文 C_i 依赖于 P_i 及先前的密文分组。
3. 在密文分组中一位或多位发生错误，则会影响到后面的密文分组。

输出反馈

输出反馈模式 (output feedback mode, OFB) 与CFB非常相似，只有一个不同点。在密文中的每一位与前面的1位或多位都无关，这就避免了错误的传播。如果在传输的过程中发生了错误，它不会影响到后面的位。注意：在CFB中，发送方与接收方都使用了加密算法。而在OFB中，只使用分组加密算法如DES或AES创建密钥流。创建下一个位流的反馈来源于前面的密钥流，而不是密文。密文并没有参加密钥流的创建。图30.23说明了OFB模式。

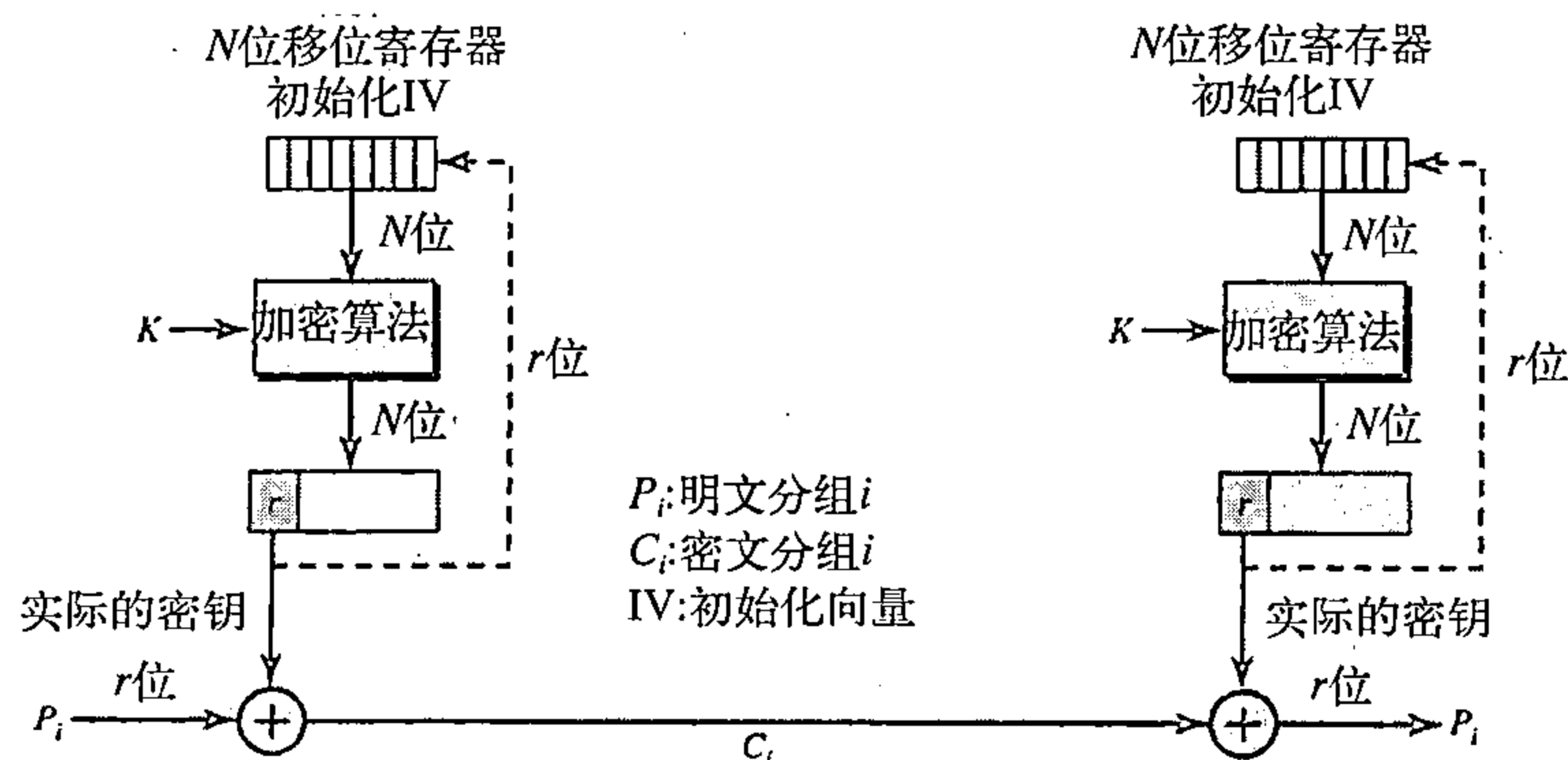


图30.23 输出反馈模式

下面是加密反馈模式的一些特征：

1. 对相同的明文进行加密，如果改变初始化向量，则密文是不相同的。
2. 密文 C_i 依赖于明文 P_i 。

3. 在密文分组中，一位或多位发生错误，并不会影响到后面的密文分组。

30.3 非对称密钥密码学

在前面几节中，我们讨论了对称密钥密码学。在本节中，我们介绍非对称密钥（公共密钥密码学）。正如我们前面所提到的，一个非对称密钥（或者公共密钥）加密算法使用2个密钥：一个是公钥，一个是私钥。我们讨论两种加密算法：RSA和Diffie-Hellman。

30.3.1 RSA

最常见的公共密钥加密算法是RSA，RSA是根据它的发明者Rivest, Shamir和Adleman (RSA) 而命名的。它使用两个数字： e 和 d ，作为公钥与密钥。如图30.24所示。

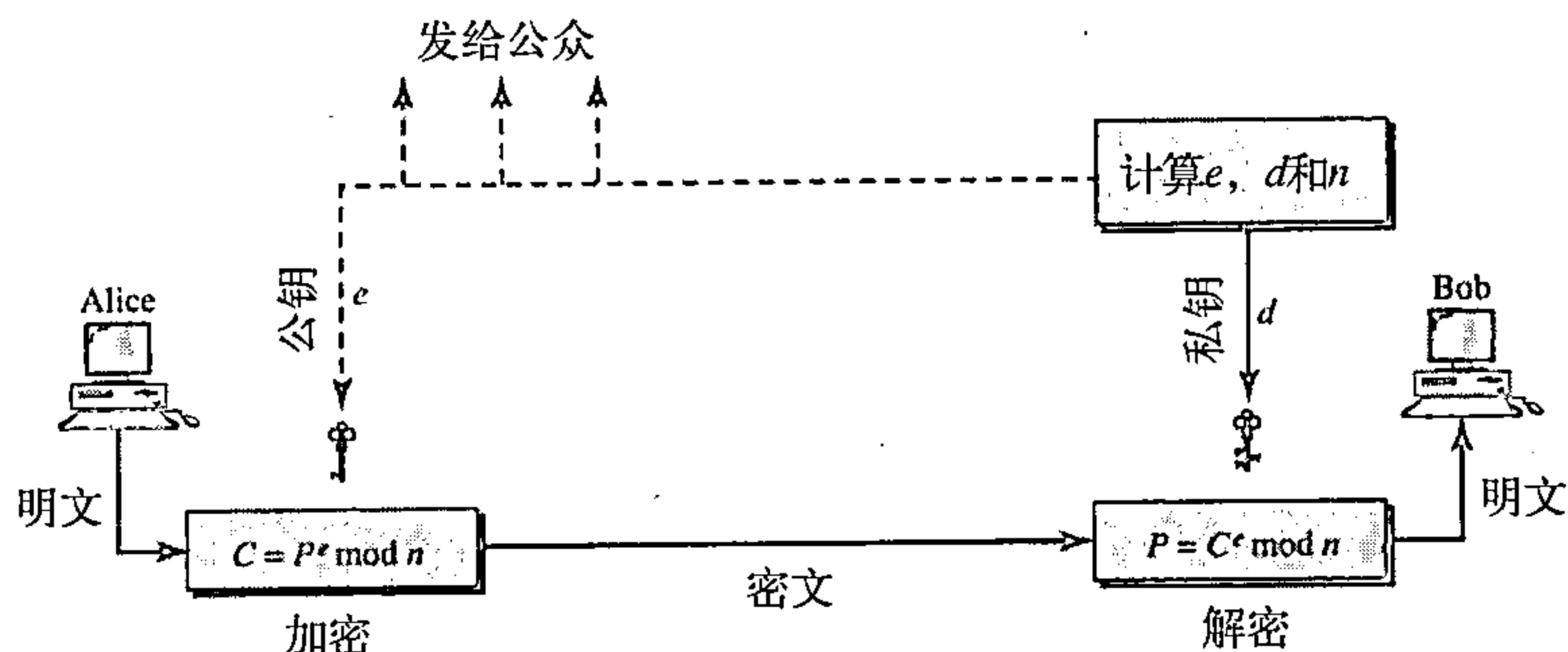


图30.24 RSA

两个密钥 e 和 d 之间有一种特殊的关系，讨论这个关系超出了本书的范围。我们说明一下怎样计算出这两个密钥而不去证明。

选择密钥

Bob使用以下的步骤选择私钥与公钥：

1. Bob选择两个非常大的素数 p 和 q 。记住素数只能被1和它本身整除。
2. Bob将上面两个素数相乘得到 n ，作为加密与解密的模。即 $n=p \times q$ 。
3. Bob计算另一个整数 $\phi = (p-1) \times (q-1)$ 。
4. Bob选择一个随机的正数 e ，然后计算出的 d ，使得 $d \times e = 1 \bmod \phi$ 。
5. Bob对外公布 e 和 n ；保留 ϕ 和 d 作为私钥。

在RSA中， e 和 n 是对外公布的；而 d 和 ϕ 保留作为私钥。

加密

任何一个想发送信息给Bob的人都可以使用 e 和 n 。例如，如果Alice需要发送信息给Bob， she可以把信息转为整数（一般情况下，选择较小的整数），这就是明文。然后通过使用 e 和 n 计算出密文。

$$C = P^e \pmod{n}$$

Alice将密文 C 发送给Bob。

解密

Bob保留 ϕ 和 d 作为私钥。当接收到密文后，他使用私钥 d 对信息进行解密。

$$P = C^d \pmod{n}$$

限制

为了RSA能够正常工作， P 的值必须要小于 n 的值。如果 P 是一个很大的整数，则明文需要分成多个分组，使得 P 小于 n 。

例30.7

Bob选择 $p=7$ 、 $q=11$ ，并计算出 $n=7 \times 11=77$ 。 $\phi=(7-1) \times (11-1)=60$ ；他选择两个密钥 e 和 d ，如果 $e=13$ ，则 $d=37$ 。现在假设Alice发送明文5给Bob，她用公钥13对明文5进行加密。

明文：5

$$C=5^{13} \pmod{77} = 26$$

密文：26

Bob接收密文26，并使用私钥37对密文进行解密：

密文：26

$$P=26^{37} \pmod{77} = 5$$

明文：5

Alice想要发送的信息。

Alice发送的明文是5，被Bob接收仍旧为明文5。

例30.8

Jennifer为自己创建了一对密钥，她选择 $p=397$ ， $q=401$ ，计算出 $n=159\,197$ ， $\phi=396 \times 400 = 158\,400$ 。然后她选择 $e=343$ ， $d=12\,007$ 。说明如果Ted知道了 e 和 n ，他如何发送信息给Jennifer。

解：

假设Ted想发送报文“NO”给Jennifer，他根据相应的字符码，把每一个字符转换为两位数（从00到25）。然后连接这两个字符码，得到一个四位的整数，即明文为1314。Ted用 e 和 n 对报文进行加密，密文为 $1314^{343} \pmod{159\,197} = 33\,677$ 。Jennifer接收到报文33 677后，使用解密密钥 d 对密文进行解密，如 $33\,677^{12\,007} \pmod{159\,197} = 1\,314$ 。然后Jennifer解码1314为报文“NO”。图30.25说明了过程。

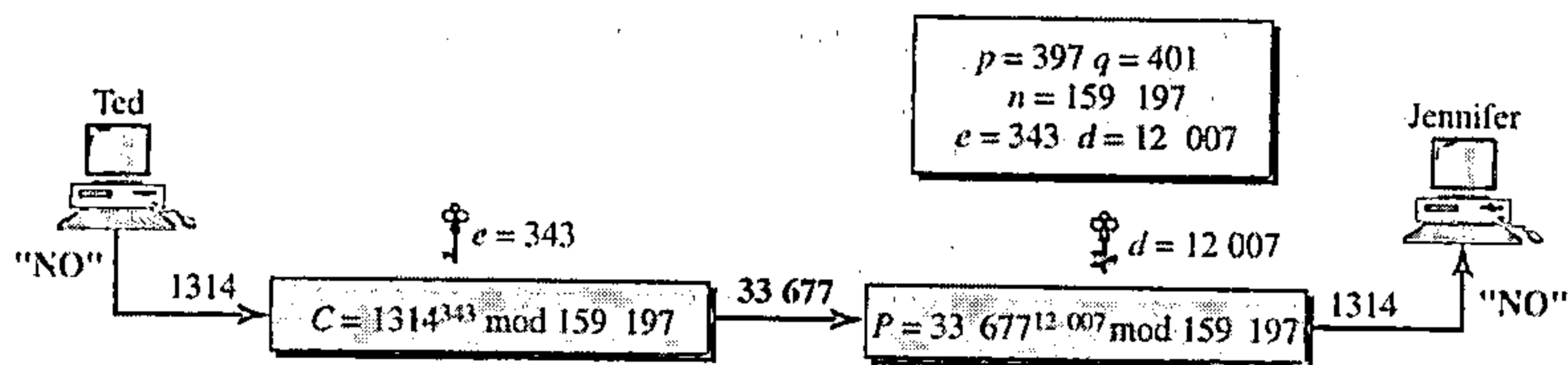


图30.25 例30.8

例30.9

我们给出一个现实的例子。我们选择512位的 p 和 q ，并计算出 n 与 ϕ 。然后选择 e ，并用 $\phi(n)$ 测试相应的素数，计算出 d 。最后显示加密与解密的结果。我们写了一个Java的程序来做这件事情，该类型的计算是计算器不能完成的。

我们随即选择一个512位的整数，整数 p 是一个159位的数。

$p = 96130345313583504574191581280615427909309845594996215822583150879647940$
 $45505647063849125716018034750312098666606492420191808780667421096063354$
 219926661209

整数 q 是160位的数。

$q = 12060191957231446918276794204450896001555925054637033936061798321731482$
 $14848376465921538945320917522527322683010712069560460251388714552496900$
 0359660045617

我们计算 n ，它是309位的整数。

$n = 11593504173967614968892509864615887523771457375454144775485526137614788$
 $54083263508172768788159683251684688493006254857641112501624145523391829$
 $27162507656772727460097082714127730434960500556347274566628060099924037$
 $10299142447229221577279853172703383938133469268413732762200096667667183$
 $1831088373420823444370953$

我们计算 ϕ ，它也是309位的整数。

$\phi = 11593504173967614968892509864615887523771457375454144775485526137614788$
 $54083263508172768788159683251684688493006254857641112501624145523391829$
 $27162507656751054233608492916752034482627988117554787657013923444405716$
 $98958172819609822636107546721186461217135910735864061400888517026537727$
 $7264467341066243857664128$

我们选择 $e=35\ 535$ ，然后找到 d 。

$e = 35535$

$d = 58008302860037763936093661289677917594669062089650962180422866111380593852$
 $82235873170628691003002171085904433840217072986908760061153062025249598844$
 $48047568240966247081485817130463240644077704833134010850947385295645071936$
 $77406119732655742423721761767462077637164207600337085333288532144708859551$
 36670294831

Alice要发送报文“THIS IS A TEST”，这个报文可以通过00-26编码方案转换为数值（26为空字符）。

$P = 1907081826081826002619041819$

Alice计算密文为： $C=P^e$ 。

$C = 4753091236462268272063655506105451809423717960704917165232392430544529$
 $6061319932856661784341835911415119741125200568297979457173603610127821$
 $8847892741566090480023507190715277185914975188465888632101148354103361$
 $6578984679683867637337657774656250792805211481418440481418443081277305$
 $9004692874248559166462108656$

Bob通过 $P=C^d$ ，将密文解密， P 为：

$P = 1907081826081826002619041819$

解密后的明文解码后为：“THIS IS A TEST”。

应用

尽管RSA可以对实际的报文进行加密与解密，但如果报文很长，则加解密的过程非常慢。因而，RSA更适合那些短报文，如短的报文摘要或在对称密钥加密系统中的对称密钥。实际上，我们将看到RSA使用在数字签名和那些经常需要加密短报文而不访问对称密钥的加密系统中。在后面我们也可以看到，RSA也可以用于鉴别。

30.3.2 Diffie-Hellman 加密系统

RSA是一个公钥加密系统，并经常用于对称密钥的加密与解密。另一方面，Diffie-Hellman最初设计是用于密钥的交换。在Diffie-Hellman加密系统（Diffie-Hellman cryptosystem）中，两方产生一个对称的会话密钥（session key）用来交换数据，而且不需要为了将来的使用而必须记住或存储密钥。他们不需要面对面地对密钥达成共识，这个可以在因特网上进行。让我们看一下：当Alice与Bob需要一个对称密钥通信时，协议是如何进行工作的。在建立对称密钥之前，两方需要选择两个数 p 和 g 。第一个数 p 是非常大的素数，大概为300位的十进制数（1024位），第二个数是一个随机数。这两个数并不需要保密，可以在因特网上传送，并且可以是公开的。

过程

图30.26说明了整个过程，步骤如下：

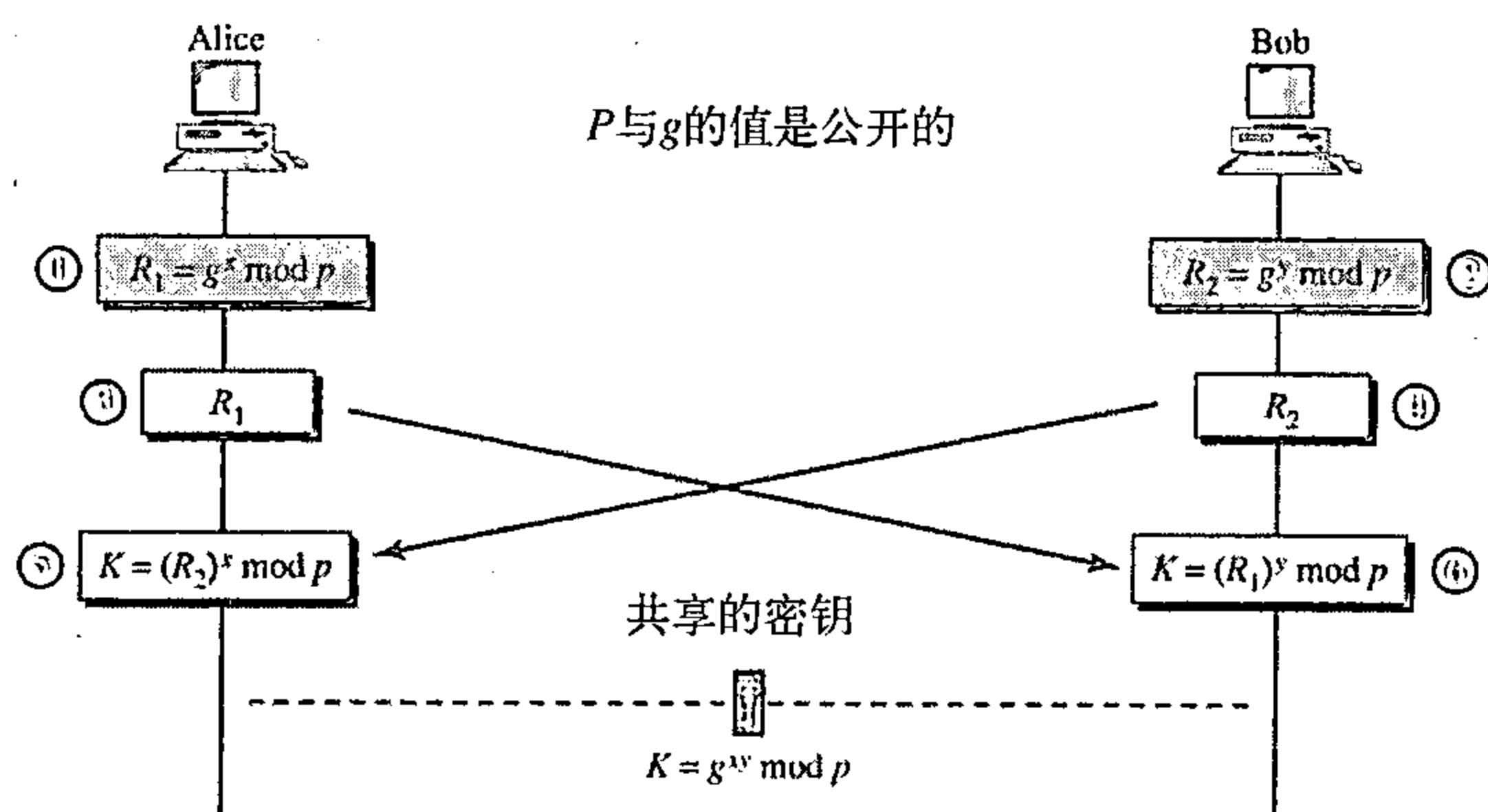


图 30.26 Diffie-Hellman方法

- 步骤1：Alice选择一个大的随机数 x ，并计算出 $R_1 = g^x \mod p$ 。
- 步骤2：Bob选择另一个大的随机数 y ，并计算出 $R_2 = g^y \mod p$ 。
- 步骤3：Alice发送 R_1 给Bob，注意Alice并不发送 x 的值，它仅发送 R_1 。
- 步骤4：Bob发送 R_2 给Alice，注意Bob并不发送 y 的值，它仅发送 R_2 。
- 步骤5：Alice计算出 $K = (R_2)^x \mod p$ 。
- 步骤6：Bob也计算出 $K = (R_1)^y \mod p$ 。

这个会话的对称密钥就是 K 。

$$(g^x \mod p)^y \mod p = (g^y \mod p)^x \mod p = g^{xy} \mod p$$

Bob已经计算出 $K = (R_1)^y \mod p = (g^x \mod p)^y \mod p = g^{xy} \mod p$ ；Alice也计算出 $K = (R_2)^x \mod p = (g^y \mod p)^x \mod p = g^{xy} \mod p$ ；两个人都得到了相同的值，并且Bob不知道 x 的值，而Alice也不知道 y 的值。

在Diffie-Hellman协议中的对称（共享）密钥是 $K = g^{xy} \mod p$ 。

例30.10

我们举一个普通的例子来说清楚这个过程。例子中使用了小的数，但必须注意到现实中，数是非常大的。假设 $g=7$ 、 $p=23$ 。步骤如下：

1. Alice选择 $x=3$ ，并计算出 $R_1 = 7^3 \mod 23 = 21$ 。
2. Bob选择 $y=6$ ，并计算出 $R_2 = 7^6 \mod 23 = 4$ 。

3. Alice发送数值21给Bob。
4. Bob发送数值4给Alice。
5. Alice计算出对称密钥 $K=4^3 \bmod 23=18$ 。
6. Bob也计算出对称密钥 $K=21^6 \bmod 23=18$ 。

对Alice与Bob来说， K 值是相同的； $g^{xy} \bmod p = 7^{18} \bmod 23 = 18$ 。

Diffie-Hellman的思想

如图30.27所示，Diffie-Hellman的概念是简单但是一流的。我们可以认为在Alice与Bob之间的密钥是由三部分组成： g ， x 和 y 。第一部分是公开的。每个人都知道密钥的1/3， g 是公开的值。另外两部分必须由Alice和Bob加入。每一个人加入一部分，Alice为Bob加入第二部分 x ；Bob为Alice加入第二部分 y 。当Alice从Bob接收到了全部密钥的2/3后，她在最后部分加入她的 x ，完成了整个密钥；当Bob从Alice接收到了全部密钥的2/3后，他在最后部分加入他的 y ，完成了整个密钥。注意到，尽管在Alice手中密钥由 $g-y-x$ 组成；在Bob手里密钥由 $g-x-y$ 组成，但这两个密钥是相同的，因为 $g^{xy}=g^{yx}$ 。

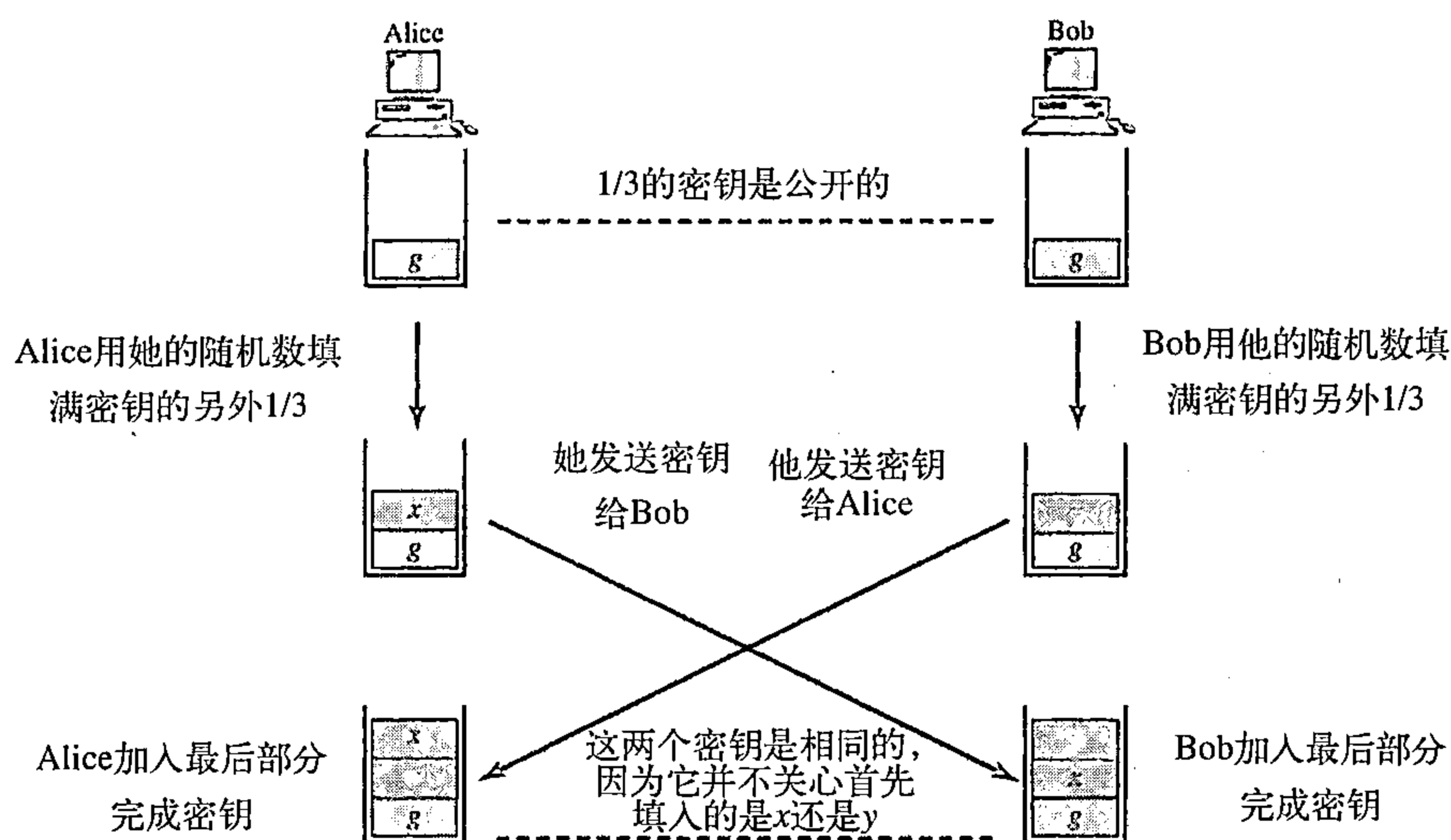


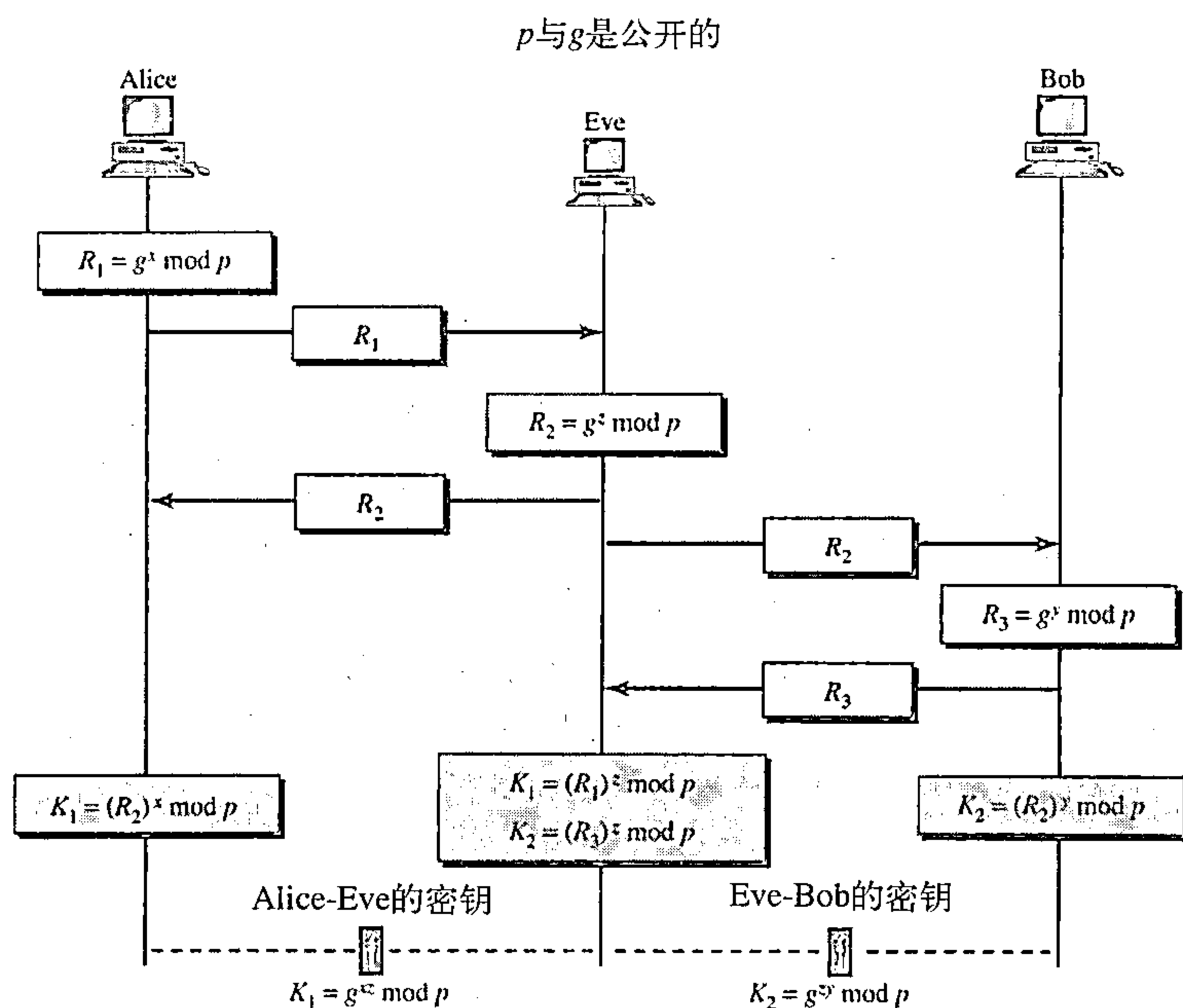
图30.27 Diffie-Hellman的概念

注意：尽管这两个密钥是相同的，但是Alice还是不能发现Bob使用的 y 值。因为计算是通过模 p 生成的；Alice从Bob接收到的是 $g^y \bmod p$ ，而不是 g^y 。

中间人攻击 (Man-in-the-Middle Attack)

Diffie-Hellman是一个非常复杂的创建对称密钥的算法。如果 x 、 y 是一个非常大的值，那么对Eve来说，只知道 p 与 g 的情况下来破解这个密钥是极其困难的。入侵者中途截获 R_1 和 R_2 ，还需要确定 x 和 y ，但从 R_1 和 R_2 中找到 x 与 y 是两件非常困难的工作。即使一个高端的计算机也有可能花上几年的时间通过尝试各个不同的数值来找到密钥。另外，Alice和Bob在下次需要通信的时候可能已经更换了密钥。

然而，协议本身的确有自身的弱点。Eve并不需要一定要找到 x 与 y 的值来攻击通信协议。她可以通过产生两个密钥来愚弄Alice与Bob：一个密钥用于Alice与她自己之间；另一个密钥用于Bob与她自己之间。图30.28说明了这个情况。



下面可能会发生：

1. Alice选择 x ，计算 $R_1 = g^x \bmod p$ ，并发送 R_1 给Bob。
2. 入侵者Eve，中途截获了 R_1 ，她选择 z ，计算 $R_2 = g^z \bmod p$ ，并发送 R_2 给Alice和Bob两个人。
3. Bob选择 y ，计算 $R_3 = g^y \bmod p$ ，并发送 R_3 给Alice， R_3 被Eve中途截获，永远不能到达Alice。
4. Alice和Eve计算 $K_1 = g^{xz} \bmod p$ ，这就成为了Alice与Eve的共享密钥。然而Alice认为这是她与Bob之间的共享密钥。
5. Eve和Bob计算 $K_2 = g^{zy} \bmod p$ ，这就成为了Eve与Bob的共享密钥。然而Bob认为这是他与Alice之间的共享密钥。

换句话说，产生两个密钥替代了原来的一个密钥：Alice与Eve之间一个；Eve与Bob之间一个。当Alice发送用 K_1 （Alice与Eve共享的）加密的数据给Bob时，它可以被Eve解密并读取。Eve可以发送用 K_2 （Eve与Bob共享的）加密的报文给Bob；或者她甚至可以更改报文并发送一个完全新的报文，Bob被愚弄并充分相信这个报文来自Alice。在另一个方向上，相同的场景也会发生在Alice方。

这种情况称为中间人攻击（man-in-middle attack），因为Eve位于中间，他中途截取Alice发送给Bob的 R_1 和Bob发送给Alice的 R_3 。因为它类似于志愿救火队列中人与人传递水桶，所以也称为水桶队列攻击（bucket brigade attack）。

鉴别

如果Bob与Alice在第一次时相互进行鉴别，就可以避免中间人攻击。换句话说，密钥交换的过程可以结合鉴别方案，来避免中间人攻击。我们将在第31章讨论鉴别。

推荐读物

为了深入讨论与本章有关的主题，推荐下面的读物。在本节末列出方括号[···]中的参考

资料。

读物

密码学可以在许多相关主题的书中找到，如[Bar02]、[Gar01]、[Sti02]、[Mao04]、[MOV97]、和[Sch96]。

本章小结

- 密码学是将报文进行转换以使其安全，免于遭受攻击的科学和艺术。
- 明文是变换前的原始报文；密文是经过变换后的报文。
- 加密算法是将明文转换为密文；而解密算法是将密文转换为明文。
- 加密算法与解密算法合在一起称为加密算法。
- 密钥是过程的一个数字或多个数字。
- 我们可以将加密算法分成两大类：对称密钥加密算法与非对称密钥加密算法。
- 在对称密钥加密算法中，发送方和接收方都使用相同的密钥。这个密钥就被称为秘密密钥。
- 在非对称密钥加密算法中，使用了一对密钥。发送方使用公钥；接收方使用私钥。
- 替换加密算法用另一个字符替代另一个字符。
- 替换加密算法可以分为两大类：单字母替换加密算法和多字母替换加密算法。
- 移位加密算法是最简单的单字符替换加密算法。它使用模26的模运算。恺撒加密算法就是密钥为3的移位加密算法。
- 换位加密算法对明文字符重新排序，从而生成密文。
- 异或加密算法是自身不可逆转的最简单的加密算法。
- 旋转加密算法是可逆转的加密算法。
- S-盒是输入为 N 位，输出为 M 位的无密钥的替换加密算法，并使用了公式定义了输入流与输出流之间的关系。
- P-盒是输入为 N 位，输出为 M 位的无密钥的换位加密算法，并使用表定义输入流与输出流之间的关系。P-盒只有在输入位与输出位相同的情况下是可逆转的；P-盒可以使用直接置换、压缩置换和扩充置换。
- 现代加密算法通常是迭代加密算法，每一次迭代都是由不同的简单加密算法组成的复杂加密算法。
- DES是美国政府采用的对称密钥方法；DES有初始和最终的置换模块及16次迭代。
- DES的核心是DES函数。DES函数由四部分组成：扩充置换、异或操作、S-盒及直接置换。
- DES使用密钥生成器产生16个48位的迭代密钥。
- 三重DES用来增加DES密钥的长度（有效位达到56位），提供更好的保密性能。
- AES是基于128位数据分组的Rijndael算法的迭代加密算法。AES有三种不同的配置：128位密钥的10次迭代；192位密钥的12次迭代和256位密钥的14次迭代。
- 操作模式是指部署如DES或AES等加密算法的技巧。通常的四种加密模式是ECB、CBC、CBF和OFB。ECB和CBC是分组加密算法；CBF和OFB是流加密算法。
- 通常使用的公共密钥加密方法是RSA算法，由Rivest, Shamir及Adleman发明。
- RSA选择 n 作为两个素数 p 和 q 的产物。
- Diffie-Hellman方法为两个组织提供了一次性的会话密钥。
- 如果两方相互之间并没有进行鉴别，那么中间人攻击可能会危及Diffie-Hellman方法的安全。

习题

复习题

1. 在对称密钥密码学中，如果Alice与Bob之间进行通信，需要多少个密钥？
2. 在对称密钥密码学中，Alice能够用相同的密钥与Bob和John两者都进行通信吗？并解释你的答案。
3. 在对称密钥密码学中，在一个10人组中的每一个人都需要对另外一个10人组中的每个人进行通信，试问需要多少密钥？
4. 在对称密钥加密学中，在一个10人组中的每一个人需要与这个组中的其他人进行通信，试问需要多少密钥？
5. 在非对称密钥密码学中，重新回答问题1。
6. 在非对称密钥密码学中，重新回答问题2。
7. 在非对称密钥密码学中，重新回答问题3。
8. 在非对称密钥密码学中，重新回答问题4。

练习题

9. 在对称密钥密码学中，你认为两个人如何在他们之间建立密钥？
10. 在非对称密钥密码学中，你认为两个人如何在他们之间建立两对密钥？
11. 用密钥为20的移位加密算法对报文“THIS IS AN EXERCISE”进行加密。忽略单词之间的空格。并对密文进行解密，得到原始的明文。
12. 如果符号仅是0和1，我们是否可以使用单字母替换加密算法？这是一个好的方法吗？
13. 如果符号仅是0和1，我们是否可以使用多字母替换加密算法？这是一个好的方法吗？
14. 使用下面的密钥，采用置换加密算法对“INTERNET”进行加密。

3	5	2	1	4
1	2	3	4	5
15. 将111001向右循环移3位。
16. 将100111向左循环移3位。
17. 一个S-盒有6位输入、2位输出，假定输入最低位是1，它将奇数位的输入（1，3，5，…）相加成为输出的低位，将偶数位的输入相加（2，4，6，…）相加成为输出的高位。如果输入是101101，试问输出是什么？如果输入是101101，试问输出是什么？假定最低位是1。
18. 一个S-盒有6位输入、2位输出，所有可能的输入的组合值是什么？所有可能的输出值是什么？
19. 一个S-盒有4位输入、3位输出，输入的最高位确定其余3位的循环。如果最高位是0，则其余3位向左循环移1位；如果最高位是1，则其余3位向右循环移1位。如果输入是1011，试问输出是什么？如果输入是0110。试问输出是什么？
20. P-盒使用下面的表进行加密，说明P-盒，并连接输入到输出。这个P-盒是直接置换、压缩置换还是扩充置换？

4	2	3	1
1	2		
21. 在RSA中，给出两个素数 $p=19$ 、 $q=23$ ，计算 n 和 ϕ 。选择 $e=5$ ，计算 d ，使得 e 和 d 能够符合要求。
22. 为了进一步理解RSA算法的安全性，如果知道了 $e=17$ ， $n=187$ ，计算 d 。这个习题说明了如

果 n 非常小的话,对Eve来说破解这个算法还是非常简单的。

23. 在RSA算法中选择一个非常大的 n ,解释:为什么Bob可以从 n 计算得出 d ,而Eve不能。
24. 在RSA算法中, $e=13$ 、 $d=37$ 和 $n=77$,采用00-25对应A-Z的编码方案,对报文“FINE”进行加密。为简单起见,逐个字符进行加密与解密。
25. 在RSA中,Bob为什么不能选择1作为公钥 e ?
26. 在RSA中,如果选择2作为公钥 e 会有什么风险?
27. Eve用Bob的公钥采用RSA发送一个报文给Bob。稍后在酒会上,Eve看到Bob并问Bob是否收到报文,Bob回答已收到。在饮酒后,Eve又问:“密文内容是什么?”Bob告诉Eve密文内容。这是否危及Bob私钥的安全?试说明其理由。
28. 在Diffie-Hellman协议中,如果 $g=7$ 、 $p=23$ 、 $x=2$ 及 $y=5$,对称密钥的值是多少?
29. 在Diffie-Hellman协议中,如果 $x=y$ 会发生什么?这就是说:Alice和Bob偶然地选择了同一个数,那么 R_1 和 R_2 的值也相等?Alice和Bob计算出来的会话密钥的值也相等?举例说明你的论点。

探究题

30. 另一个非对称密钥加密算法是ElGamal。请研究并查找关于此算法的资料,并请说明RSA与ElGamal的区别。
31. 另一个非对称密钥加密算法是基于椭圆曲线的,如果你熟悉椭圆曲线,请研究并找出基于椭圆曲线的算法。
32. 为了使Diffie-Hellman算法更加健壮,有人使用了cookies。请研究并找出在Diffie-Hellman算法中,是如何使用cookies的。

第31章 网络安全服务

第30章介绍了密码学，本章讨论密码学在网络安全中的一些应用。首先，我们介绍网络中典型的安全服务，然后说明如何用密码学提供这些服务。在本章末，我们还讨论对称与非对称密钥的分发问题。本章为第32章提供必要的基础，在第32章讨论因特网中的安全问题。

31.1 网络安全服务种类

网络安全能提供如图31.1所示的五种服务。其中的四种服务与使用网络的报文交换有关：报文保密性、完整性、鉴别和不可否认。第五种服务是提供实体的鉴别或认证。

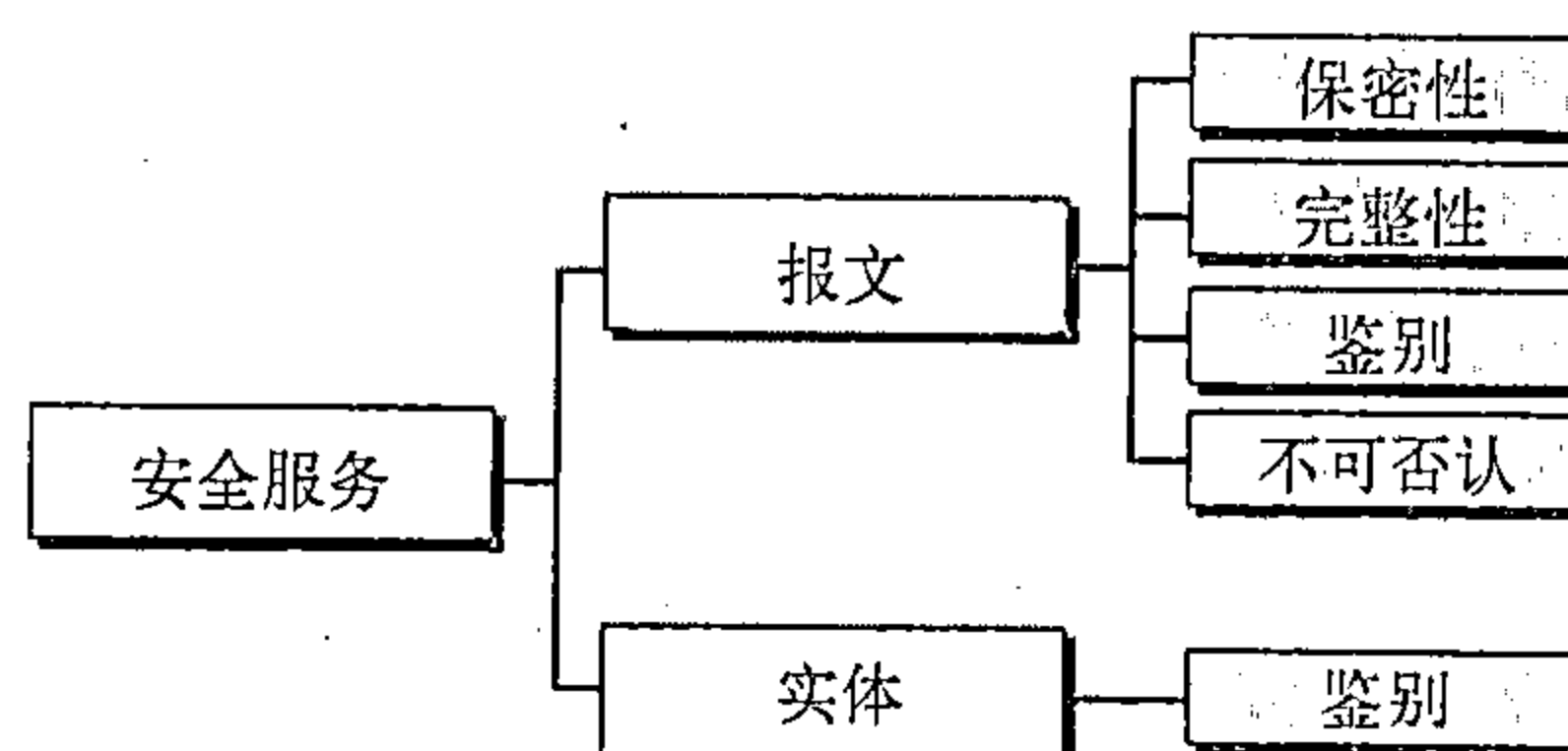


图31.1 报文或实体有关的安全服务

31.1.1 报文保密性

报文保密性 (message confidentiality or privacy) 是指发送方和接收方期望保持机密。传输的报文必须只对预定的接收方有意义。对所有其他人，报文都是毫无意义的。当顾客与银行通信时，他期望通信是保密的。

31.1.2 报文完整性

完整性 (message integrity) 是指数据到达接收方时必须和它们发送时保持一致。在传输过程中不能发生意外的或者恶意的改变。随着越来越多的货币交易在因特网上进行，完整性变得至关重要。例如，如果请求转账\$100变成请求\$10 000或\$100 000，这将是灾难性的。在安全通信中必须保证报文的完整性。

31.1.3 报文鉴别

除了报文完整性之外，报文鉴别 (message authentication) 是指接收方需要确认发送者的身份，而不是未发送该报文的冒充者。

31.1.4 不可否认性

不可否认性 (nonrepudiation) 是指接收方必须能够证明报文来自特定的发送方。发送方无法否认他事实上已经发送过的报文。证明的责任落在接收方身上。例如，当顾客发送一份报文要求将资金从一个账户转到另一个账户时，银行必须拥有该顾客确实请求转账的证据。

31.1.5 实体鉴别

在实体鉴别（或用户认证）中，实体或用户在访问系统资源（例如，文件）之前核实身份。例如，需要访问学校资源的学生在登录过程中必须被核实身份，这是为保护学校与学生的利益。

31.2 报文保密性

如何实现报文保密性的观念数千年来都没有改变：报文必须在发送方加密而在接收方解

密。也就是说，报文必须处理成未经授权人员无法理解的形式。良好的保密技术能够在一定程度上保证入侵者（窃听者）无法理解报文的内容。正如第30章讨论的那样，这可用对称密钥加密或非对称密钥加密。我们复习这两种技术。

31.2.1 对称密钥密码学的保密性

尽管现代的对称密钥算法比长期历史以来所用的秘密写法复杂得多，但基本原理是一致的。为了用对称密钥密码学中的密钥提供保密性，发送方与接收方必须共享密钥。在过去，两个特定人员（例如，两个朋友或一个统帅和军队的首领）可能当面交换个人的密钥，今天的通信不可能提供这样的机会。如一个居住在美国的人不可能与居住在中国的人碰面交换密钥。更进一步，上亿人之间的通信不是一个小数目。

为了能使用对称密钥加密，我们需要解决密钥共享问题。这可用会话密钥（session key）来完成。这个会话密钥仅在一次会话期间使用，正如我们将在后面将看到的，会话密钥本身要使用非对称密钥加密方法进行交换。图31.2表示了Alice和Bob之间双方发送保密报文所用的会话对称密钥。注意：虽然今天我们不推荐采用对称密钥实施双方通信，但对称密钥的性质允许双方实施通信。使用两个不同的密钥更安全，因为如果一个密钥被泄露，那么在另一方向的通信保密性还依旧存在。

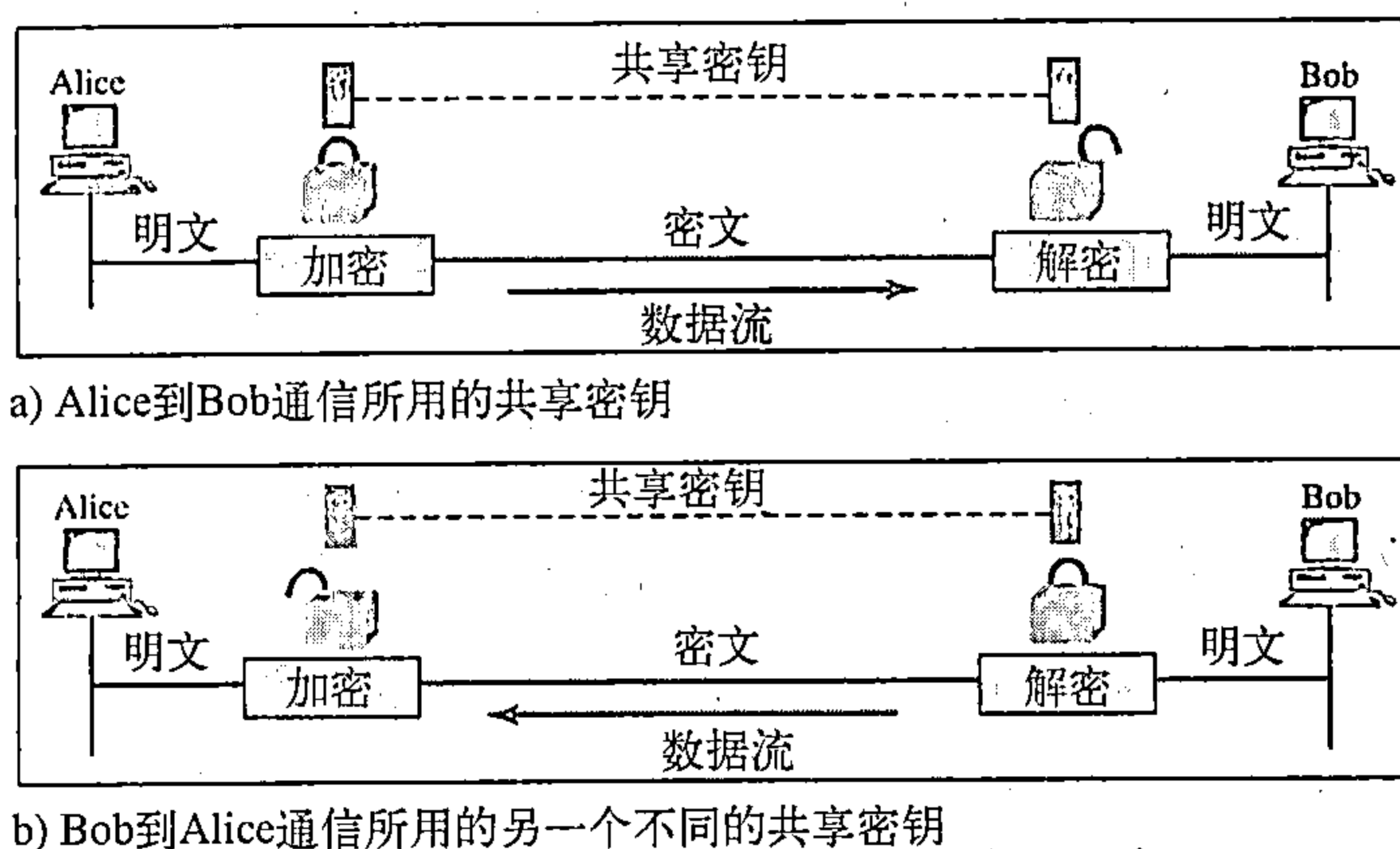


图31.2 在双方采用对称密钥报文的保密性

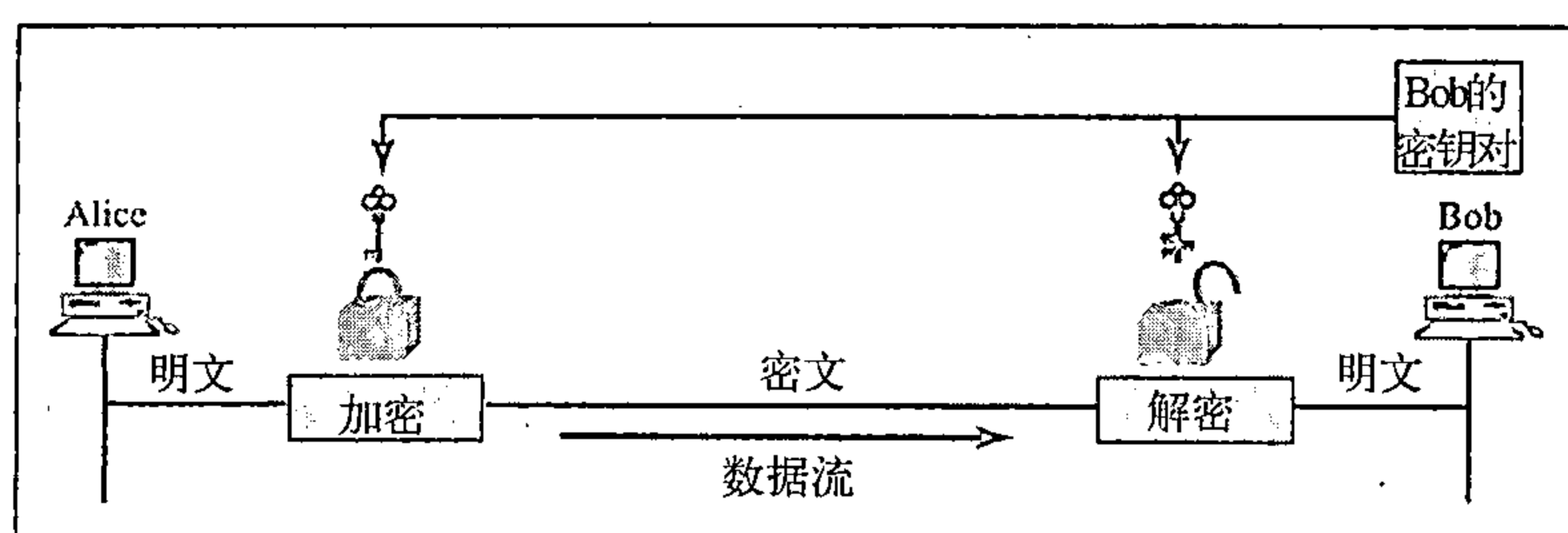
对称密钥加密仍是报文保密性的主要方法，其原因是它的效率。特别是对于长报文，对称密钥加密比非对称密钥加密的效率更高。

31.2.2 非对称密钥密码学的保密性

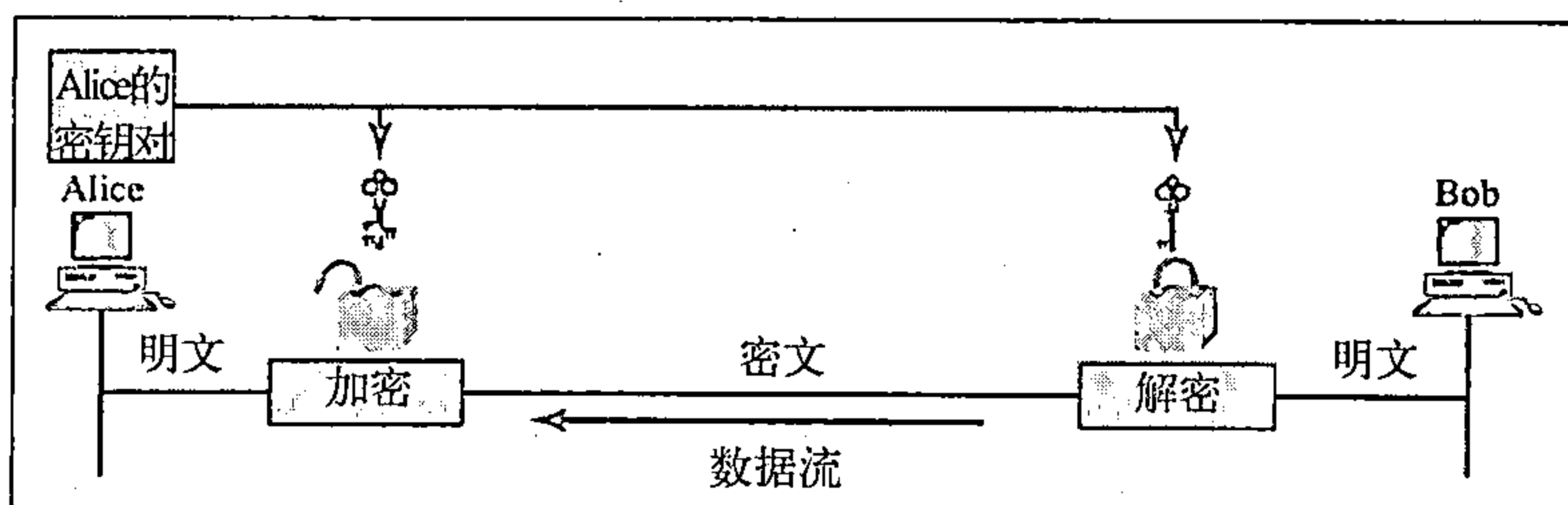
在对称密钥加密中，我们提到密钥交换是通过非对称密钥加密的保密性来完成。在非对称密钥加密过程中，没有共享密钥，而公钥是公开分发的。Bob创建两个密钥：一个是私钥，另一个是公钥。他保留私钥用于解密，向所有人公布他的公钥。公钥仅用于加密，而私钥仅用于解密。公钥锁定报文，私钥解锁报文。

Alice与Bob之间双向通信需要二对密钥。当Alice向Bob发送报文时，她使用Bob的公钥。当Bob向Alice发送报文时，他使用Alice的公钥。如图31.3所示。

非对称密钥加密的保密性存在两个问题：第一，该方法基于用长密钥进行复杂的数学计算。就是说，对于长报文，系统效率较低；它只适用于短报文的加密。第二，报文的发送还需要接收方的公钥。例如，Alice到Bob的通信，Alice需要确信Bob的公钥是真实的，因为Eve可用Bob的名义发布她自己的公钥。在本章后面将看到，系统需要信任。



a) Alice到Bob的通信使用Bob的密钥对



b) Bob到Alice的通信使用Alice的密钥对

图31.3 使用非对称密钥对的报文保密性

31.3 报文完整性

加密与解密提供保密性，但没有提供完整性 (integrity)。但在有些场合下，我们甚至不要求保密性而要求完整性。例如，Alice立下关于她死后财产分配的遗嘱。遗嘱不要求保密性，死后任何人都可查阅遗嘱。但必须要保持遗嘱的完整性，Alice不希望遗嘱的内容有任何篡改。另一个例子，假定Alice发送一个报文通知银行工作人员Bob支付给Eve咨询费，该报文不需要对Eve保密，因为她知道要付钱给她。但是，报文需要安全不能被篡改，特别不能被Eve篡改。

31.3.1 文档与指纹

保持文档完整性的一个方法是使用指纹 (fingerprint)。如果Alice需要确保文档的内容不被非法篡改，她可在文档末端按下她的指纹。Eve不能篡改该文档的内容，或者伪造文档，因为她不可能伪造Alice的指纹。要确信文档没有被篡改可比较文档上的Alice的指纹与文件上的Alice的指纹。如果它们是不同的，则该文档不是Alice的。

为了保持文档的完整性，需要文档与指纹两者。

31.3.2 报文与报文摘要

电子文档与指纹等价于报文 (message) 和报文摘要 (message digest)。为了保持报文的完整性需要使用称为散列函数 (hash function) 的算法，创建报文的压缩映像用作指纹。图31.4表示了报文、散列函数与报文摘要。

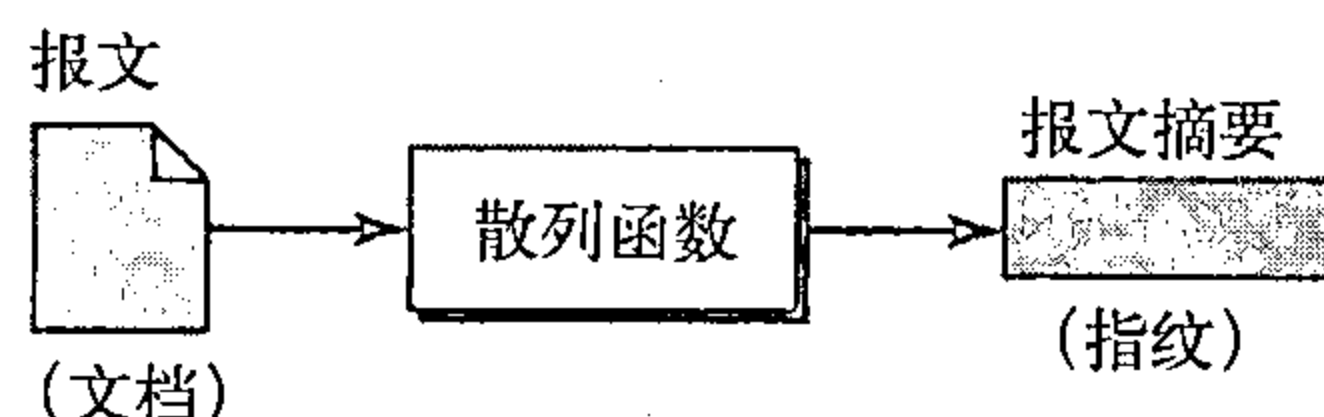


图31.4 报文与报文摘要

31.3.3 差异

文档/指纹对与报文/报文摘要对有一些相似之处，但也有一些差异。文档与指纹在物理上是连在一起的，而且两个都不需要保密，而报文与报文摘要彼此独立的 (发送)，最重要的是报文摘要保密。如果需要通过通信通道发送报文摘要，那么报文摘要需要置于安全的地方加以保密或加密。

报文摘要需要保密。

31.3.4 创建和检验报文摘要

报文摘要要在发送方站点创建，它与报文一起发送到接收方。为了检验报文或文档的完整性，接收方对报文使用相同的散列函数算法再次产生报文摘要，与接收到的摘要比较。如果两份摘要完全相同，则接收方确信原有报文没有被改变。当然，我们假定摘要是加密发送。图31.5表示了这个思想。

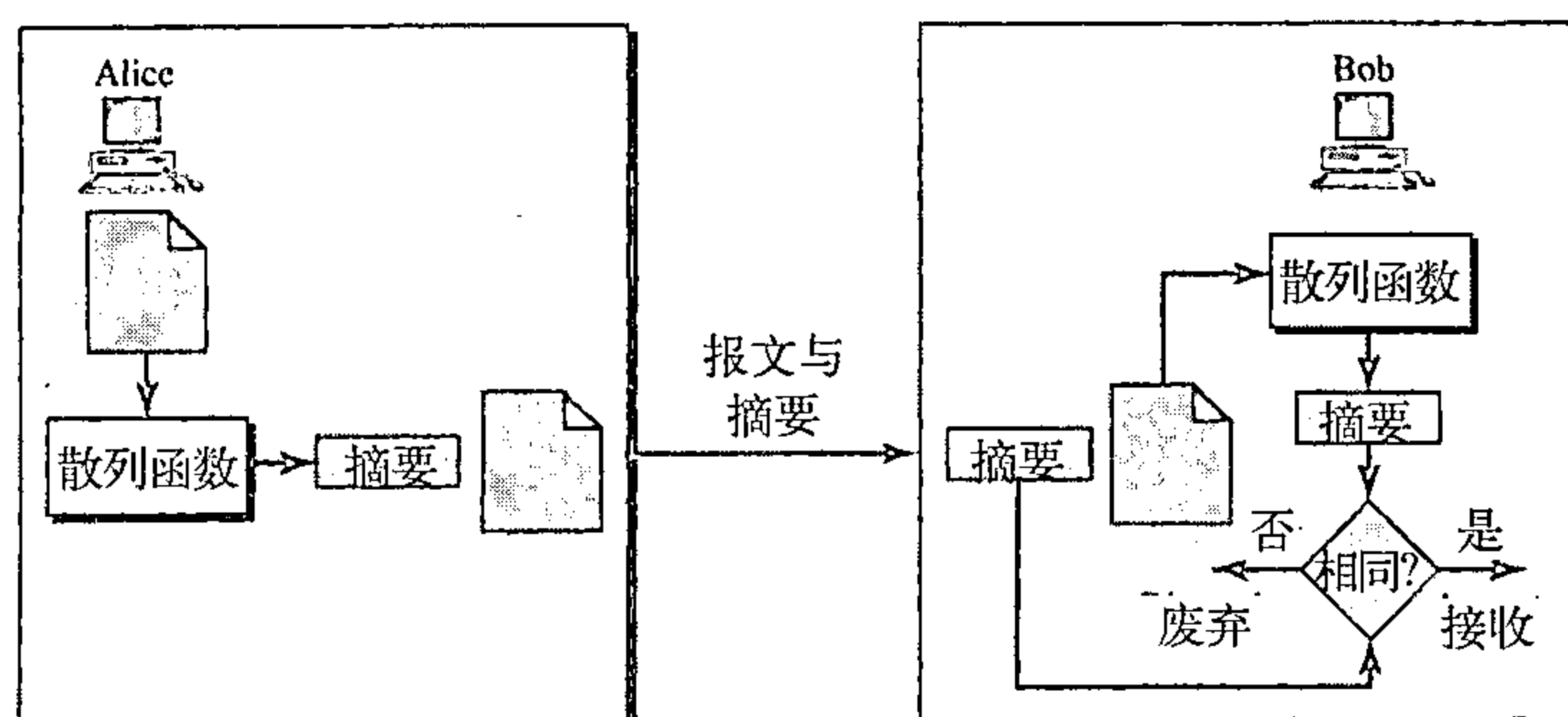


图31.5 检验完整性

31.3.5 散列函数准则

满足安全要求的散列函数必须有三个条件：单向性、抗弱碰撞攻击和抗强碰撞攻击。如图31.6所示。

单向性

散列函数必须具有单向性 (one-wayness)，用单向散列函数创建的报文摘要。不能从报文摘要中重新求出原来的报文。要百分之百做出单向散列函数有时是困难的。准则表明如果已知报文摘要，则求出原来的报文极其困难或不可能。这种情况与文档/指纹类似，没有一个人能从指纹求出文档。

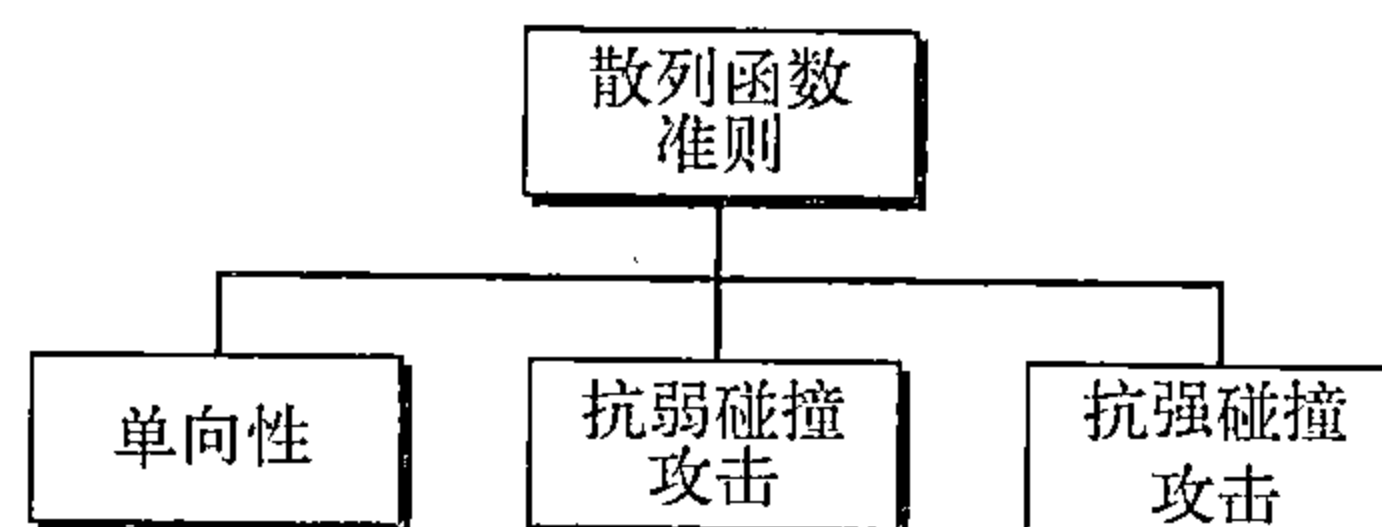


图31.6 散列函数准则

例31.1

能否将传统的无损压缩方法作为一个散列函数？

解：

不可以。因为无损压缩报文是可逆的，可对压缩报文进行解压缩求出原来的报文。

例31.2

能否用校验和方法作为一个散列函数？

解：

可以，校验和函数是不可逆的。它满足第一个条件，但是不能满足其余的条件。

抗弱碰撞攻击

散列函数第二个准则是抗弱碰撞攻击 (weak collision)，确保一个报文不容易被伪装。如果Alice创建报文和摘要，并向Bob发送这两者。Eve不容易创建另一个报文，对它进行散列产生完全相同的报文摘要。换言之，任意给定一个报文以及该报文创建的摘要，不可能找到另一个报文，它的摘要与前一摘要相同，这在计算上是不可行的或很困难的。

当两个不同的报文能创建相同的摘要时，我们称为碰撞。在弱碰撞中，已知一个报文摘要，某人想创建一个报文具有完全相同摘要极其不可能的。散列函数必须具有抗弱碰撞攻击性质。

抗强碰撞攻击

散列函数第三个准则是抗强碰撞攻击 (strong collision)，确保任意二个不同报文，不会有相同的报文摘要。需要这个准则是为了保证报文发送方Alice不能伪造她自己的报文引起问题。如果Alice可创建具有相同摘要的两个不同报文，她可否认发给Bob的第一个报文，声明她仅向Bob发送了第二个报文。

因为这种类型碰撞的概率比前一种情况更高，所以称为强碰撞。对方可创建两个不同报文具有相同的摘要。例如，当报文摘要位长较短时，Alice完全有可能创建具有相同摘要的两个不同的报文。她发送第一个报文给Bob，而自己保存第二个报文。以后Alice可以说第二个报文是她原来同意的文档。

假定创建具有相同文摘的两份不同的遗嘱。当遗嘱执行时，第二份遗嘱呈给继承人。因为两份遗嘱的摘要是相同，偷换成功。

31.3.6 散列算法：SHA-1

虽然已设计出许多散列算法，但常用的是SHA-1。安全散列算法1 (Secure Hash Algorithm, SHA-1) 是美国国家标准与技术协会 (NIST) 修订版，并由联邦信息处理标准 (FIPS) 发布。

该算法与其他算法一样，其要点都遵循同一个概念。每一个算法都创建 N 位长的摘要。一个报文被分成多个块，每块512位。如果报文长不是512位的倍数，填充0扩充成512位的整数倍。如图31.7所示。

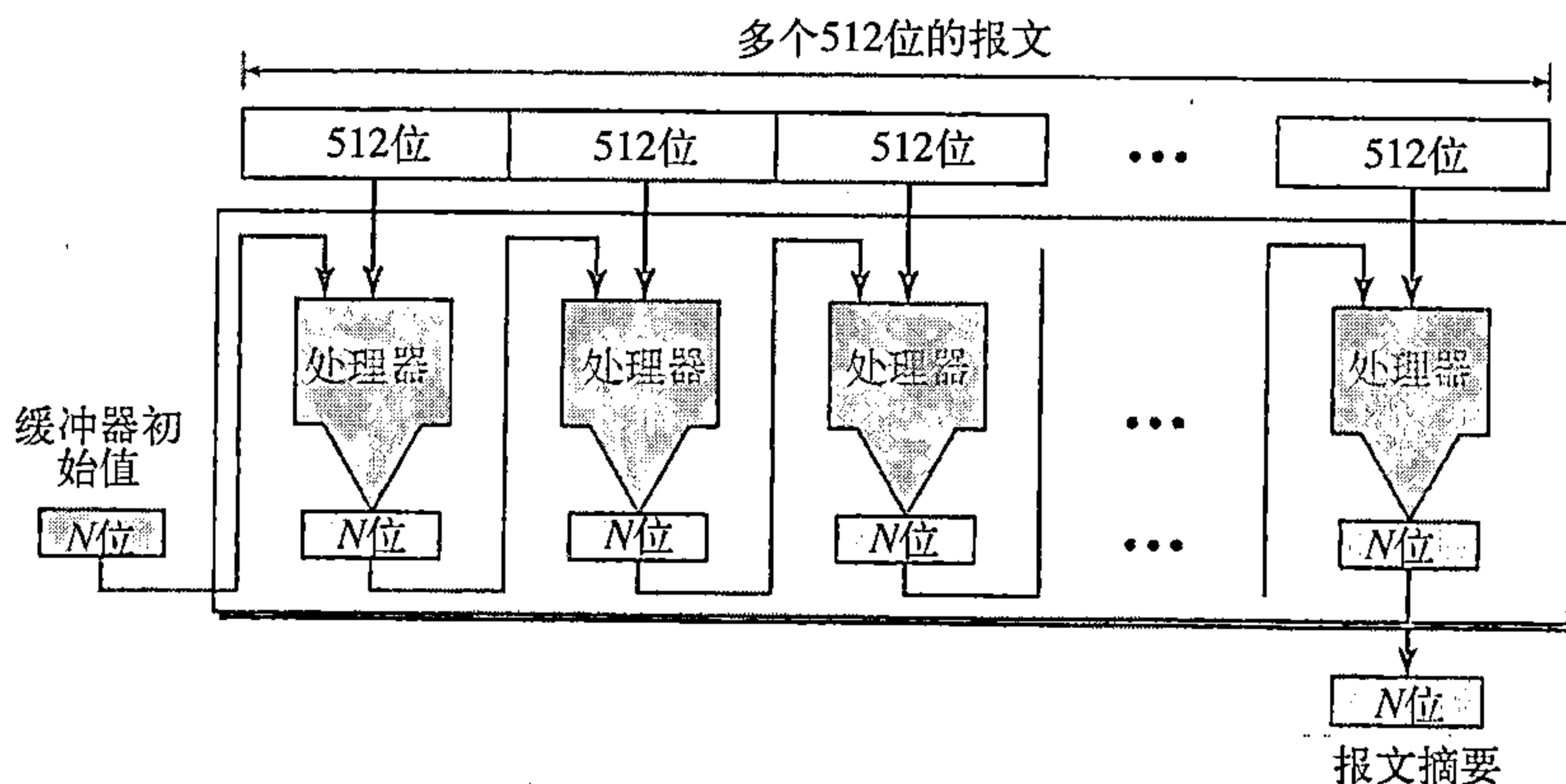


图31.7 创建报文摘要

在 N 位的缓冲器预置一个初始值。算法将该初始值与报文的第一个512位通过杂凑运算生成第一个 N 位的中间报文摘要，然后该中间报文摘要与第二个512位的块通过杂凑运算生成第二个 N 位的中间报文摘要。第 $n-1$ 个 N 位中间报文摘要与第 n 个512位的块通过杂凑运算生成第 n 个 N 位的中间报文摘要。最后一个块处理所生成的摘要就是整个报文的摘要。SHA-1算法的摘要长度是160位 (5个字，每字32位)。

SHA-1算法是对超过512位的报文创建 N 位报文摘要。

SHA-1算法的报文摘要是160位 (5个字，每字32位)。

字扩展

在处理前，需要对块进行扩展，一个块是由512位或16个32位字组成，但在处理阶段要求80个字。因此16个字块需要扩展到80个字，即字0到字79。

每块的处理过程

图31.8表示了每块处理过程总的概要。处理过程共计80步。每一步将扩展块中的一个字与一个32位常数进行杂凑运算创建一个新的摘要。开始时，摘要字的值（A、B、C、D和E）保存在5个临时变量中，处理结束（第79步）时，这些变量与第79步生成的值相加。每步详细情况超出了本书讨论的范围。我们只需了解每一步中一个数据字与一个常数通过杂凑运算创建一个结果送到下一步。

31.4 报文鉴别

散列函数保证报文的完整性，它保证报文不被篡改，但是散列函数不能鉴别报文发送者的身份。当Alice向Bob发送报文时，Bob需要知道是否真的来自Alice，还是Eve伪造的。为了提供报文鉴别，Alice需要提供证据证明该报文确实是Alice发送的，而不是其他人假冒的。散列函数本身不能提供这样的证明，由散列函数创建的摘要称为修改检测码（modification detection code, MDC）。这个码可检测报文中的任何修改。

MAC

为了提供报文鉴别，我们需要将修改检测码改变成报文鉴别码（message authentication code, MAC）。MDC使用无密钥散列函数，MAC使用密钥散列函数。密钥散列函数包括当创建摘要时在发送方与接收方之间的对称密钥。图31.9表示了Alice如何使用密钥散列函数鉴别她的报文以及Bob如何核实报文的真实性。

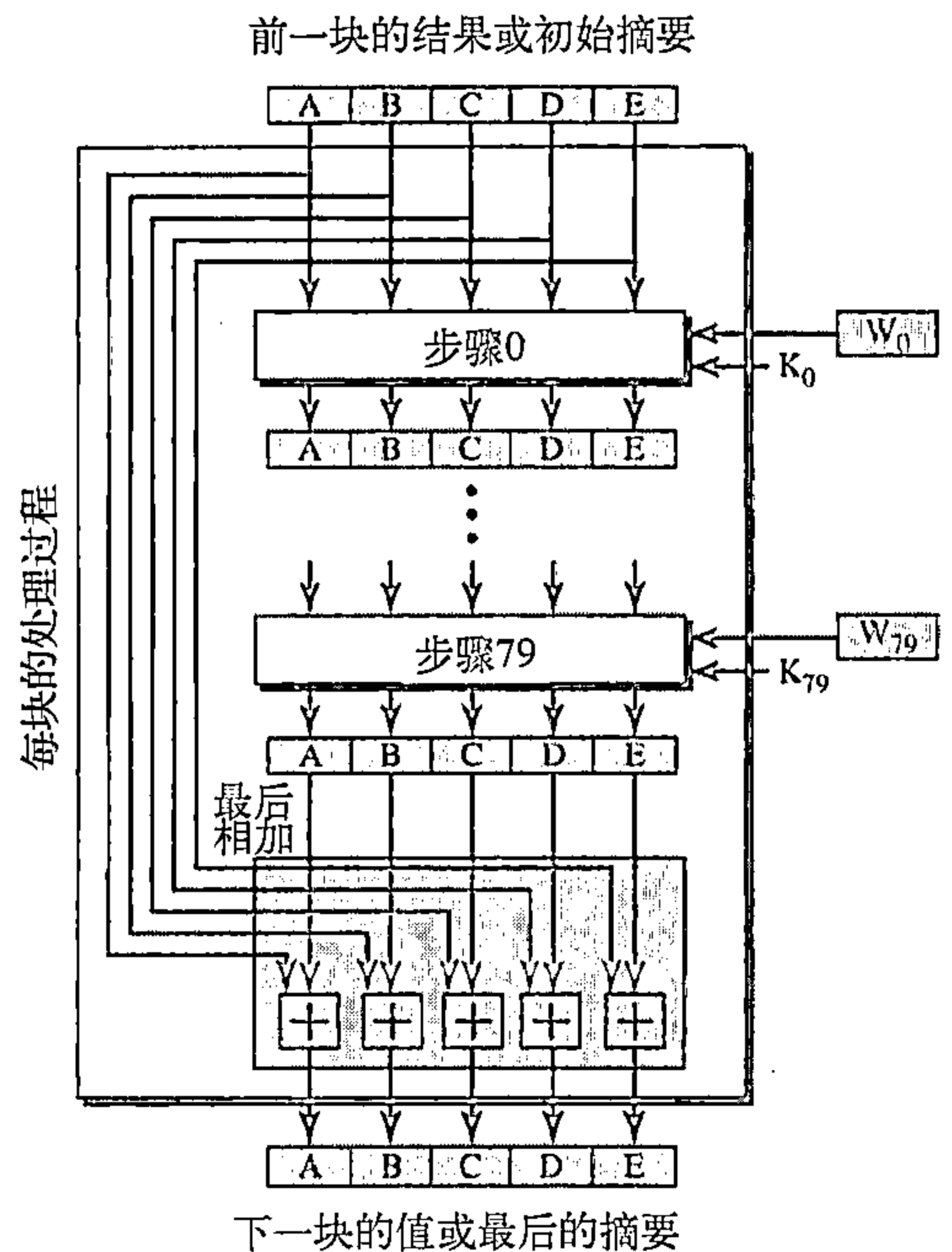


图31.8 SHA-1算法中一个块处理过程

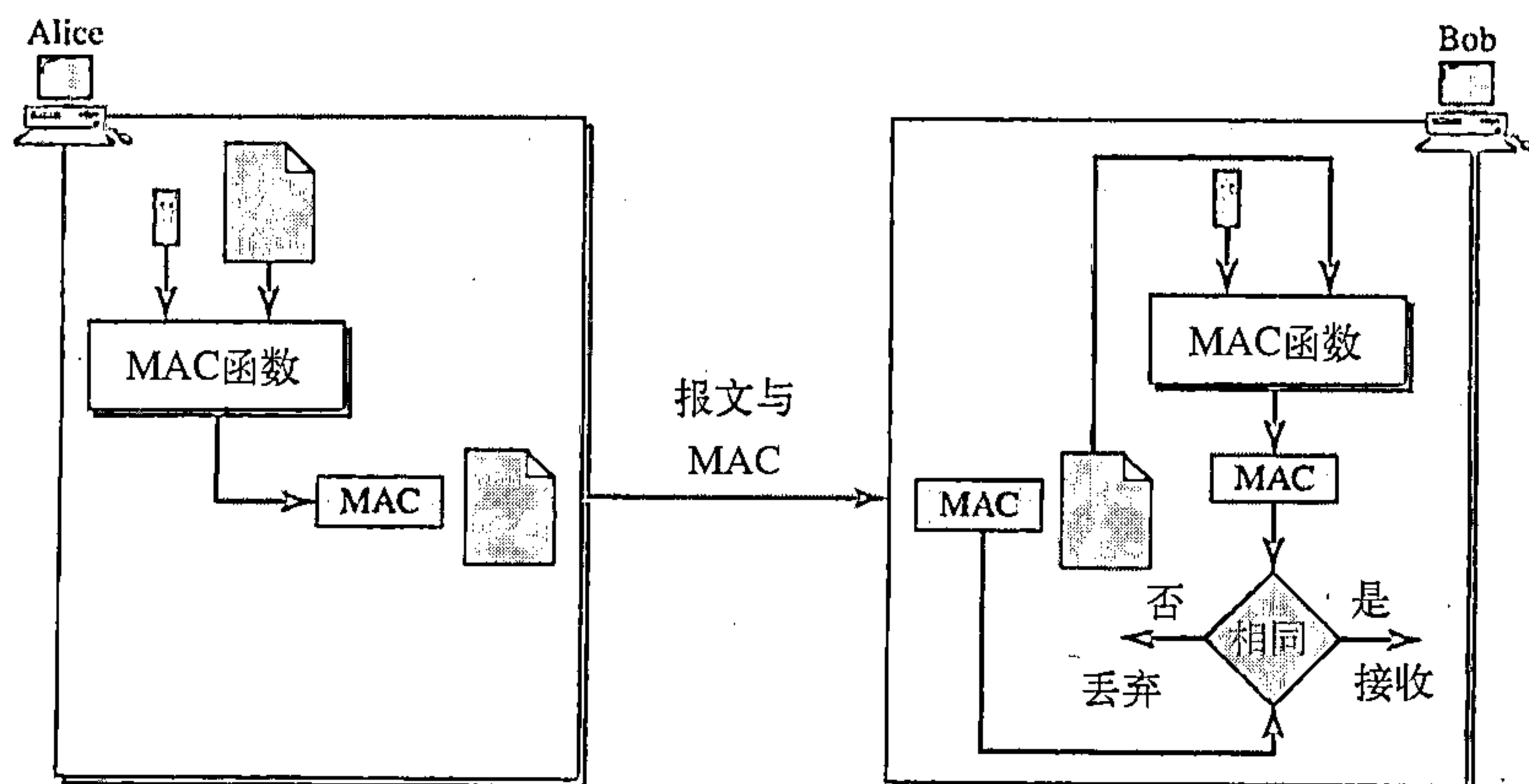


图31.9 Alice创建MAC和Bob检验MAC

Alice使用她与Bob之间的对称密钥 (K_{AB}) 和密钥散列函数生成MAC, 然后她将原来的报文与这个MAC一起发送给Bob。Bob接收到报文与MAC后, 他将报文与MAC分离。他利用对称密钥对报文应用同一密钥散列函数获得新的MAC, 并将它与接收到的MAC比较。如果相同, 报文没有被修改, 而发送方确实是Alice。

HMAC

当今所用的MAC有多种实现方法, 但是近几年来已设计出一些基于无密钥散列函数的MAC, 如SHA-1。这个思想是散列MAC (hashed MAC), 称为HMAC, 它直接使用任何标准的无密钥散列函数, 如SHA-1。HMAC用无密钥散列函数对报文与对称密钥串进行运行创建嵌入的MAC, 图31.10表示了该思想。将预先假定的对称密钥的副本链接到报文, 对这样的链接使用无密钥散列函数, 如SHA-1。其结果是一个中间HMAC, 再用密钥 (同一密钥) 与它链接, 使用同一散列算法获得的最后结果就是HMAC。

接收方接收到这个HMAC和报文。接收方从接收到的报文创建自己的HMAC, 并比较这两个HMAC是否一致来核实报文的完整性并鉴别原有数据。注意: HMAC详细情况比我们此处介绍的要复杂得多。

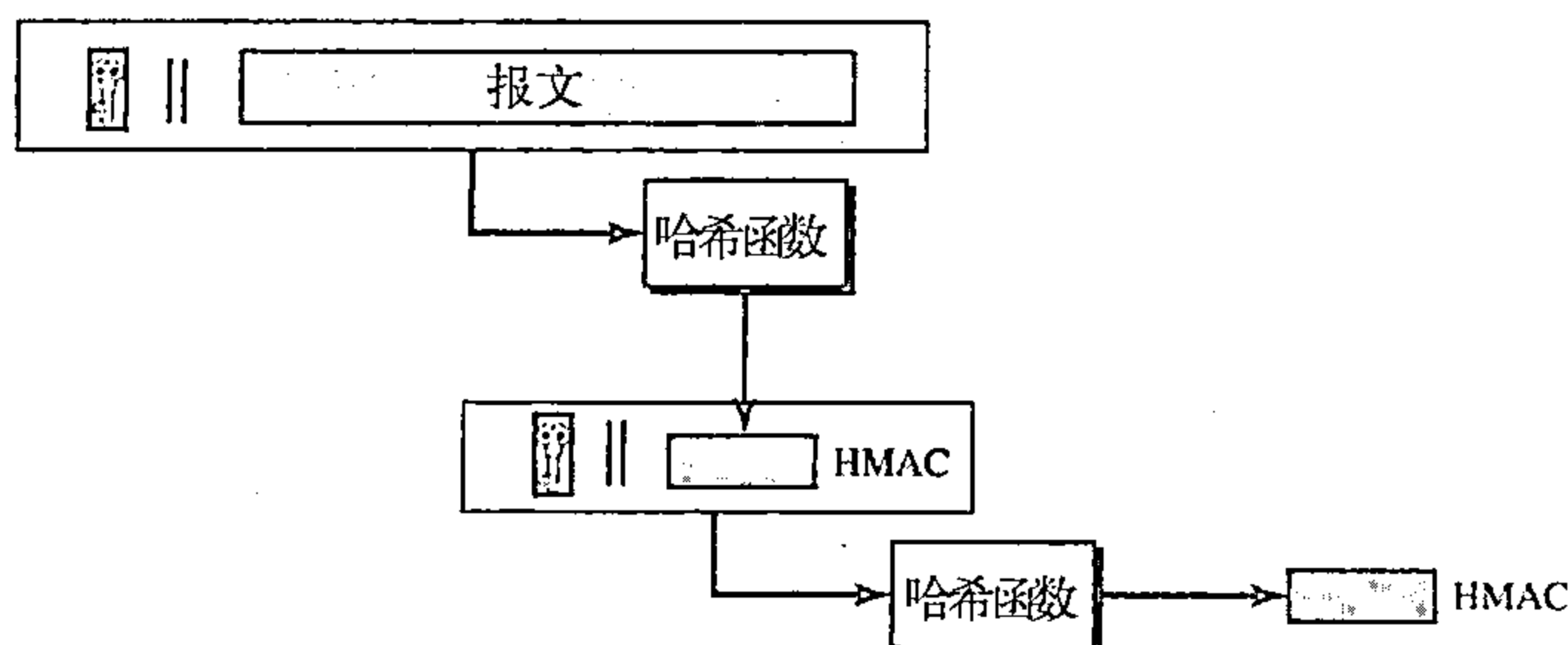


图31.10 HMAC

31.5 数字签名

虽然MAC能提供报文完整性与报文鉴别, 但它有一个缺点。它必须要在发送方与接收方之间建立对称密钥。此外, 数字签名可使用一对非对称密钥 (一个公钥与一个私钥)。

我们都熟知签名的概念, 在文档上签名表示该文档是我们发出的, 或是被我们认可的。签名向接收方证实文档来自正确的实体。当一个顾客在支票上签上他的名字时, 银行需要通过检验确信该支票是由那位顾客签发的, 而不是其他人签发的。换言之, 核实文档上的签名是鉴别符号, 即核实文档是否可信的。考虑一个艺术家在一幅绘画作品上签名, 如果作品上签名是可信的, 就是说这幅绘画作品确实是这个艺术家的作品。

当Alice向Bob发送报文时, Bob需要通过检验核实发送方。他需要确信报文来自Alice而不是窃听者Eve。Bob可请求Alice在报文上进行电子签名。换言之, 电子签名可证明Alice发送报文的真实性。我们称这种类型的签名为数字签名 (digital signature)。

31.5.1 比较

在继续讨论前, 我们考虑两种签名类型的差别: 手书签名和数字签名。

包含性

手书签名包含在文档中, 它是文档的一部分。当我们填写支票并在支票上签名时, 签名不是独立部分。另一方面, 当我们数字地签署文档时, 我们将签名作为独立文档发送。发送方发送两个文档: 报文与签名。接收方接收到两个文档, 并核实签名是否属于假定的发送方。

如果是真的，保存报文；否则拒绝。

验证方法

两种不同签名类型的第二个不同点是验证方法。在手书签名中，当接收方接收到文档时，她将文档的签名与文件中的签名比较。如果相同，则文档是可信的。接收方需要有文件上这个签名的副本用于比较。在数字签名中，接收方接收到报文和签名。没有什么地方保存签名副本。接收方需要对报文与签名的组合用验证技术去核实其真实性。

关系

在手书签名中，签名与文档之间构成一对多的关系。例如，某人有一个签名，可用于签发多张支票、多个文档等等。在数字签名中，签名与文档之间是一一对应的关系。每个文档有它自己的签名，一个文档的签名不能用于另一个文档。如果Bob一个接着一个地接收到来自Alice的二个文档，他不能用第一个报文的签名去验证第二个报文。每一个报文需要新的签名。

二重性

两种不同签名类型的另一个不同是称为二重性的性质。在手书签名中，被签文档的签名可用文件上原来的签名来识别。在数字签名中，除非在文档上有时间因素（如时间戳），否则没有这样的特性。例如，假定Alice发送一个文档指示Bob付款给Eve。如果Eve截取文档与签名，稍后她重发再从Bob处获得付款。

31.5.2 需要密钥

在手书签名中，签名好像一个属于文档签名者的专用“私钥”。签名者用它签名文档，任何其他人都不具有这样的签名。签名的副本好像是一个公钥，任何人都可以用它与原有签名进行比较来验证文档。

在数字签名中，签名者用她的私钥（即应用签名算法）签名文档。另一方面，验证者用签名者的公钥（即应用验证算法）验证文档。

我们能否用秘密（对称）密钥来实现签名与验证签名呢？回答是否定的，有几点理由。第一，秘密密钥只有通信两个实体（例如，Alice与Bob）知道，于是如果Alice需要签名另一个文档发送给Ted，又需要另一个秘密密钥。第二，正如我们将要看到的，为一次会话创建的秘密密钥包括了鉴别，鉴别通常是用数字签名实现。这样就形成了恶性循环。第三，Bob可用他与Alice之间秘密密钥，伪装Alice发送文档给Ted。

数字签名需要一个公钥系统。

31.5.3 过程

数字签名有两种：一种是对整个文档签名，另一种是对文档摘要签名。

签名整个文档

签名整个文档可能比较容易，但效率低。签名文档是用发送方的私钥加密，而验证一个文档则用发送方的公钥进行解密。签名与验证，如图31.11所示。

我们应该区分数字签名中所用的私钥和公钥与保密性加密中所用的私钥与公钥。在保密性过程中，使用接收方的私钥与公钥。发送方用接收方的公钥加密，接收方用自己的私钥解密。在数字签名中，使用发送方的私钥与公钥。发送方用她的私钥，而接收方用发送方的公钥。

在加密系统中，我们使用接收方的私钥与公钥。

在数字签名中，我们使用发送方的私钥与公钥。

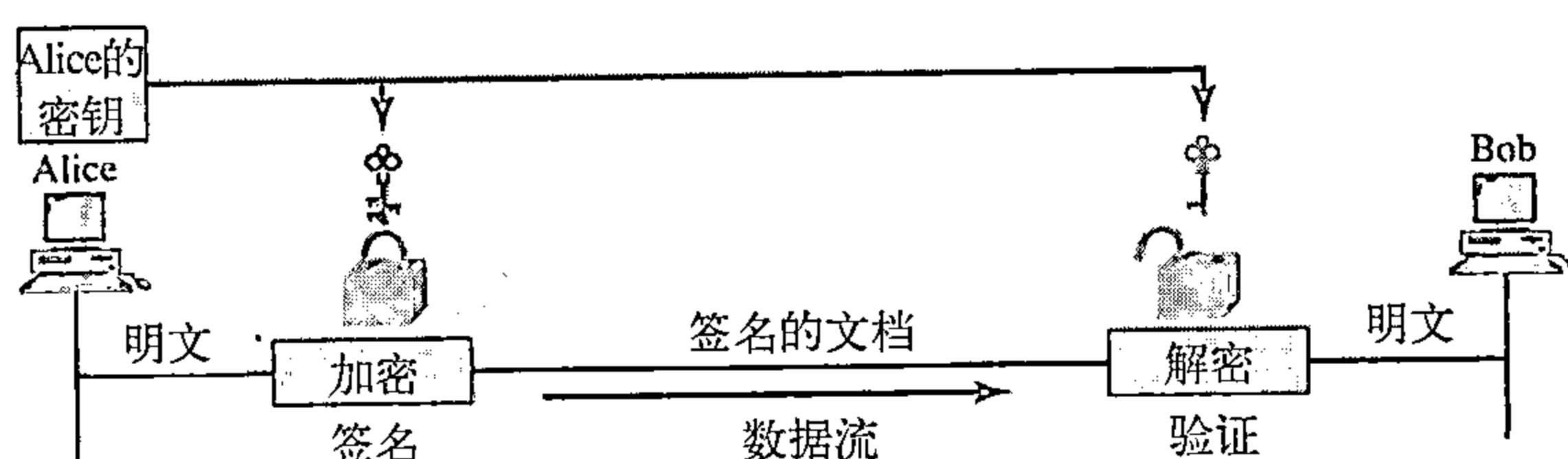


图31.11 在数字签名中，签名报文本身

签名报文摘要

在前面加密系统中提到，报文很长时，使用公钥的效率很低。在数字签名系统中，报文通常是很长的，但我们还必须使用公钥。解决办法不是签名整个报文，而是签名报文摘要。我们已知道，精心计算的摘要与报文是一一对应的关系。发送方签名报文摘要，接收方验证报文摘要的签名。图31.12表示了数字签名系统中的签名摘要。

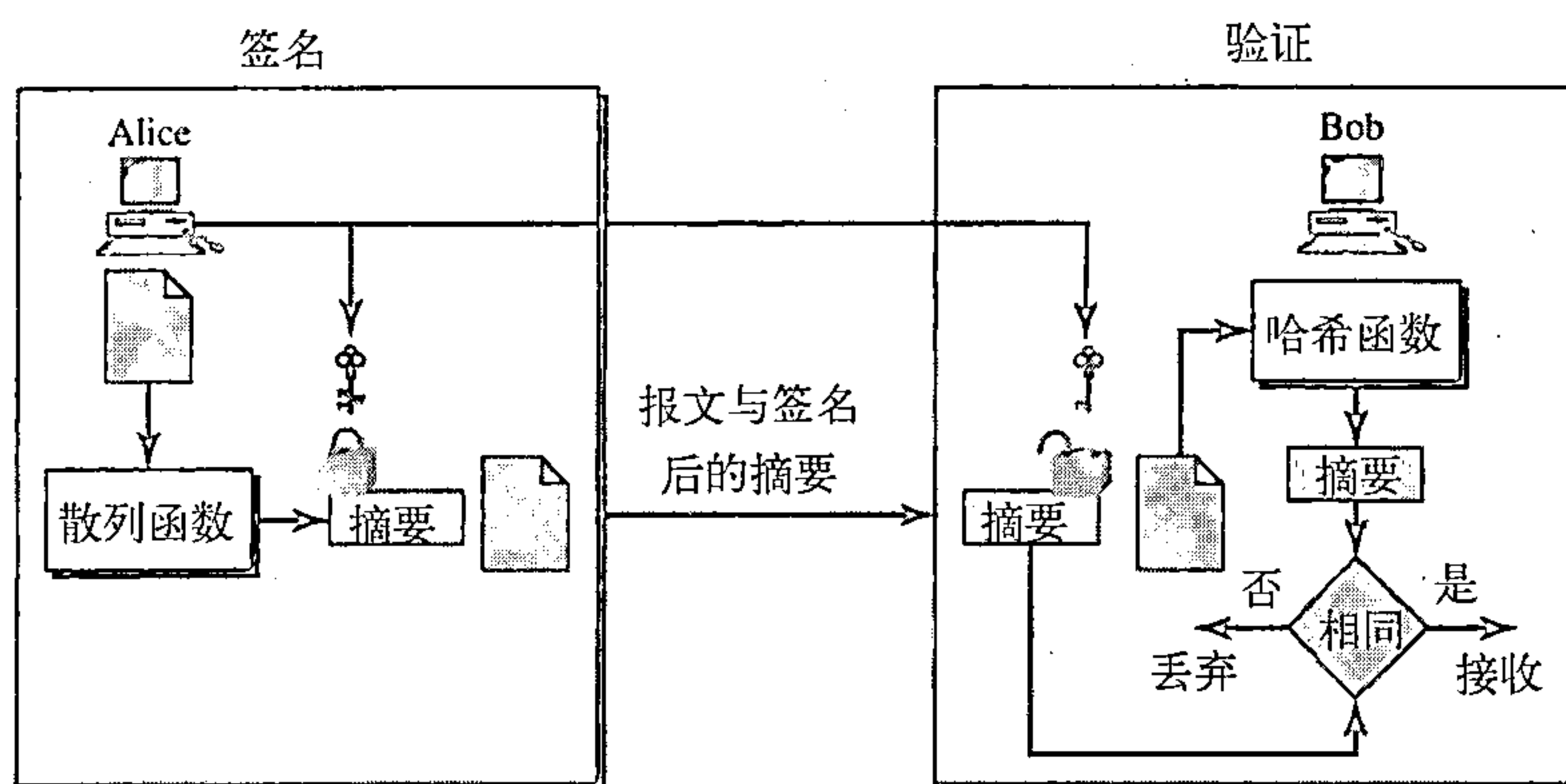


图31.12 数字签名系统中的签名摘要

在Alice站点从报文中创建报文摘要，然后用Alice的私钥签名摘要，再向Bob发送报文和签名后的摘要。在本章后面将会看到这个过程与系统有关，不同系统有一些差异。例如，在创建摘要前可能还有一些附加的计算，或者使用其他的密钥。在某些系统中，签名是一组值。

在Bob站点，首先用同一个散列函数从接收到的报文中创建摘要。然后对鉴别和摘要进行计算。验证过程还将标准应用于计算结果以确定签名的真实性。如果是可信的，接收报文；否则拒绝。

31.5.4 服务

数字签名提供安全系统五种服务中三种：报文的完整性、报文鉴别和不可否认性。注意：数字签名方案没有提供保密的通信。如果要求保密，那么报文和签名必须用秘密密钥加密或公钥加密系统进行加密。

报文完整性

如果我们签名整个报文，那么就保持了报文的完整性。因为如果报文被修改了，那么就不能得到相同的签名。当今的数字签名方案在签名与验证算法中使用散列函数，保持报文的完整性。

当今的数字签名提供报文完整性。

报文鉴别

一个安全的签名方案如手书签名一样（任何人都不易模仿）能提供报文鉴别。Bob因为可用Alice的公钥验证Alice发送的报文，Alice的公钥不可能创建出用Eve的私钥相同的签名。

数字签名提供报文鉴别。

报文不可否认性

如果Alice签名一个报文，事后又否认它，Bob是否可证明Alice的确签名过？例如，Alice发送一个报文给银行工作人员Bob请求从她的账号转\$10 000到Ted的账号，事后Alice是否可赖掉她发送过该报文？到目前为止我们这个方案，Bob可能有一个问题，Bob必须保留文件上的签名和以后要用的Alice的公钥，使用它们可以创建原始报文以证明文件中的报文与新创建的报文是相同的。因为Alices可在这个期间更换她的私钥/公钥，还可声称文件包含的签名是不可信的，所以这是不可行的。

一种解决办法是要有一个可信任的第三方。人们在他们活动中形成一个可信任方。在第32章中，我们将看到，可信任方可解决许多问题如安全服务和密钥交换。图31.13表示了可信任方如何防止Alices否认她发送过的报文。

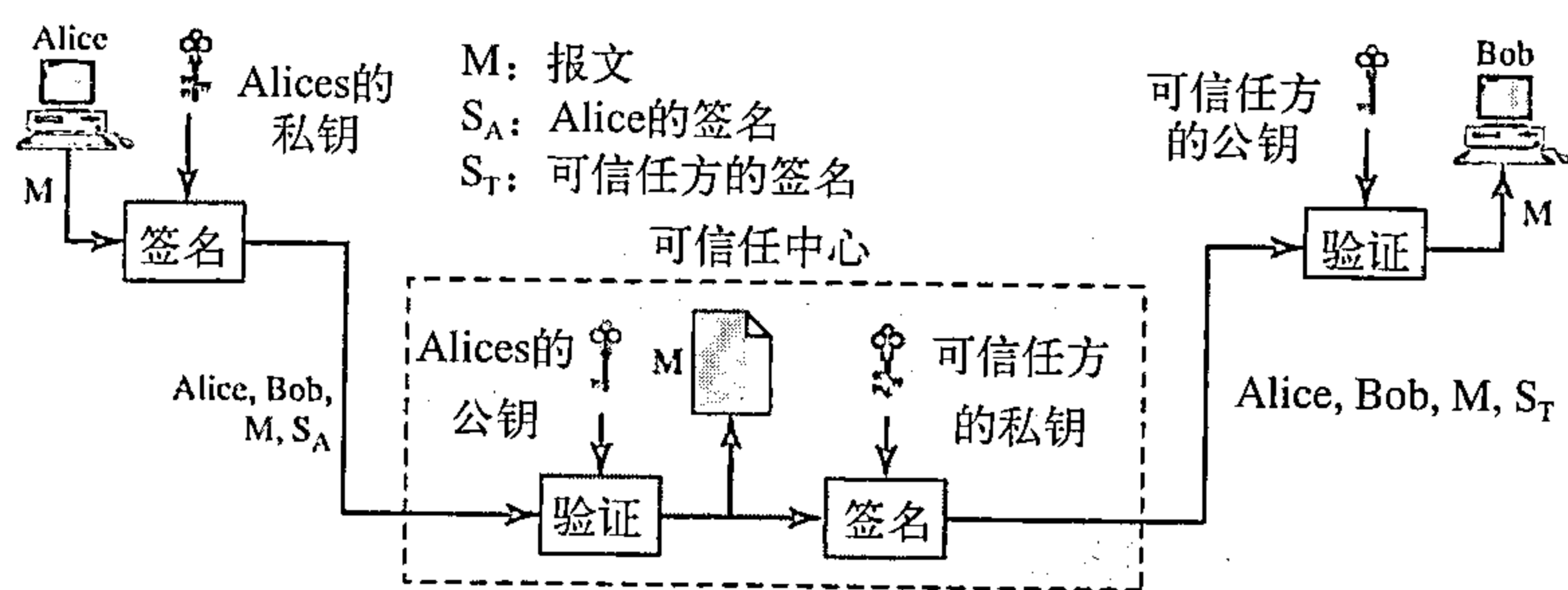


图31.13 使用可信中心防否认

Alices对报文签名（ S_A ）并向可信中心发送报文、Alices和Bob标识符以及签名。可信中心验证Alices的公钥有效后，通过Alices的公钥验证报文是来自Alices。然后可信中心保存报文副本、发送方标识符、接收方标识和时间戳在档案中。可信中心使用它的私钥创建报文的另一个签名（ S_T ）。然后，中心向Bob发送该报文、新的签名、Alices的标识符和Bob标识符。Bob用可信中心的公钥验证报文。

如果将来Alices否认她发送过该报文，可信中心可出示一份保存的报文副本。如果Bob的报文是保存在信任中心的报文副本，则Alices将无法争辩。为了使每一报文都是可信任的，在下一节对方案增加加密/解密。

用可信任的第三方提供不可否认性。

31.5.5 签名方案

最近几十年来，逐渐地设计出一些签名方案。其中有些已实现，如RSA与DSS（数字签名标准）方案。后者可能成为今后的标准，而这些方案的讨论超出了本书范围。

31.6 实体鉴别

实体鉴别是让一方证明另一方的身份的一种技术。一个实体可以是人、进程、客户机或服务器，需要证明身份的实体称为申请者（claimant），试图证明申请者身份的一方称为验证

者 (verifier)。当Bob试图证明Alices身份时，Alices是申请者，Bob是验证者。

报文鉴别与实体鉴别 (entity authentication) 有两点不同。第一，报文鉴别不提供实时性，实体鉴别是实时性。在前者，Alices向Bob发送报文，Bob鉴别报文时，在通信过程中Alices可以出现也可以不出现。而Alices请求实体鉴别时，在Bob鉴别Alices之前，没有实际的报文通信。在鉴别过程中，Alices需要在线并参与其中。只有Alices被鉴别后，Alices与Bob之间才可进行报文通信。当一份电子邮件从Alices发送到Bob时，需要报文鉴别。而当Alices从一部自动取款机提取现金时，需要实体鉴别。第二，报文鉴别只简单鉴别一个报文，对每一个新报文需要重复鉴别，而实体鉴别在一次会话期间证实申请者身份。

在实体鉴别中，验证者必须精确验证申请者的身份。鉴别实体依靠三种基本途径之一实现：所知的、所有的和固有的特征。

- 所知的 这是只有申请者所知的或所掌握的，用于验证者检验申请者所有的“物”。例如，口令、个人识别号 (PIN)、密钥和私钥。
- 所有的 这是所具有的东西，可证明申请者身份的。例如，护照、驾驶证、身份证、信用卡和智能卡。
- 固有的特征 这是申请者的固有特征。例如，手写签名、指纹、语音、脸型、视网膜图案和笔迹。

31.6.1 口令

口令 (password) 是实体鉴别最简单的与最古老的方法。它是申请者拥有的东西，当用户需要访问系统资源 (登录) 时，每个用户有公开的用户标识符和专用的口令。我们将这种鉴别方案分成二组：固定口令 (fixed password) 和一次性口令 (one-time password)。

固定口令

在这个组中，口令是固定的。每次访问都用相同的口令，可是这种方案会遭受到各种攻击。

- 窃听。Eve可注视Alices输入口令。多数系统不显示用户输入的口令字符作为安全措施。窃听有更加多样的形式。例如，Eve可使用在线窃听，拦截报文，由此捕获口令为她所用。
- 窃取口令。Eve直接偷取口令是第二种攻击。Alices不写下口令，而记住她自己的口令，这样可防止窃取口令。所以口令应是简单的，或是Alices易记住熟悉的对象，但它易受到其他类型的攻击。
- 访问文件。Eve像程序员那样操作系统，获取访问存储口令的文件。Eve读取该文件并搜索Alices口令，甚至篡改它。为了防止这类攻击，采用文件读/写保护。但是，多数系统需要这种类型的文件有公开的可读性。
- 猜测。Eve可登录系统，用字符的各种不同组合猜测Alices的口令。如果用户使短口令 (少数几个字符)，特别容易遭受攻击。如果口令选用某些平凡的记号，如她的生日、孩子的名字或她喜欢的明星，也易遭受攻击。为了防止猜测，推荐使用长的随机口令，这样的口令就不明显。但这样的随机口令也存在问题：Alices要在某处存储该口令，这样使她不会忘记它，这又会受到窃取口令攻击。

一个更安全的办法是将口令的散列存储在口令文件中 (不用口令明文存储)。任何用户都可读取文件内容，但由于散列函数是单向的，几乎不可能猜测出口令值。即使Eve窃取口令文件，散列函数阻止她访问系统。但是，会遭受到另一种类型称为字典攻击 (dictionary attack) 的攻击。在这种攻击中，Eve不考虑用户ID而穷举搜索用户口令。例如，口令是6位数字，Eve可建立一张6位数字表 (000000到999999)，然后对每个数字应用散列函数得到一散列值，

形成一张一百万个散列值的表。她再用读取到的加密口令文件第二列的项与她自己形成的散列值表比较，看哪些密文重合。这可在Eve专用的计算机上脱机编程完成。找到匹配后，Eve联机使用口令访问系统。我们将会看到这种攻击在第三种方法中更加困难。

另一种方法称为掺杂 (salting) 口令。当创建口令时，一种称为掺杂的随机数与口令链接，然后对掺杂口令应用散列运算。用户ID、掺杂与散列存储在文件中。现在，当用户请求访问时，系统获取掺杂，并将它与接收到的口令链接，用该链接做出散列，然后与存储在文件中的散列比较。如果匹配，准予访问；否则拒绝。掺杂使字典攻击更加困难。如果原有的口令是6位数字，而掺杂是4位数字，那么散列是在10位数字上运算。这就是说，Eve要做出一张10兆项的表。做出比较长。如果掺杂是一个很长的随机数，掺杂口令是很有效的。UNIX操作系统使用这种方法的变种。

另一种方法是两种认证技术的组合。这种类型鉴别的一个很好的实例是ATM卡与PIN (个人识别号) 结合使用。ATM卡属于“所有的”类，而PIN属于“所知的”类。PIN实际上是口令，它增强了卡的安全性。如果卡被偷窃，除非知道PIN，否则卡不能使用。但是习惯上PIN为自己易于记忆通常较短，这又易受到猜测攻击类型的攻击。

一次性口令

这个方案的口令只使用一次，称为一次性口令 (one-time password)。一次性口令使窃听与窃取失效，但是这种方案很复杂。留在某些专著中讨论。

31.6.2 询问-应答

在口令鉴别中，申请者通过提供她的密码 (口令) 证明她的身份。但是，由于申请者出示这个密码，密码可能被入侵者截取。在询问-应答鉴别 (challenge-response authentication) 中，申请者可不泄露密码证明自己身份。换言之，申请者不用向验证者出示密码，验证者或者可得到它或者可找到它。

在询问-应答鉴别中，申请者不用泄露密码就可证明自己的身份。

询问是一个随时间变化的值，它是由验证者发送的一个随机数或时间戳。申请者对询问使用一个特定的函数，并向验证者发送其结果，这称为应答。应答表示了申请者已经知道密码。

询问是验证者发送的一个随时间变化的值，响应是对询问使用一个特定的函数。

使用对称密钥加密

第一类中，使用对称密钥加密实现询问-应答鉴别。在这一类中，申请者与验证者共享一个秘密密钥。函数是应用于询问的加密算法，图31.14表示了这种方法。图31.14中的第一个报文不属于询问-应答部分，它仅是通知验证者申请者想要询问。第二个报文是询问， R_B 是验证者为询问申请者随机选取的一个瞬时值。申请者使用只有申请者和验证者知道的共享密钥加密瞬时值 (nonce)，并向验证者发送其结果。验证者解密该报文，如果解密得到的瞬时值与验证者发送的瞬时值相同，则同意Alices访问。

注意：在这个过程中，申请者与验证者必须保存保密所用的对称密钥，验证者必须保留用于鉴别申请者的瞬时值直到应答已经返回。

读者可能已注意到了使用瞬时值可以防止Eve重发第三个报文。Eve不可能重发第三个报文，伪装成这是一个新的鉴别请求，因为一旦Bob接收到应答， R_B 的值就不再有效，下次要用新的值。

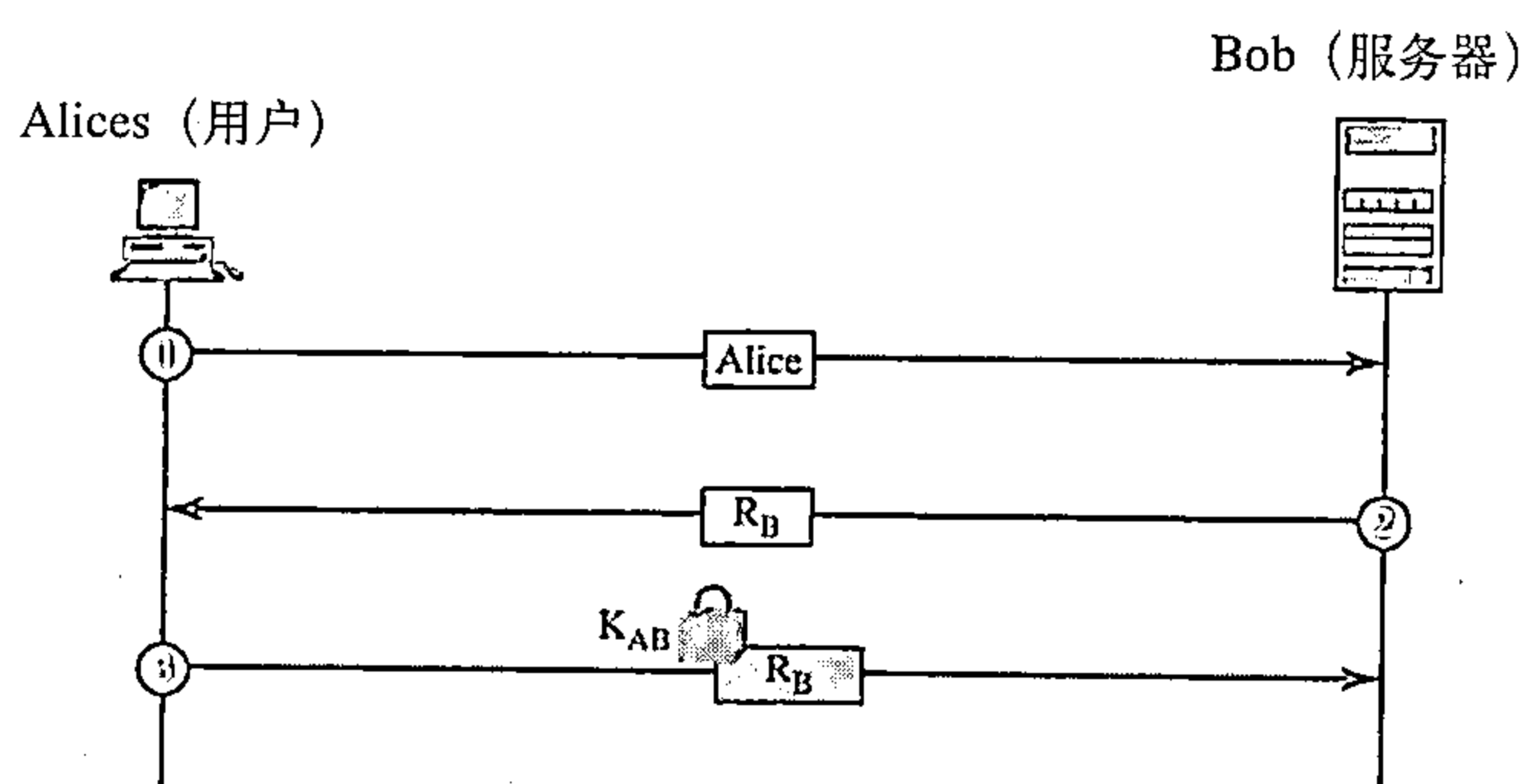


图31.14 使用瞬时值的询问-应答鉴别

这一类中的第二种方法，随时间变化的值是时间戳，因为时间戳显然随时间变化。在这种方法中，询问报文是验证者向申请者发送的当前时间。但是，这是假定客户机与服务器的时钟是同步的，申请者知道当前时间。这就是说，不需要询问报文。第一个报文与第三个报文可结合，其结果是用一个报文、对隐含询问的应答和当前时间完成鉴别。图31.15表示了这种方法。

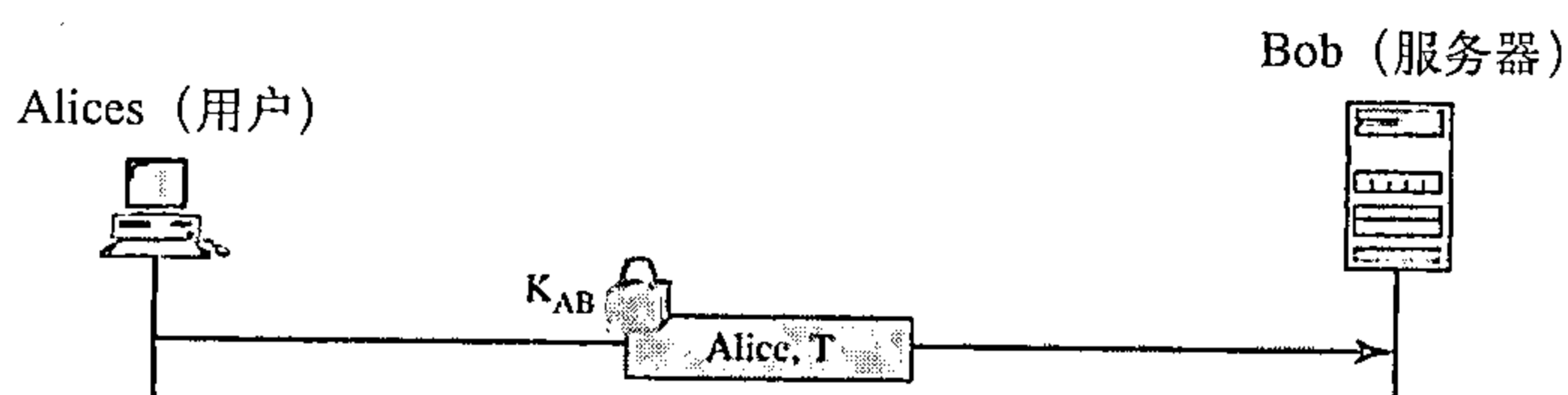


图31.15 使用时间戳的询问-应答鉴别

使用密钥散列函数

实体鉴别不用加密/解密方法，而是使用密钥散列函数（MAC）。这个方法有两个优点。第一，加密/解密算法不能进入某些国家。第二，使用密钥散列算法可保持询问与应答报文的完整性，并同时使用密钥。让我们观察使用密钥散列函数如何创建带有时间戳的询问报文，图31.16表示了该方案。

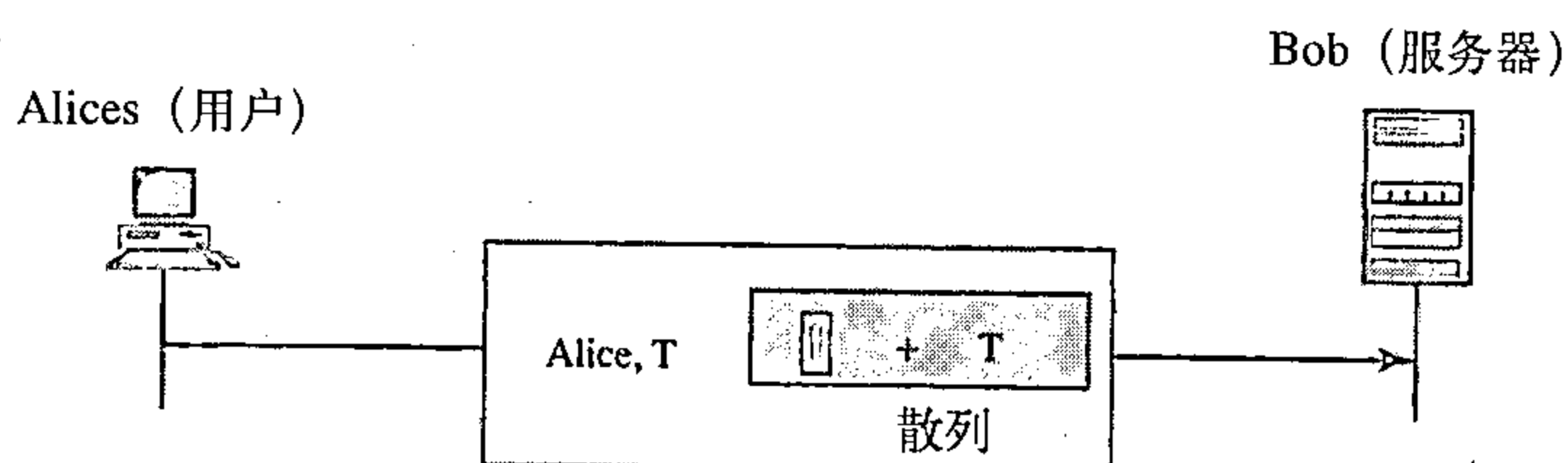


图31.16 使用密钥散列函数的询问-应答鉴别

注意在这种情况下，时间戳同时作为明文和用密钥散列函数杂凑的文本发送。当Bob接收到该报文时，他提取出明文T，对它使用密钥散列函数，然后将计算结果与接收到的散列比较来确定Alices的真实性。

使用非对称密钥加密

不用对称密钥加密，我们使用非对称密钥加密对实体进行鉴别。此时的密码必需是申请者的私钥，申请者要向每个人公开与她自己私钥有关的可用的公钥。这就是说，验证者用申请者的公钥加密询问，然后申请者用她的私钥解密。对询问的应答是解密的询问。我们说明

了两种方法：一种用于单向鉴别，另一种用于双向鉴别。在第一种方法中，Bob用Alices的公钥加密询问，Alices用她的私钥解密报文，并向Bob发送瞬时值。图31.17表示了这种方法。

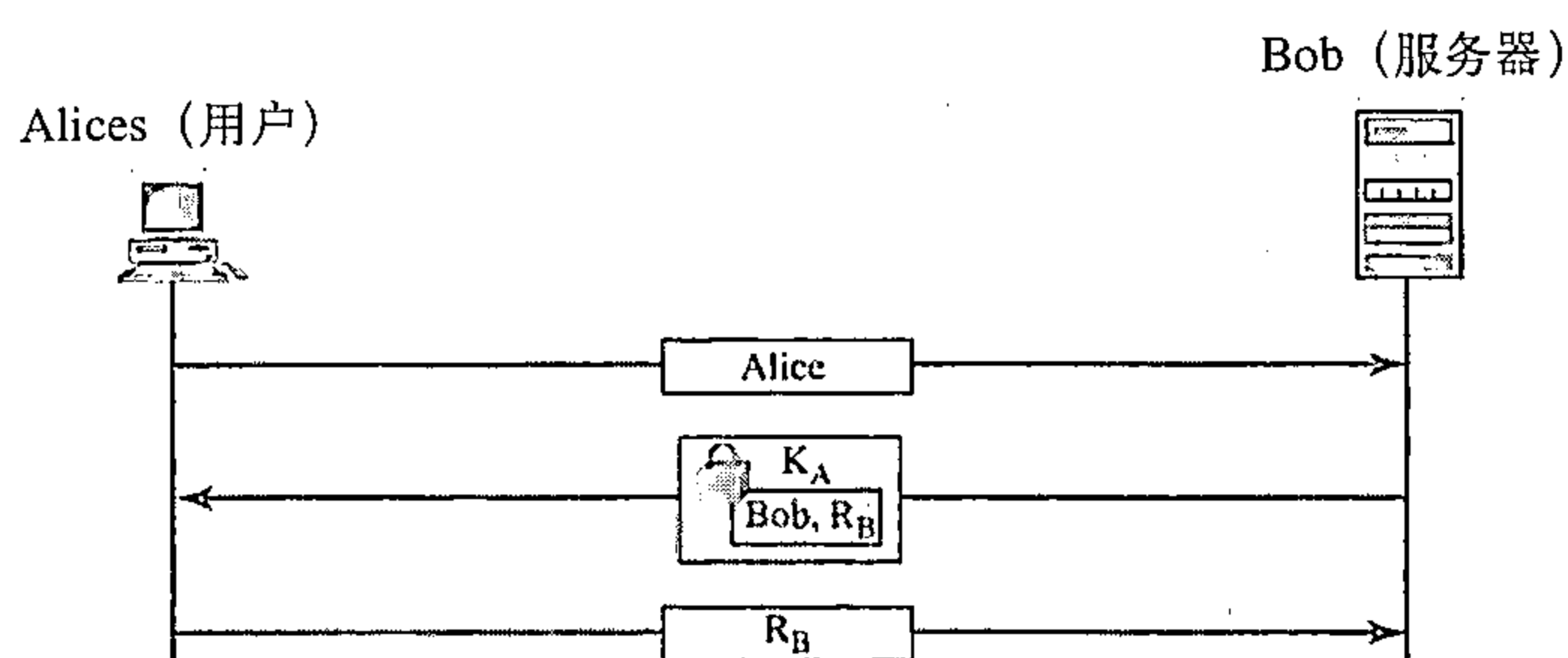


图31.17 使用非对称密钥鉴别

使用数字签名

我们可用数字签名进行实体鉴别。在这种方法中，我们让申请者使用她的私钥签名，而不是用私钥解密。图31.18表示了这种方法，Bob用明文询问，Alices签名应答。

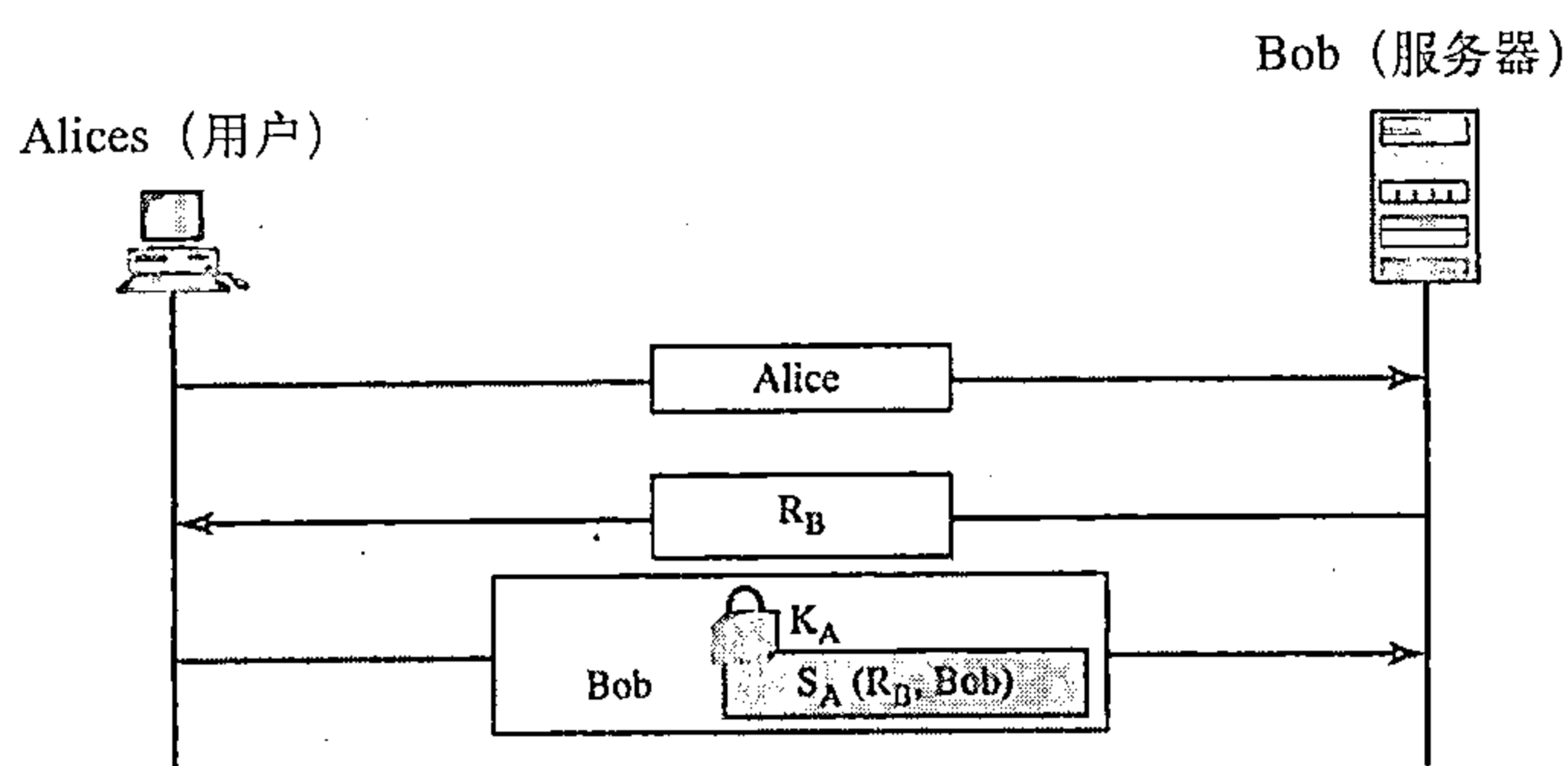


图31.18 用数字签名鉴别

31.7 密钥管理

我们已经在本章讨论了对称密钥密码学和非对称密钥密码学的应用，但未讨论对称密钥密码学中的秘密密钥和非对称密钥密码学中的公钥是如何分发和维护的。本节我们讨论这两个问题，首先讨论对称密钥的分发，然后讨论非对称密钥的分发。

31.7.1 对称密钥的分发

我们已知道当加密与解密大的报文时，对称密钥密码学比非对称密钥密码学更有效。但对称密钥密码学在双方之间需要一个共享的秘密密钥。

如果Alice需要与 N 个人交换保密报文，那么她需要 N 个不同的密钥。如果 N 个人需要彼此通信，要多少个密钥？总的密钥数共计 $N(N-1)/2$ 。因为每个人需要与其他 $N-1$ 个人通信，但由于密钥可共享，所以仅需要 $N(N-1)/2$ 个密钥。这就是说，如果1百万人需要彼此通信，则每个人需要约50万个不同的密钥。因为 N 个实体要求的密钥数近似于 N^2 ，这通常认为是 N^2 问题。

密钥的个数不是唯一存在的问题，还有密钥的分发问题。如果Alice与Bob想要通信，则他们需要以某种方法交换密钥。如果Alice要与1百万个人通信，她如何与1百万个人交换1百万个密钥？使用因特网没有一个明确的安全方法。

显然，我们需要一个维护与分发密钥的有效方法。

密钥分发中心：KDC

一个实际的解决方案是使用一个称为密钥分发中心（key distribution center, KDC）的可信任方。为了减少密钥的个数，每个人建立一个带有KDC的共享密钥，如图31.19所示。

在KDC与每一个成员之间建立一个私钥。Alice与KDC有一个私钥，我们称为 K_{Alice} 。Bob与KDC有一个私钥，称为 K_{Bob} 等等。现在的问题Alice如何向Bob发送保密的报文？其过程如下：

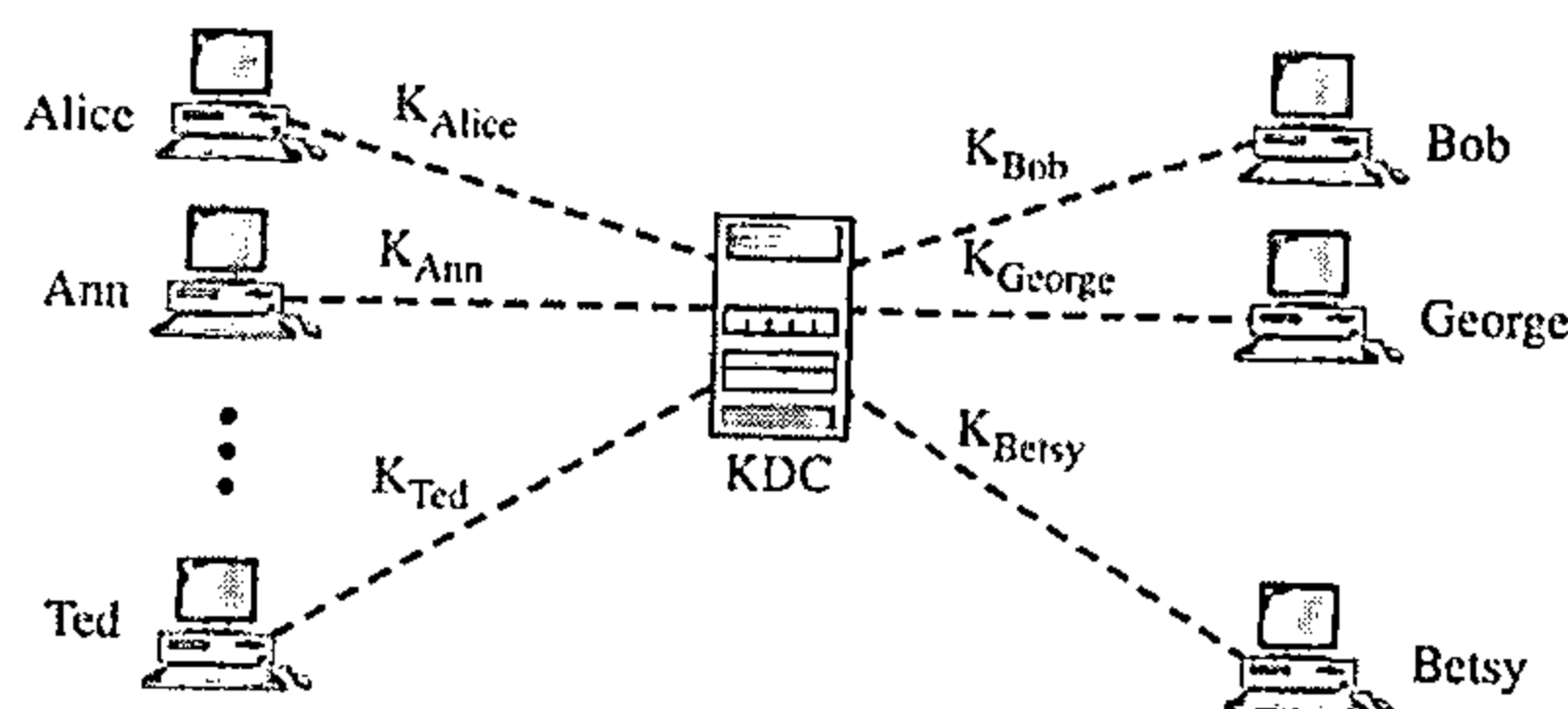


图31.19 KDC

1. Alice向KDC发送一个请求，表明她需要和Bob之间的会话（暂时的）密钥；

2. KDC向Bob通告Alice的请求；

3. 如果Bob同意，则在两者之间创建一个会话密钥。

用KDC在Alice与Bob之间创建的密钥用于KDC鉴别Alice和Bob，并防止Eve假装他们中的任一个。在本章的后面，我们将讨论在Alice和Bob之间创建一个会话密钥。

会话密钥

KDC为每个成员创建一个秘密密钥，该秘密密钥仅用于成员与KDC之间，而不是用在两个成员之间。如果Alice需要与Bob保密通信，她需要在她自己与Bob之间的一个秘密密钥。

双方之间的对称会话密钥仅使用一次。

使用前面讨论过的实体鉴别思想可以提出多种不同的方法创建会话密钥。

让我们讨论最简单的一种，如图31.20所示。尽管这个系统有些缺点，但它表明了思想实质。更复杂的方法可在一些安全书籍中找到。

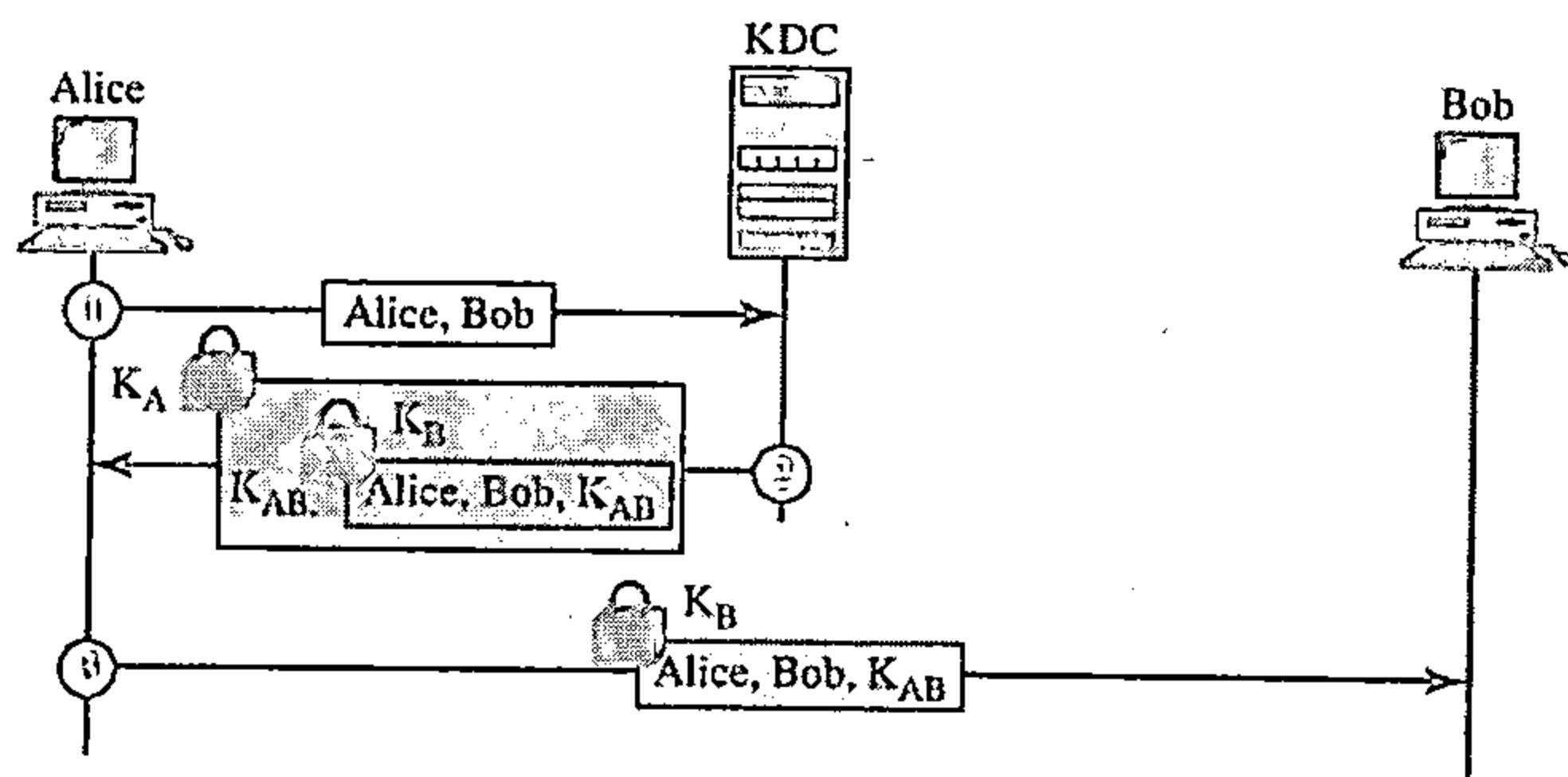


图31.20 使用KDC在Alice与Bob之间创建会话密钥

- 步骤1。Alice向KDC发送明文报文以获得她和Bob之间的对称会话密钥。报文包含了她的注册身份（图中的Alice）和Bob的身份（图中的Bob）。该报文没有加密，是公开的。KDC并不关心这些。
- 步骤2。KDC接收到报文，生成了所谓的票证（ticket）。该票证是用Bob的密钥（ K_B ）加密的。票证包含了Alice和Bob的身份，以及会话密钥（ K_{AB} ）。带有会话密钥副本的票证发送给Alice。Alice接收到该报文，对它进行解密，得到会话密钥。她无法解密Bob的票证，因为票证是给Bob的，不是给Alice的。注意：在该报文中采用了两次加密，票证是加密的，整个报文也是加密的。在第二个报文中，Alice被KDC鉴别，因为只有Alice能用她的带有KDC的密钥打开整个报文。

- 步骤3。Alice将票证发送给Bob。Bob打开票证，知道Alice需要使用 K_{AB} 作为会话密钥来向他发送报文。注意：在这个报文中，Bob被KDC鉴别，因为只有Bob能打开票证。因为Bob被KDC鉴别，他也被信任KDC的Alice鉴别。同样的方法，Alice也能被Bob鉴别，因为Bob信任KDC，KDC向Bob发送票证，该票证包括了Alice的身份。

Kerberos

Kerberos是一个鉴别协议，同时又是一个KDC，它已经变得十分流行。包括Windows 2000在内的几个操作系统都使用Kerberos。Kerberos是根据希腊神话中为冥王哈得斯守门的三头狗命名的。这个协议最初麻省理工学院设计出来的，目前已经经历了好几个版本。这里只讨论最常用的第4版。

服务器 Kerberos协议中包括三个服务器：一台鉴别服务器（AS），一台票证授予服务器（TGS）和一台用来向他人提供服务的实际的（数据）服务器。在我们所给出的一些实例及其插图中，Bob是实际的服务器，Alice是一个请求服务的用户。图31.21显示了这三台服务器之间的关系。

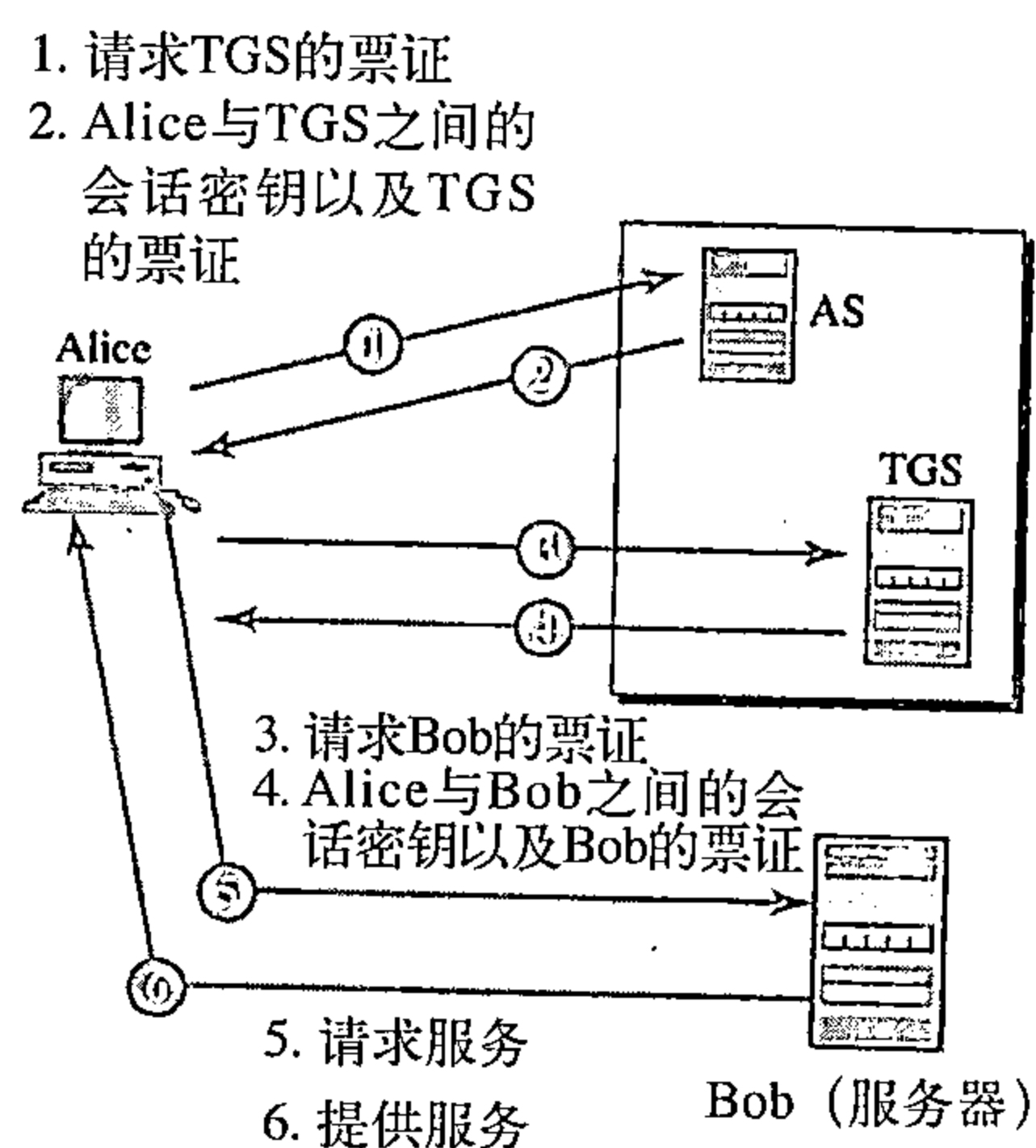


图31.21 Kerberos服务器

- 鉴别服务器（AS）是Kerberos协议中的KDC。每个用户在AS中进行注册，并被分配一个用户身份和口令。AS拥有一个包含这些用户身份和相应口令的数据库。AS验证用户，发布Alice和TGS之间使用的会话密钥，并发送一张给TGS使用的票证。
- 票证授予服务器（TGS）为实际服务器（Bob）发布票证。它还提供Alice和Bob之间的会话密钥（ K_{AB} ）。Kerberos将用户验证与票证发布分离开来。通过这种方式，尽管Alice只与AS验证了一次她的ID，但是她可以与TGS联系多次以获得不同的实际服务器的票证。
- 实际服务器（Bob）为用户（Alice）提供服务。Kerberos设计成一个诸如FTP的客户/服务器的程序，用户可以采用客户端进程访问服务器进程。Kerberos不是用来进行人对人的鉴别的。

操作 客户端进程（Alice）可以通过六个步骤，访问运行在实际服务器上的进程，如图31.22所示。

- 步骤1。Alice用明文向AS发送她的请求，使用的是她注册的身份。
- 步骤2。AS发送一份用Alice的对称密钥 K_A 加密过的报文。该报文包括两项：一份Alice用来与TGS联系时所使用的会话密钥 K_S 和一张TGS的票证。该票证是用TGS的对称密钥 K_{TG} 加密的。Alice不知道 K_A ，但是当报文到达时，她输入她的口令。如果口令正确，口令和相应的算法一起会生成 K_A ，口令立即被销毁，它不发送给网络，也不驻留在终端，它只是暂时用来生成 K_A 。现在，进程使用 K_A 来解密发送的报文，从而得到 K_S 和票证。
- 步骤3。现在，Alice向TGS发送三项内容。第一项是从AS接收的票证，第二项是实际服务器（Bob）的名称，第三项是采用 K_S 加密的时间戳。时间戳用来制止Eve的重放攻击。
- 步骤4。现在，TGS发送两张票证，每张都包括Alice和Bob之间的会话密钥 K_{AB} 。给Alice的票证采用 K_S 加密，给Bob的票证采用Bob的密钥 K_B 进行加密。注意：Eve无法得到 K_{AB} ，因为她不知道 K_S 或 K_B 。她不能重放步骤3，因为她无法用新的时间戳（她不知道 K_S ）来

替换真正的时间戳。即使她很快，即在时间戳超时之前发送了步骤3的报文，她得到的仍然是相同的她无法解密的两张票证。

- 步骤5。Alice发送Bob的带有用 K_{AB} 加密的时间戳的票证。
- 步骤6。Bob通过将时间戳加1来确认接收。报文用 K_{AB} 加密后发送给Alice。

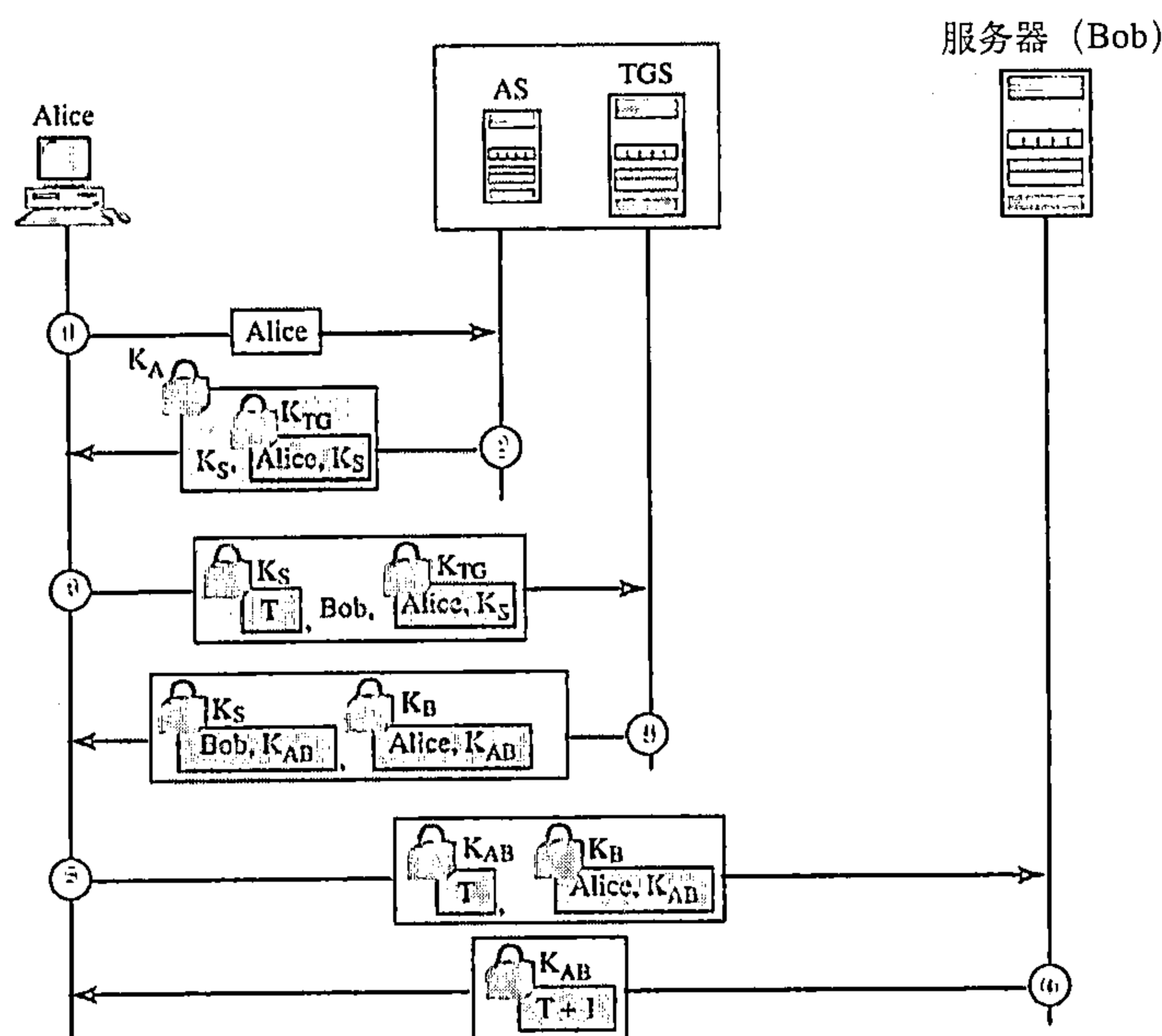


图31.22 Kerberos实例

使用不同的服务器 注意：如果Alice需要从不同的服务器接收服务，那么她只需要重复最后的一步（步骤3～步骤6）。最初的两步已经验证了Alice的身份，不需要重复。Alice通过重复步骤3～步骤6，就可以让TGS发布多个服务器的票证。

领域 Kerberos允许AS和TGS的全球发布，每个系统称为一个领域（realm）。用户可以通过获得本地服务器或远程服务器的票证。例如，在第二种情况下，Alice可能要求她的本地TGS发行一张由远程TGS接收的票证。如果该远程TGS在本地TGS中进行了注册，那么本地TGS可以发布这张票证，Alice从而可以使用该远程TGS来访问远程的实际服务器。

31.7.2 公钥分发

在非对称密钥密码学中，人们不需要知道对称共享密钥。如果Alice想给Bob发送报文，她只需要知道Bob的公钥，该公钥是公开的，任何人都能得到。如果Bob想给Alice发送报文，他只需要知道Alice的公钥，她的公钥也是任何人都知道的。在公钥加密中，每个人都隐藏私钥，并将公钥公开发布。

在公钥密码学中，任何人都可以访问其他人的公钥，公钥是公开发布的。

公钥像秘密密钥一样为应用需要而分发，让我们简要讨论公钥分发的方法。

公钥发布

最简单的方法是公开发布公钥。Bob可把他的公钥放在他的Web网站或在本地的与全国的报纸上公布。当Alices向Bob发送保密报文时，她从Bob的网站或报纸上得到他的公钥，甚至她可以发报文询问。图31.23表示这种情形。

但是，这种方法是不安全的，会遭受到假冒。例如，Eve可将她自己的公钥作为Bob公钥的一个公开的发布，在Bob做出反应前，破坏可能发生。Eve可混淆Alices发给Bob的报文。Eve也可用一个相应的私钥签名一个文档，使每个人相信文档是Bob签名。如果Alices直接向Bob询问公钥，也易受到攻击。Eve可截取Bob的应答，用她的公钥取代Bob的公钥。

可信任中心

一个更安全方法要有一个可信任中心保存公钥的目录，这个目录像电话簿一样是动态更新的。每个用户可选取一个私钥/公钥对，保留私钥并将公钥插入目录。可信任中心要求每个用户在中心注册并检验他或她的身份。目录可由可信任中心公开发布，中心也应答有关公钥的查问。图31.24表示了这个概念。

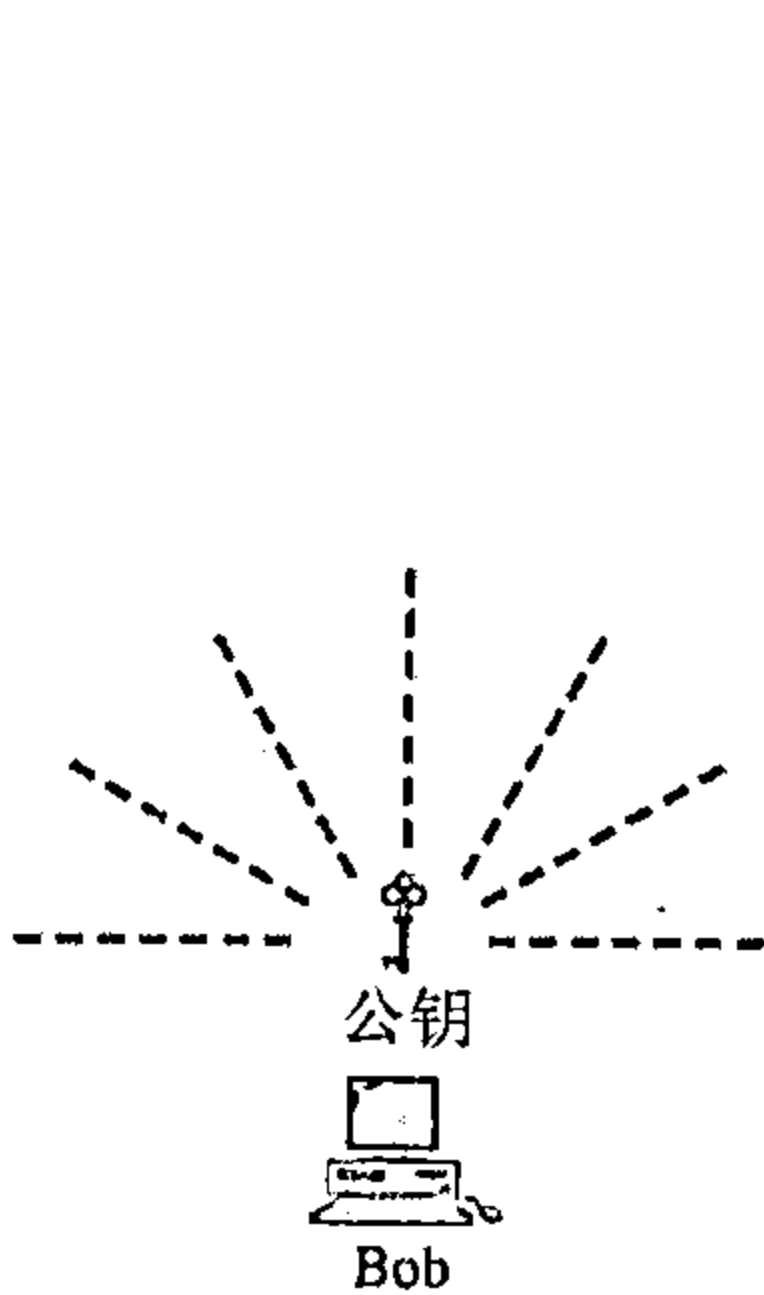


图31.23 发布公钥

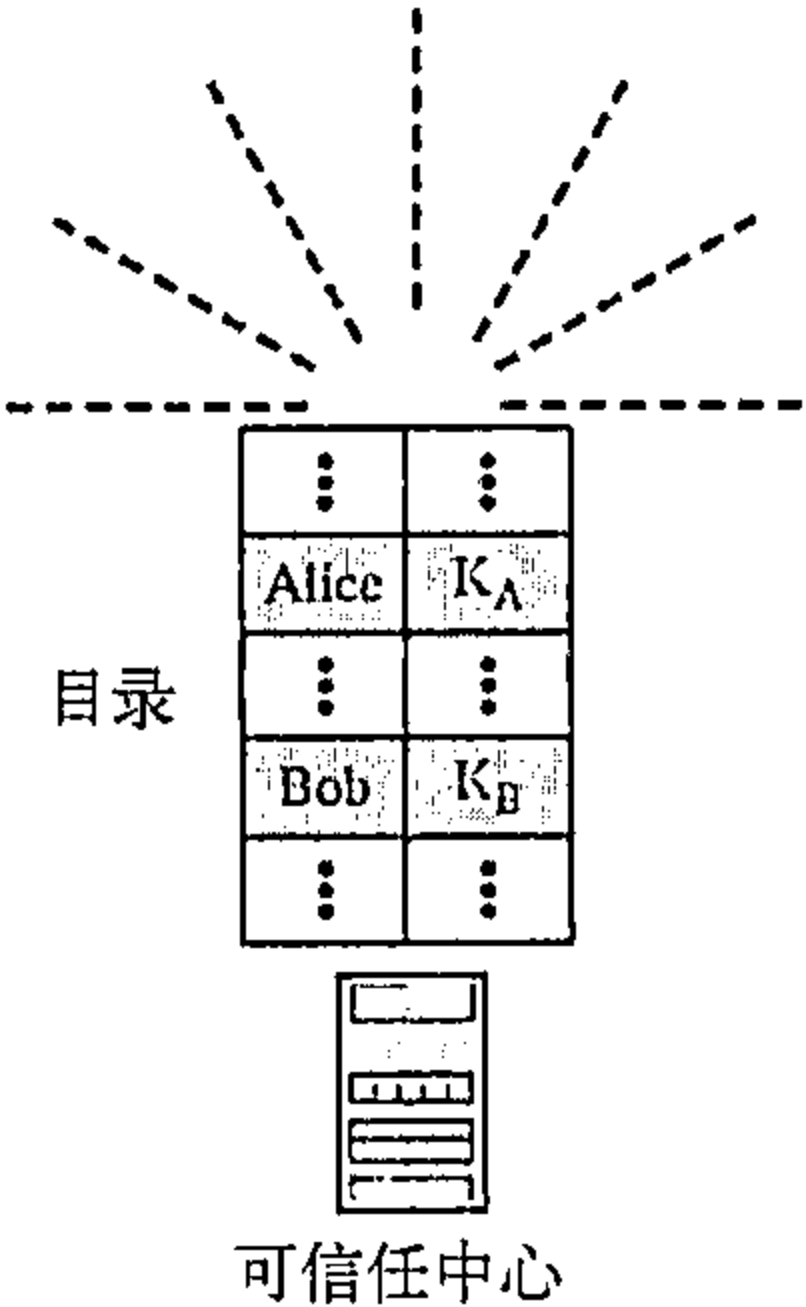


图31.24 可信任中心

受控制的信任中心

如果对公钥的分发增加一些控制，那么可以得到更高级的安全。公钥发布可包括时间戳和权威机构的签名从而可防止应答的截取与修改。如果Alices需要知道Bob的公钥，她可以向中心发送一个包括Bob名和时间戳的一个请求。中心用Bob公钥应答原有的请求而时间戳用中心的私钥签名。Alices使用大家都知道的中心公钥解密报文并获得Bob公钥。图31.25表示了这种情况。

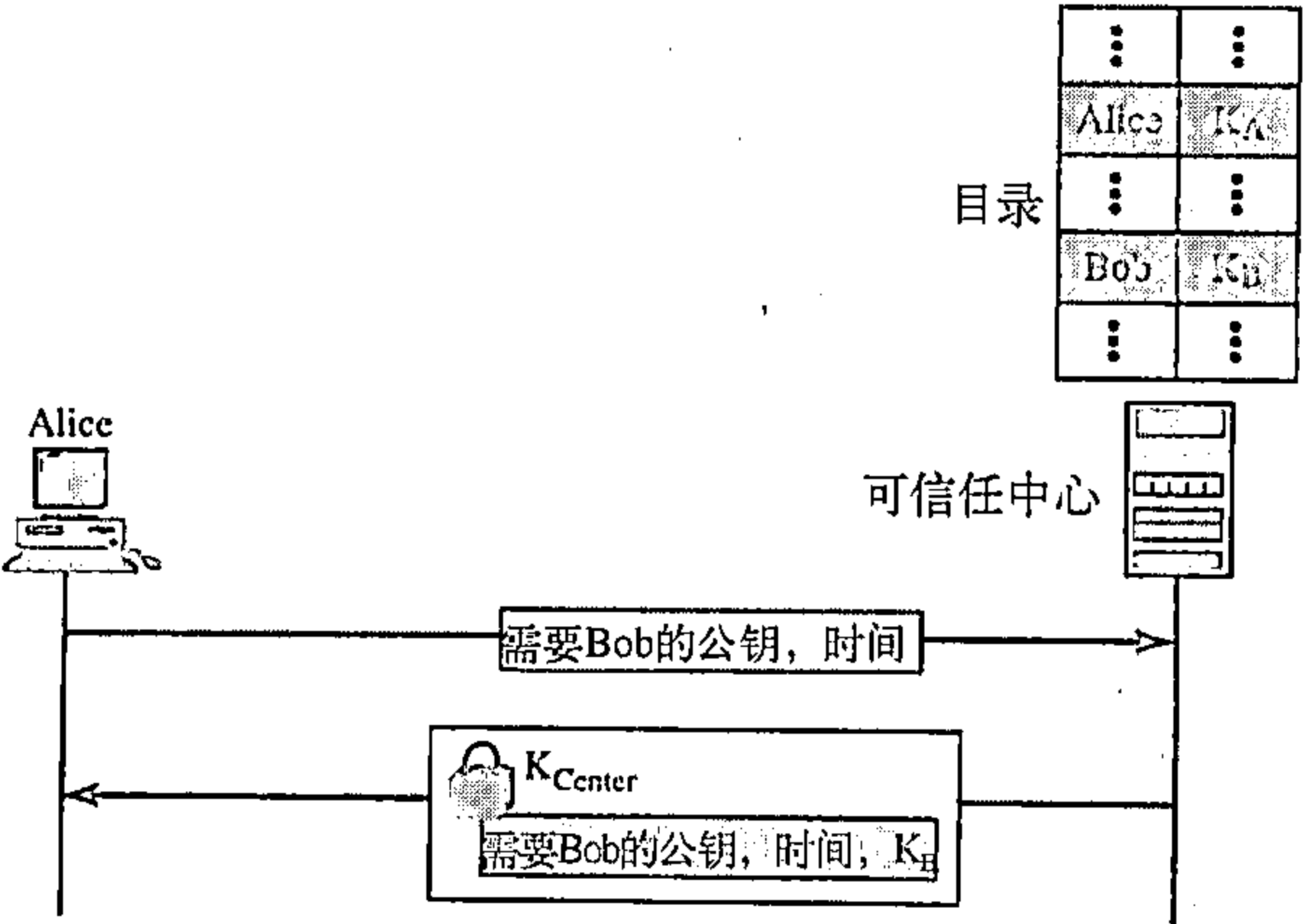


图31.25 受控制的信任中心

认证中心

如果请求的个数很多，前述的方法在中心形成沉重的负担，另一种选择创建公钥的认证。

Bob需要做两件事情：他需要人们知道他的公钥，他还需要确保任何人无法获得以Bob名义伪造的公钥。Bob可以去认证中心（certification authority, CA），认证中心是一个联邦或州级组织，用来将公钥与实体绑定，并且发布一份证书。该CA本身有一个众所周知的无法伪造的公钥。CA核实Bob的身份（identification）（采用图片ID和其他证据）。然后向Bob索要公钥并将其写在证书上。为防止证书被伪造，CA根据证书生成一份摘要并用它的私钥对摘要进行加密。现在Bob可以上传明文形式的证书和加密后的报文摘要。任何人只要想得到Bob的公钥，都可以下载证书和加密后的摘要。然后根据证书生成摘要，并使用CA的公钥对加密的摘要进行解密。对两份摘要进行比较，如果它们相等，就说明证书是有效的，没有冒充者假装Bob。图31.26表示这个概念。

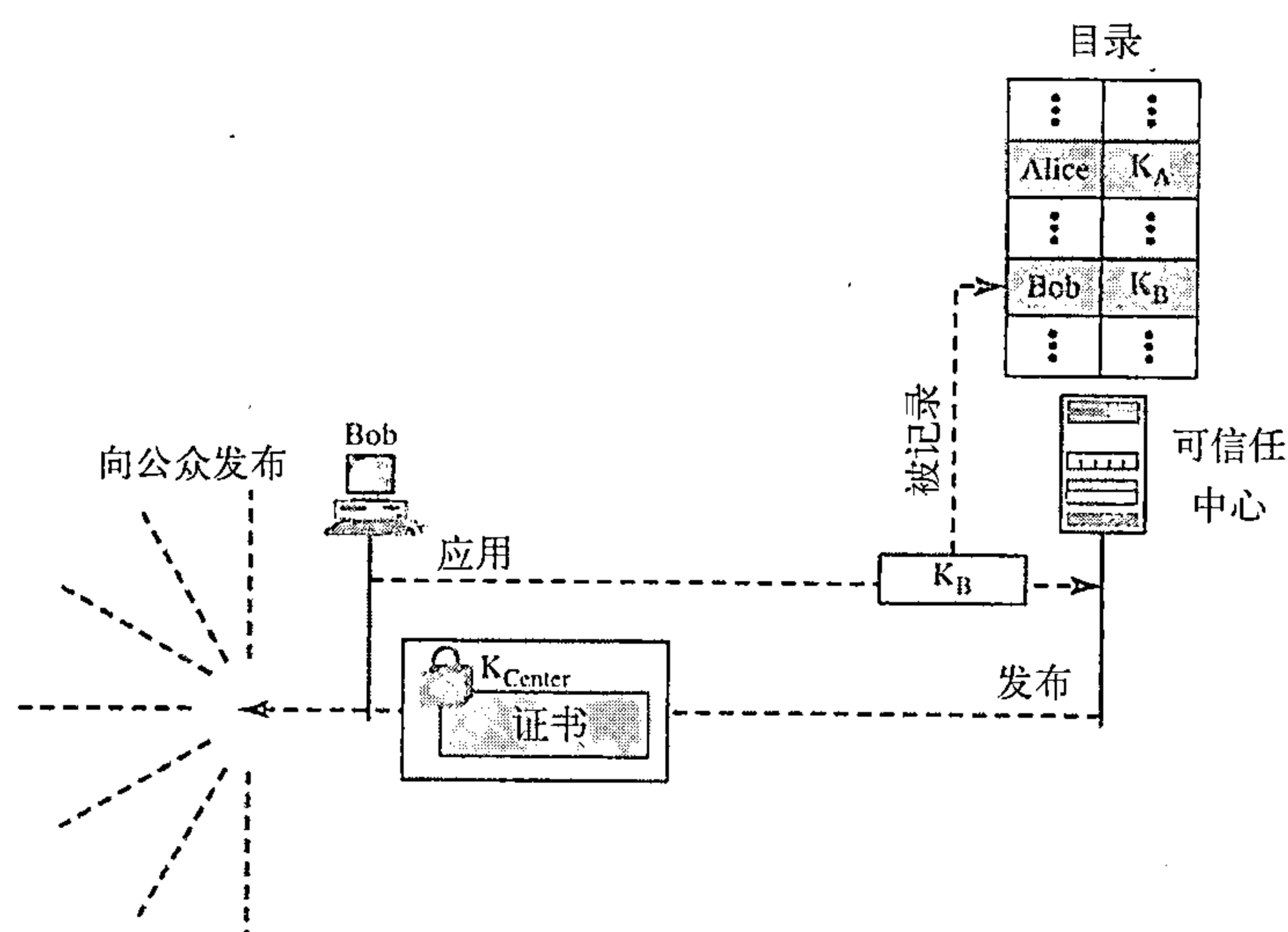


图31.26 认证中心

X.509 尽管采用CA解决了伪造公钥的问题，但是它出现了副作用。每份证书可能有不同的格式。如果Alice想使用程序自动下载属于不同人的不同的证书和摘要，程序可能无法做到这一点。一份证书中的公钥可能是一种格式，另一份可能又是另一种格式。公钥可能在一份证书中位于第一行，在另一份中就有可能位于第三行。任何要普遍使用的事物都必须有统一的格式。

要消除这种副作用，ITU制定了一个称为X.509的协议，该协议经过一些修改已经被因特网所采纳。X.509协议采用结构化方式描述证书的规范。它使用一个熟知的称为ASN.1的协议来定义C程序员非常熟悉的字段。

- **版本**。这个字段定义认证X.509的版本号。版本号从0开始，现在的版本号是2（第3版本）。
- **系列号**。这个字段定义赋给每个证书的一个号码，其值对每个发出的证书是唯一的。
- **签名**。这个字段标识签名证书所用的算法，名字是不适当的。为这个签名所需的参数也在这个字段定义。
- **签发者**。这个字段定义发布证书的认证中心的名称。该名称通常是一个由国家、州、组织机构和部门等定义的有层次的串。
- **有效期**。这个字段定义证书有效期的起始和结束时间。
- **主体**。这个字段定义公钥所有者的实体，它也是一个层次串。字段部分定义了称为公用名的字段，它是密钥的观看者真实名。
- **主体的公钥**。这个字段定义主体的公钥，证书的核心。这个字段也定义主体的算法（例如，RSA）和它的参数。

- 签发者唯一的标识符。这是一个可选字段，它允许两个发布者具有相同的发布者字段的值，如果发布者唯一的标识符是不同的。
- 主体唯一的标识符。这个可选字段允许两个主体具有相同的主体字段的值，如果主体唯一的标识符是不同的。
- 扩展项。这个字段允许发布者为证书增加更多的私有信息。
- 加密。这个字段包含算法标识符、其他字段的安全散列和该散列的数字签名。

公钥基础设施 (PKI)

当人们想统一地使用公钥时，会遇到一个类似于密钥分发的问题。人们发现不能仅有一个KDC来应答请求，而是需要很多服务器。另外，人们发现最佳方案是将服务器配置为层次关系。类似地，公钥查询的一种解决方案是采用称为公钥基础设施 (public key infrastructure, PKI) 的层次化结构。图31.27表示了这种层次结构的例子。

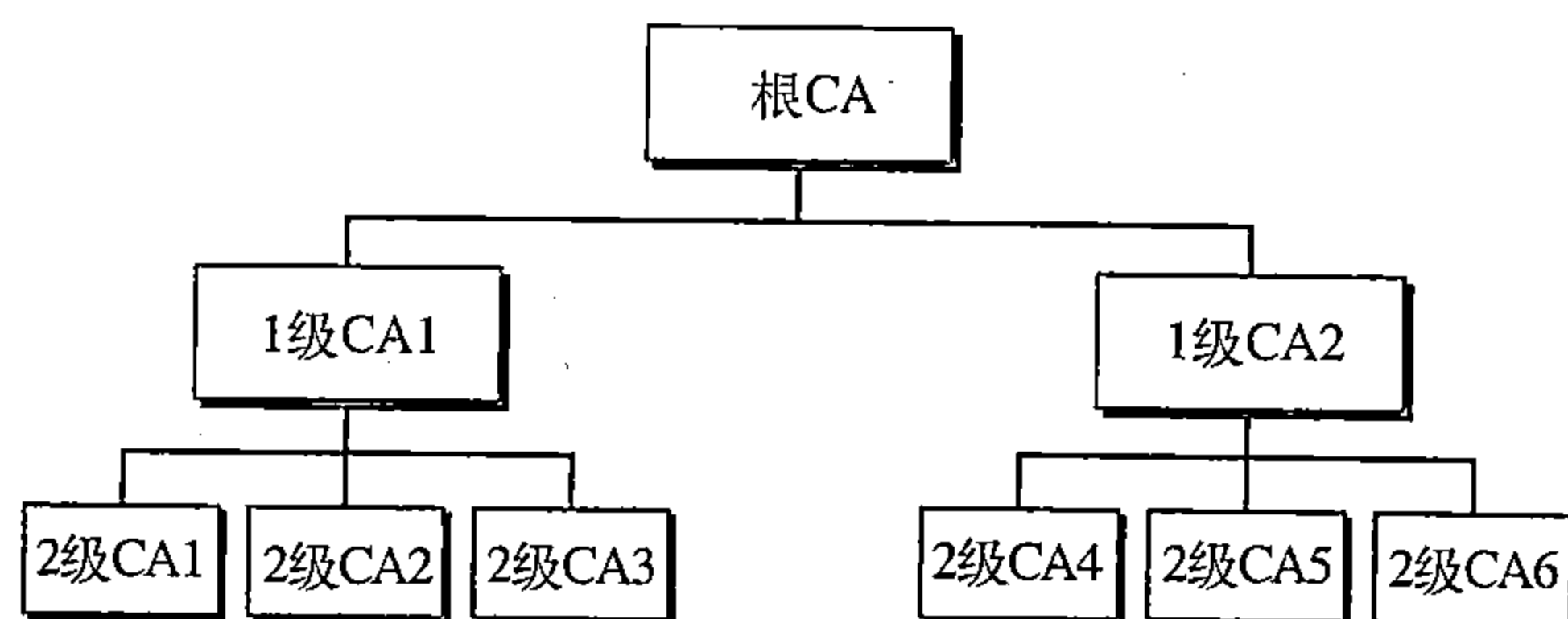


图31.27 PKI层次结构

在第一级中，拥有一个根CA来验证第二级CA；这些1级的CA可以在一个很大的地理区域或逻辑区域的范围内运作。2级的CA可以在一个相对较小的地理区域内运作。

在这个层次结构中，每个人都信任根节点。但人们可以信任也可以不信任中间的CA。如果Alice需要得到Bob的证书，她可以找发布该证书的某个地方的CA，但Alice不一定信任该CA。在层次结构中，Alice可以通过上一级CA来验证原始的CA。该查询可以一直进行到根CA。

推荐读物

为了深入讨论与本章有关的主题，推荐阅读下列读物，在本书末列出方括号[···]中的参考资料。

读物

有关网络安全的专著如[PHS02]、[Bis03]和[Sal03]。

本章小结

- 密码技术提供五种服务，其中四种是关于Alices与Bob之间的报文交换，第五种是关于试图访问系统并使用系统资源的实体的。
- 报文的保密性是指发送方与接收方期待保密。
- 报文的完整性是指数据必须正确地到达接收方。
- 报文的鉴别是指接收方确信报文来自假定的发送方而不是冒名顶替者。
- 不可否认性是指发送方不能否认已发送的数据。
- 实体鉴别是指证明试图访问系统资源的实体的身份。
- 报文摘要用于保护文档或报文的完整性，散列函数从报文创建报文摘要。

- 散列函数必须满足三个条件：单向性、抗弱碰撞性与抗强碰撞性。
- 无密钥报文摘要用作修改检测码 (MDC)，它保证报文完整性。要鉴别数据源，需要报文鉴别码 (MAC)。
- MAC是密钥散列函数，它用报文和密钥创建压缩的摘要。其方法与加密算法的基本原理相同。
- 数字签名方案提供了与手书签名相同的服务。手书签名包含在文档中，而数字签名是独立的实体。
- 数字签名提供报文完整性、鉴别和不可否认性。但数字签名不能提供报文保密性，如果需要保密性，必须在方案上应用加密系统。
- 数字签名需要一个非对称密钥系统。
- 在实体鉴别中，申请者用三种基本途径：所知的物、所有的物和固有特征向验证者证明自己的身份。
- 在口令鉴别中，申请者用字符串作为所知的物。
- 口令鉴别可划分两大类：固定口令与一次性口令。
- 在询问-应答鉴别中，申请者证明她知道一个密钥，但并没有实际发送它。
- 询问-应答鉴别可划分为四大类：对称密钥加密、密钥散列函数、非对称密钥加密和数字签名。
- 密钥分发中心 (KDC) 是可信任的第三方，它向双方分配对称密钥。
- KDC仅在成员与中心之间创建密钥。当两个成员联系KDC时，成员之间的密钥需要作为会话密钥创建。
- Kerberos是一个流行的会话密钥创建协议，它需要一个鉴别服务器和一个票证服务器。
- 公钥基础设施 (PKI) 是一个应答有关密钥认证询问的层次结构的系统。

习题

复习题

1. 瞬时值是什么？
2. N^2 问题是什么？
3. 列出一个使用 KDC 作为用户鉴别协议的名称？
4. Kerberos鉴别服务器的目的是什么？
5. Kerberos票证服务器的目的是什么？
6. X.509目的是什么？
7. 什么是认证中心？
8. 使用长的口令有哪些优点与缺点？
9. 讨论作为二种极端的固定和一次性的口令，口令改变频率是多少？你认为这个方案如何实现？优点与缺点是什么？
10. 一个系统如何防止口令猜测攻击？如果已发现银行卡被偷窃并试图使用它，银行能否防止PIN猜测？

练习题

11. 一个报文由00到99之中的10个数字组成，一个散列算法是这些数相加的和模100运算从这个报文中创建一个摘要，产生的摘要是00到99中的一个数。这个算法是否满足散列算法第一个条件？是否满足第二个条件？是否满足第三个条件？

12. 一个报文是由100个字符组成，散列算法是挑选1, 11, 21, ……和91字符创建该报文的摘要。其结果的摘要要有10个字符。这个算法是否满足散列算法第一个条件？是否满足第二个条件？是否满足第三个条件？
13. 一个散列算法创建 N 位摘要，这个算法能够有多少个不同的摘要？
14. 在一方，一个人的生日是特定的一天或两个（多个）人有相同生日，哪一个可能性更大？
15. 如何解决习题14有关散列函数第二个与第三个条件？
16. 固定长度的摘要和可变长度的摘要，哪一个更切实可行？试说明其理由。
17. 一个报文是20 000个字符，使用SHA-1创建这个报文的摘要，然后决定改变为最后10个字符。是否可说摘要中有多少位将被改变？
18. 创建一个MAC过程与签名一个散列是否相同？
19. 某人使用自动出纳机提取现金，这是报文鉴别还是实体鉴别，还是两者兼有？
20. 将图31.14改为提供双向鉴别（Alices鉴别Bob与Bob鉴别Alices）。
21. 将图31.16改为提供双向鉴别（Alices鉴别Bob与Bob鉴别Alices）。
22. 将图31.17改为提供双向鉴别（Alices鉴别Bob与Bob鉴别Alices）。
23. 将图31.18改为提供双向鉴别（Alices鉴别Bob与Bob鉴别Alices）。
24. 在大学中，学生在登录发送口令前需要加密口令，这加密是保护学校还是保护学生，试说明理由。
25. 在习题24中，如果在加密之前，学生附加时间戳，这有帮助吗？试说明理由。
26. 在习题24中，如果学生有一个口令表并每次使用一个不同口令，这有帮助吗？试说明理由。
27. 在图31.20中，如果KDC发生故障，将会发生什么？
28. 在图31.21中，如果AS发生故障，将会发生什么？如果TGS发生故障，将会发生什么？如果主服务器发生故障，将会发生什么？
29. 在图31.26中，如果可信任中心发生故障，将会发生什么？
30. 为了提供保密性，试在图31.11中增加对称密钥加密/解密。
31. 为了提供保密性，试在图31.11中增加非对称密钥加密/解密。

探究题

32. 有一个MD5散列算法，找出这个算法与SHA-1的不同处。
33. 有一个RIPEMD-160散列算法，找出这个算法与SHA-1的不同处。
34. 比较MDS, SHA-1和RIPEMD-160。
35. 讨论有关RSA数字签名一些信息。
36. 讨论有关DSS数字签名一些信息。

第32章 因特网中的安全措施

在本章中，我们要说明某些安全问题，特别是保密性和报文鉴别是如何应用于因特网模型的网络层、传输层和应用层的。我们简要说明了IPSec协议如何在IP协议中增加鉴别和安全、和SSL（或TLS）如何针对TCP协议完成相同的任务以及PGP如何针对SMTP协议（电子邮件）完成相同的任务。

在所有这些协议中，我们需要考虑一些共同的问题。首先，需要生成一个MAC；其次要加密报文，或许要加密MAC。这就是说，尽管有某些细小的差异，但本章所讨论的三个协议都会从适当的层得到一个分组并创建一个加密的新分组。如图32.1所示。

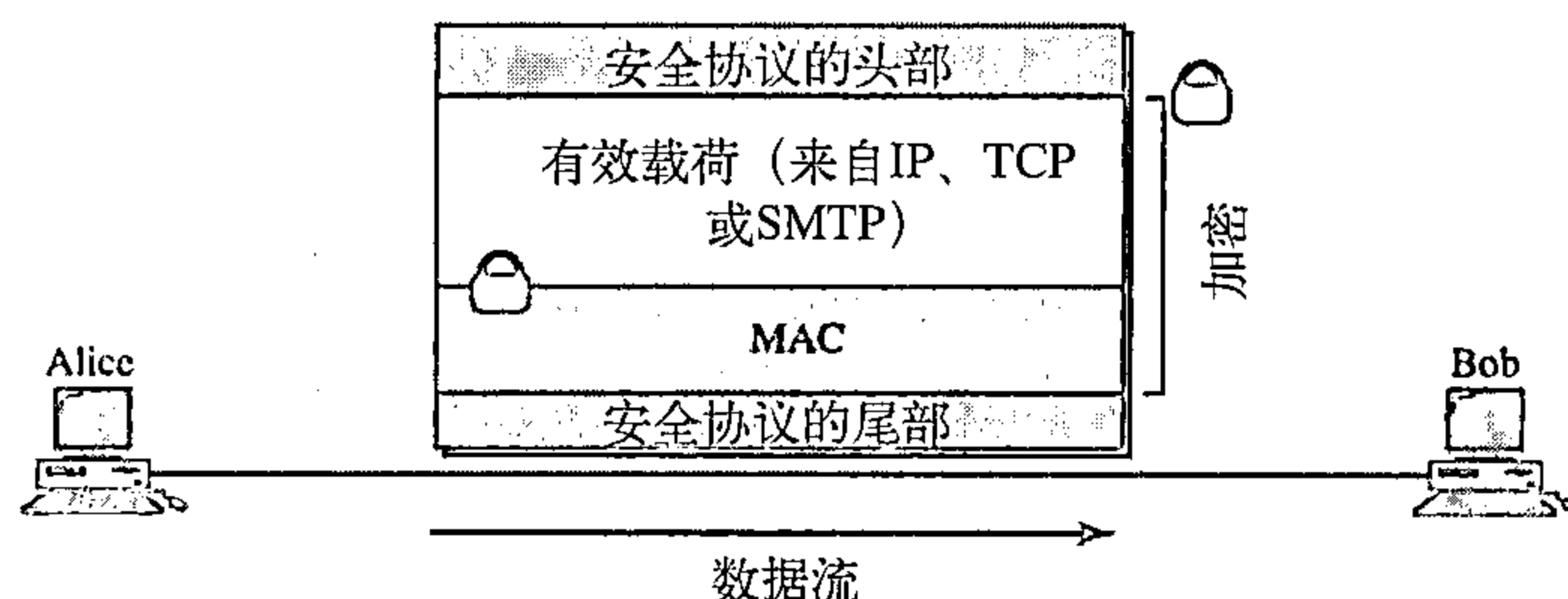


图32.1 三个安全协议的共同结构

注意：在加密过程中，可以包括也可以不包括安全协议的头部和尾部。还应注意有些协议还需要在被加密的分组中有更多的信息。图32.1仅表示了一般的思想。

在所有这些协议中还有一个共同的问题是安全参数。即使在图32.1的简化结构中也间接地表明Alice与Bob彼此在发送加密的分组之前，需要知道有关一些安全参数的信息。他们尤其需要知道用于鉴别和加密/解密分组的算法。即使这些算法为全球用户预先设计的，正如我们所见，他们是不知道的。Bob与Alice至少还需要两个密钥：一个是MAC的而另一个是加密/解密的。换句话说，这些协议的复杂性不存在于计算MAC数据和执行加密的方法。事实上，它存在于计算MAC数据和实施加密之前，我们需要在Alice与Bob之间创建一组安全参数。

先粗略看一下，这些协议中的任一个都好像必须有无限多的步骤。因为要发送安全的数据，必须要用一组安全参数，而安全参数的安全交换又需要另一个第二组安全参数，而第二组安全参数的交换又需要第三组安全参数，等等，直至无穷。

为了减少步骤，我们可使用公开密钥加密方法。如果每一方都有一对私钥和公钥，则步骤的数可减到一步或二步。在一步情况下，我们可用会话密钥生成该MAC并加密数据和MAC。会话密钥和算法列表可用分组发送，但用公开密钥加密算法加密。在二步的情况下，首先用公开密钥加密算法建立安全参数。然后用安全参数去安全地发送真正的数据。这三个协议中，PGP使用第一种方法，而其余两个IPSec和SSL/TLS使用第二种方法。

32.1 IP层的安全：IPSec

IP层安全（IP Security, IPSec）是由因特网工程任务组（IETF）设计的用来为IP层的分组提供安全的一组协议。IPSec帮助生成经过鉴别的与安全的IP层的分组，如图32.2所示。

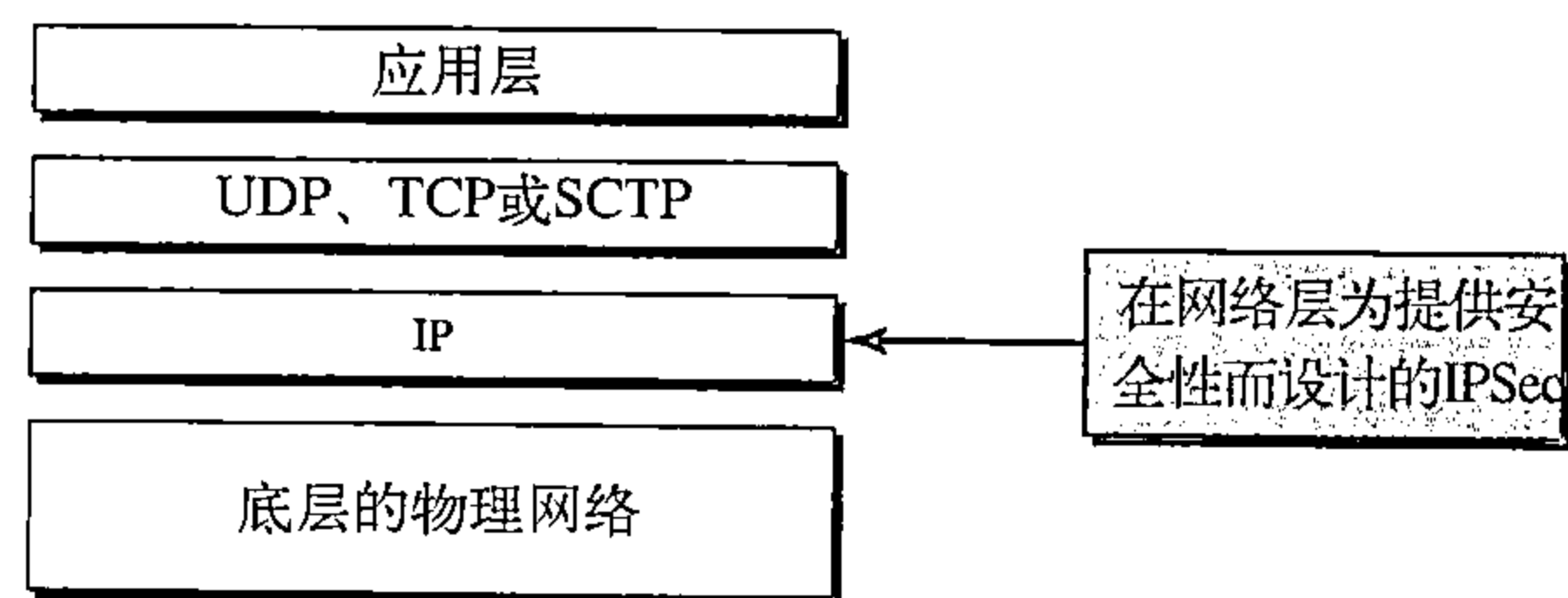


图32.2 TCP/IP协议族与IPSec

32.1.1 两种方式

IPSec以两种不同的方式运行：传输方式和隧道方式，如图32.3所示。

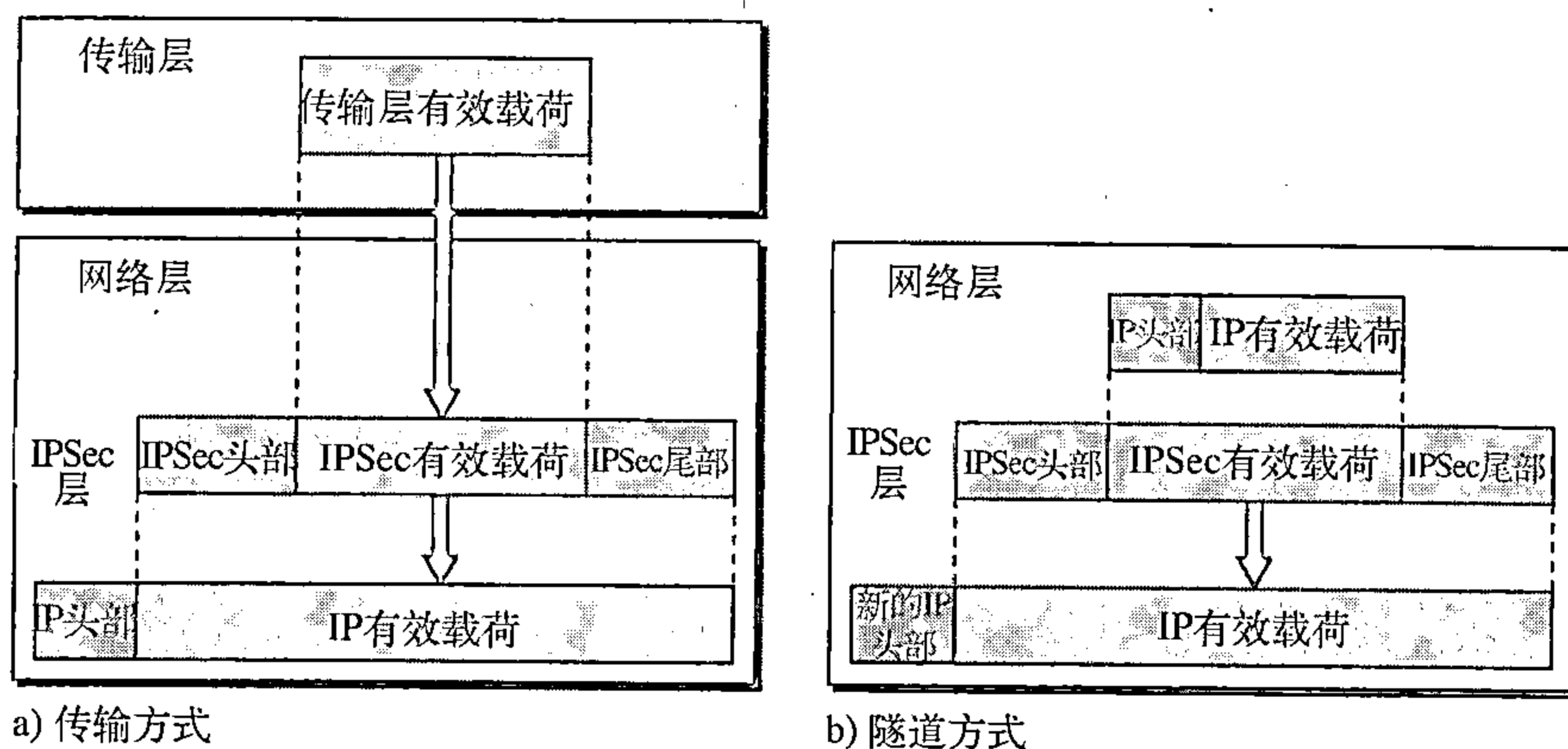


图32.3 IPSec协议的传输方式和隧道方式

传输方式

在传输方式（transport mode）下，IPSec保护传输层到网络层传递的内容。换言之，传输方式保护网络层的有效载荷，在网络层中封装有效载荷。

注意：传输方式不保护IP头部。换言之，传输方式不保护整个IP分组；而仅保护来自传输层的分组（IP层有效载荷）。在这种方式下，对来自传输层的信息增加IPSec的头部与尾部。然后再增加IP头部。

在传输方式下，IPSec不保护IP头部，仅保护来自传输层的信息。

传输方式通常用于主机到主机（端到端）的数据保护。发送主机使用IPSec鉴别和/或加密来自传输层的有效载荷。接收主机使用IPSec检验鉴别和/或解密IP分组并将它传递给传输层。图32.4说明了这个概念。

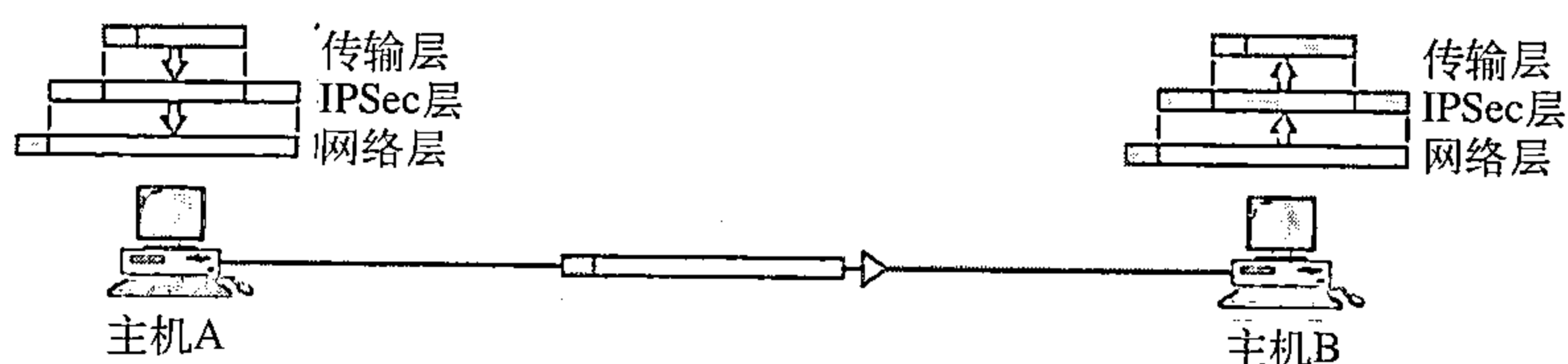


图32.4 传输方式

隧道方式

在隧道方式（tunnel mode）下，IPSec保护整个IP分组，包括头部。将IPSec加密方法用

于整个分组，然后增加一个新的头部，如图32.5所示。

不久将会看到，这个新的IP头部与原来的IP头部有一些不同的信息。隧道方式通常用于两个路由器之间，或一个主机与一个路由器之间以及一个路由器与一个主机之间。如图32.5所示。

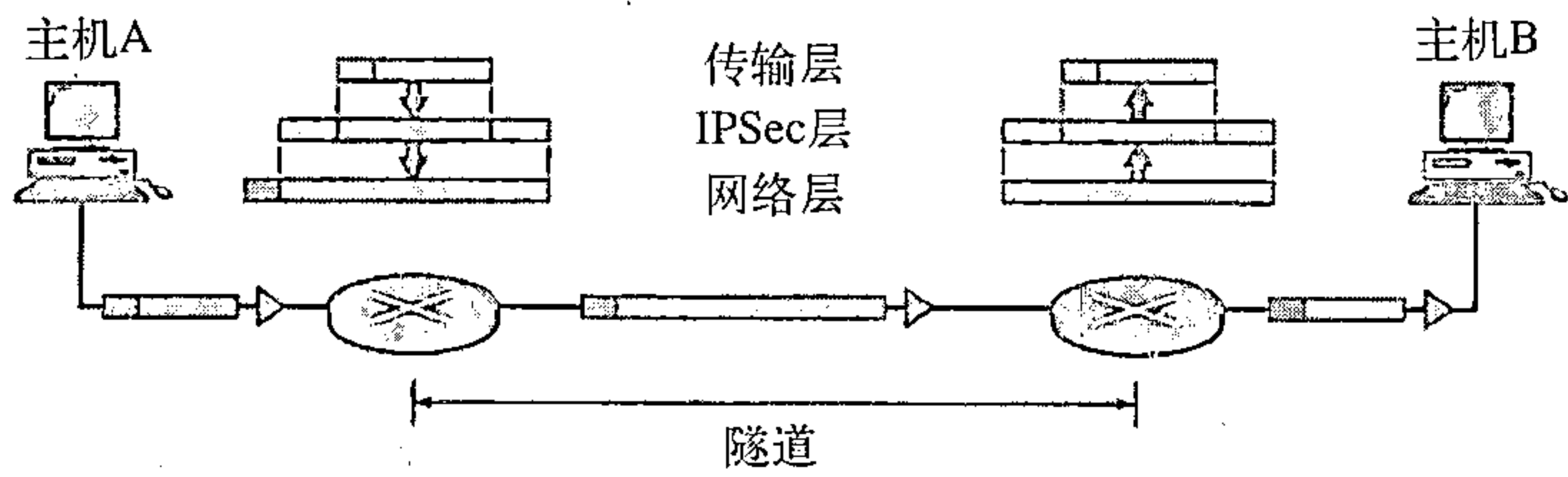


图32.5 隧道方式

换言之，当发送方或接收方不是主机时，我们使用隧道方式。对来自发送方与接收方之间的侵入提供整个原始分组的保护。它好像整个分组通过假想的隧道。

在隧道方式下，IPSec保护原始IP头部。

32.1.2 两种安全协议

IPSec定义了两种安全协议：鉴别头部（AH）协议和封装安全载荷（ESP）协议。在IP层提供鉴别和/或加密。

鉴别头部协议（AH）协议

鉴别头部协议（Authentication Header Protocol）用于鉴别源端主机以确保IP分组所携带载荷的完整性。协议使用散列函数和一个对称密钥生成报文摘要，并将摘要插入鉴别头部中。根据其运行方式（传输方式或隧道方式），将AH插入到相应的位置。图32.6表示了传输方式下各个字段以及鉴别头部的位置。

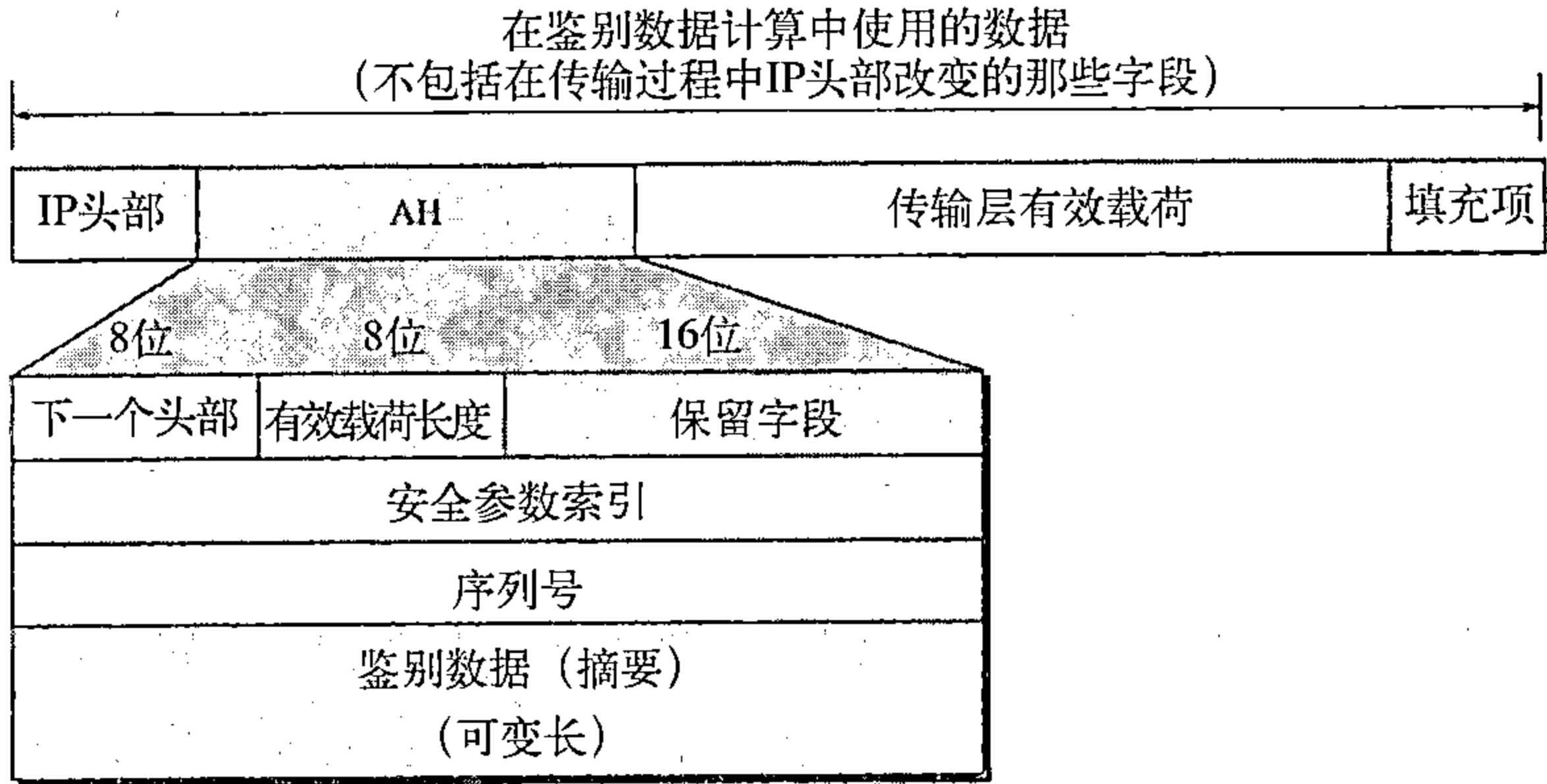


图32.6 传输方式中的鉴别头部（AH）协议

当由IP数据报携带鉴别头部时，IP头部中协议字段的初始值就替换为值51。在鉴别头内部的一个字段（下一个头部字段）规定了协议字段的初始值（由IP数据报携带的有效载荷类型）。加入鉴别头部的步骤如下：

- 1. 鉴别头部加入到有效载荷中，将鉴别数据字段的值置为零。
- 2. 有可能加入填充项以使总长度适合于特定的散列算法。
- 3. 散列处理是根据总的分组计算的。但是，IP头部中只有在传输过程中不发生改变的那